

目 录

第一章 绪论	1
1.1 研究背景与意义	1
1.2 研究目标	3
1.3 研究内容	5
1.3.1 面向大规模行程请求流的持续最优路径组合规划	5
1.3.2 基于轨迹数据的群体联合移动模式挖掘	6
1.3.3 大规模移动对象上的持续密切接触发现	6
1.3.4 面向时空运动范围的轨迹相似性连接方法	7
1.4 研究所面临的挑战	7
1.5 论文的主要贡献	9
1.6 论文的组织结构	11
第二章 相关技术及研究综述	13
2.1 时空数据建模和索引	13
2.2 路径搜索和推荐	17
2.2.1 基于位置的路径规划	18
2.2.2 基于关键词的路径规划	20
2.3 轨迹数据挖掘和应用	22
2.3.1 轨迹相似度连接	22
2.3.2 轨迹模式挖掘	25
2.4 本章小结	26
第三章 全局持续最优路线组合规划	28
3.1 引言	28
3.1.1 研究背景与意义	28
3.1.2 研究内容与挑战	31
3.1.3 研究成果与贡献	32
3.2 问题描述	33
3.3 精确遍历算法	34
3.4 自感知的批处理算法	35
3.4.1 初始路线搜索	35
3.4.2 批量优化处理	37

3.4.3 总体解决方案	39
3.5 实验研究	41
3.5.1 实验设定	41
3.5.2 实验结果与分析	43
3.6 本章小结	48
第四章 群体联合移动模式挖掘	50
4.1 引言	50
4.1.1 研究背景与意义	50
4.1.2 研究内容与挑战	51
4.1.3 研究成果与贡献	53
4.2 问题描述	54
4.3 精确搜索算法	55
4.4 社区感知的近似搜索算法	58
4.4.1 近似联合移动对搜索	59
4.4.2 社区感知聚类	60
4.5 实验评估	61
4.5.1 实验设置	61
4.5.2 实验结果与分析	63
4.6 本章小结	68
第五章 面向大规模移动对象的暴露接触搜索	70
5.1 引言	70
5.1.1 研究背景与意义	70
5.1.2 研究内容与挑战	71
5.1.3 研究成果与贡献	72
5.2 问题描述	73
5.3 精确遍历算法	74
5.4 精确分组处理算法	74
5.4.1 组距离	75
5.4.2 分组划分	75
5.4.3 预检查策略	76
5.4.4 算法细节	77
5.4.5 时间复杂度分析	78

5.5 近似分组处理算法	79
5.5.1 首尾优先检查策略	79
5.5.2 跳跃检查	80
5.5.3 算法细节	80
5.5.4 时间复杂度分析	81
5.6 实验研究	82
5.6.1 实验设置	82
5.6.2 实验结果与分析	83
5.7 本章小结	88
第六章 运动范围感知的轨迹交集相似度连接	90
6.1 引言	90
6.1.1 研究背景与意义	90
6.1.2 研究内容与挑战	93
6.1.3 研究成果与贡献	94
6.2 问题描述	95
6.3 时间段预检查	98
6.4 基于 M^* 树的连接算法	99
6.4.1 M^* 树结构	100
6.4.2 交集检查	100
6.4.3 算法细节	101
6.5 基于混合球树的连接方法	102
6.5.1 基于混合球树的交集预检查	102
6.5.2 基于混合球树的连接	104
6.6 实验研究	106
6.6.1 实验设置	106
6.6.2 实验结果与分析	108
6.7 本章小结	111
第七章 总结与展望	112
7.1 全文总结	112
7.1.1 面向大规模行程请求流的持续最优路线组合规划	112
7.1.2 基于轨迹数据的群体联合移动模式挖掘	113
7.1.3 大规模移动对象上的持续密切接触发现	114
7.1.4 面向时空运动范围的轨迹相似度连接	115

7.2 展望	116
致 谢	118
参考文献	119
攻读博士学位期间取得的成果	130

图目录

图 1-1	本文组织结构	11
图 2-1	轨迹与路径示例	16
图 2-2	基于位置的路径搜索与基于关键词的路径搜索示例	18
图 3-1	个人最优下潜在交通拥堵示例	30
图 3-2	行程请求数目对算法运行时间和全局通行时间的影响。(a) NY 路网下的算法运行时间；(b) NY 路网下的总通行时间；(c) TG 路网下的算法运行时间；(d) TG 路网下的总通行时间	43
图 3-3	行程请求到达速率的影响。(a) 对算法运行时间的影响；(b) 对总通行时间的影响	45
图 3-4	路段通行时间函数中两个参数的影响	46
图 3-5	批优化时间间隔的影响	47
图 3-6	提升参数和预检查策略的影响。(a) 运行时间；(b) 通行时间；(c) 预先检查策略的影响评估	48
图 4-1	伴随群体发现问题示例 (空间距离阈值为 10 米，联合移动的最小联合移动时长阈值为 3 分钟，伴随群体规模阈值为 3)	52
图 4-2	轨迹网格编号及网格编号序列示例	60
图 4-3	轨迹数目的影响。(a) 北京数据集上的命中率 (b) 北京数据集上的运行时间 (c) 波尔图数据集上的命中率 (d) 波尔图数据集上的运行时间	64
图 4-4	联合移动时长阈值的影响。(a) 北京数据集上的命中率 (b) 北京数据集上的运行时间 (c) 波尔图数据集上的命中率 (d) 波尔图数据集上的运行时间	65
图 4-5	伴随群体规模阈值的影响。(a) 北京数据集上的命中率 (b) 北京数据集上的运行时间 (c) 波尔图数据集上的命中率 (d) 波尔图数据集上的运行时间	66
图 4-6	SCAN 聚类结构相似度参数的影响。(a) 北京数据集上的命中率 (b) 北京数据集上的运行时间 (c) 波尔图数据集上的命中率 (d) 波尔图数据集上的运行时间	68
图 5-1	持续暴露接触搜索示例	72

图 5-2	潜在的近距离接触候选集	76
图 5-3	查询对象数量的影响。(a) 北京数据集上暴露接触案例数量；(b) 北京数据集上算法运行时间；(c) 波尔图数据集上暴露接触案例数量；(d) 波尔图数据集上算法运行时间	84
图 5-4	距离阈值的影响。(a) 北京数据集上暴露接触案例数量；(b) 北京数据集上算法运行时间；(c) 波尔图数据集上暴露接触案例数量；(d) 波尔图数据集上算法运行时间	85
图 5-5	接触持续时长阈值的影响。(a) 北京数据集上暴露接触案例数量；(b) 北京数据集上算法运行时间；(c) 波尔图数据集上暴露接触案例数量；(d) 波尔图数据集上算法运行时间	86
图 5-6	近似算法的评估。(a) 变化查询对象数目；(b) 变化接触阈值	87
图 5-7	预检查策略的评估。(a) 北京数据集；(b) 波尔图数据集	88
图 6-1	交集相似度连接示例	93
图 6-2	时间点运动范围示例	96
图 6-3	时间段交集相似度示例	97
图 6-4	时间段运动范围示例	99
图 6-5	M* 树的结构示例	100
图 6-6	混合球树结构示例	103
图 6-7	不同移动对象数目下的构建时间。(a): Geolife；(b): 波尔图	108
图 6-8	不同移动对象数目下的查询时间。(a): Geolife；(b): 波尔图	109
图 6-9	兴趣点数量对运行时间的影响。(a): Geolife；(b): 波尔图	110
图 6-10	时间段对运行时间的影响。(a): Geolife；(b): 波尔图	110
图 6-11	放宽参数对运行时间的影响。(a): Geolife；(b): 波尔图	111

表目录

表 2-1 相似度度量方法比较	23
表 3-1 主要符号解释	33
表 3-2 评估参数设定	41
表 3-3 比较算法	42
表 4-1 主要符号解释	55
表 4-2 评估参数设定	62
表 4-3 比较算法	63
表 5-1 主要符号解释	73
表 5-2 评估参数设定	82
表 5-3 比较算法	83
表 6-1 主要符号解释	95
表 6-2 比较算法	107
表 6-3 评估参数设置	107
表 6-4 剪枝性能	108

第一章 绪论

1.1 研究背景与意义

随着智能交通系统、移动导航设备以及传感技术的普及，大量基于位置的服务（如百度地图、滴滴打车以及各类签到程序）在人们的日常生活中发挥着不可或缺的作用，由此产生的时空数据呈现爆炸式增长。根据数据之间的依赖关系，时空数据可以划分为两类：独立数据和关联数据。独立数据如单个位置点信息通常是由经纬度组成的地理坐标。关联数据则刻画了多个独立数据之间的相互连接关系。基于单个对象所包含的信息，代表性的关联数据包含路径数据和轨迹数据等。其中位置点作为最基本的数据单元，刻画了实体在特定时间的空间坐标信息，是轨迹与路径数据构建的基础。具体而言，路径数据被定义为道路网络上一系列位置的序列，每个位置可包含诸如文本描述等附加信息；类似地，轨迹数据被定义为按时间顺序排列的一组位置序列，其中每个位置由空间信息、时间信息以及其他可选信息组成。

时空数据的产生具有广泛性、持续性和大规模性。首先，时空数据来源广泛，涵盖移动社交平台、导航服务系统、车联网设备、共享出行平台以及各类城市感知系统等多个领域。其中，移动社交网络是时空位置数据的重要来源之一。用户在使用社交媒体平台时，常常伴随地理标签进行内容发布。例如，据推特官方公开数据^①，全球每天发布的带地理标签的推文数量长期保持在千万级规模。维基百科^②中带有地理坐标标注的条目已超过百万级。其次，时空数据的产生具有持续性。根据 Statista 统计^③，截至 2024 年全球智能手机用户规模已超过 65 亿，智能终端持续性上传定位数据。因此，智能设备中的导航与地图服务平台产生了大规模的位置点与轨迹数据等时空数据。据公开报告显示，谷歌地图^④的月活跃用户超过 10 亿，其每天处理的定位与路径请求达到数十亿次；与此同时，网约车与共享出行平台在全球范围内每天产生数亿条行程记录，每一条行程均可视为由一组路网中的顶点序列或路段序列，或由一组采样位置点构成的轨迹序列。根据 Ericsson 报告^⑤，全球联网车辆数量已超过 3 亿辆，每辆车通过车载 GPS 与通信模块持续上传实时位置信息。

综上所述，位置点、轨迹与路径等时空数据在来源上高度多样，在规模上呈指

① <https://x.com/>

② <https://zh.wikipedia.org/>

③ <https://www.statista.com/>

④ <https://www.google.com/maps>

⑤ <https://www.ericsson.com/en/reports-and-papers/mobility-report/dataforecasts/mobile-traffic-forecast>

数级增长，已成为城市计算中数据挖掘研究的重要数据基础。城市计算是利用大规模城市时空数据，结合数据挖掘、人工智能和计算模型，对城市运行进行分析、理解和优化的一门交叉研究领域。海量、多源、高频且动态演化的时空数据，为城市计算中的高效算法设计提供了广阔而富有挑战性的应用场景，催生了许多基于地理位置的服务。代表性的基于时空数据挖掘的城市计算应用包括路径规划、轨迹模式挖掘、时空密切接触发现以及轨迹相似度连接等。其中，路径规划不仅直接影响人们的日常出行方式，也关系到物流运输、智慧城市管理以及能源优化等多个领域。此外，在诸多轨迹模式挖掘的实际应用中，高效且准确地从移动对象轨迹中发现联合移动模式具有重要意义。例如，在拼车推荐与出行规划中，可以通过识别稳定的同行模式群体，使其共同出发或考虑彼此对交通的影响等，从而提升资源利用率；在交通拥堵缓解中，可通过及早发现道路上的高密度移动模式群体来规避潜在拥堵；在疾病传播控制等公共卫生预警、信息扩散分析中，可以通过识别具有密切接触模式或联合移动模式的个体或群体为个体感染风险评估与资源调度提供数据支撑。再者，时空密切接触发现作为疫情防控与风险追踪的关键技术手段，在及时识别潜在传播链条、阻断扩散路径方面发挥着不可替代的作用。最后，轨迹相似度连接已广泛应用于移动物体间的共同行为检测、城市交通模式分析以及异常轨迹检测等场景，是轨迹数据挖掘的最基本也是最重要的研究工作之一。

首先，针对城市计算应用中的路径规划研究。可以观察到：近年来城市路网中行程请求呈现出高频、密集和动态的特征。在通勤时间等高峰时期，大规模用户很有可能会在极短的时间间隔内密集地发布行程请求，从而形成了持续的行程请求流。又比如，自动驾驶技术在人工智能、传感器融合与高精度地图等关键技术的推动下实现了快速发展，并逐步渗透到实际交通场景中。这种技术演进不仅改变了车辆的控制方式，也深刻影响了人类的驾驶行为与出行模式，使传统以人为主导的驾驶方式逐步向智能化、自动化方向演进。在自动驾驶场景下，大规模行程的批量规划天然存在。在这种场景下，实现针对行程请求流的路径规划这一需求变得更加迫切。已经存在的相关研究大多旨在为行程请求中的单个行程依次制定个人最优路线。然而，在为行程请求流规划路线时，仅仅基于当前的交通状态以个人最优为最终目标规划个人最优的路线，可能会导致交通拥堵。更合理的路线规划应该考虑到之前已经规划的行程会对未来的交通状态产生影响，因为它们也会动态影响路网中各路段的车流量。基于以上观察，本研究提出并研究了城市计算中面向大规模行程请求流的持续最优路线组合规划。其次，针对城市计算应用中的联合移动模式挖掘研究，可以观察到：在现实应用中，尤其是在公共安全紧急预警与传染病防控等对时效性要求极高的场景下，及时获得最新的联合移动结果至关重要。因此，联合移动模式挖掘任务往往需要以连续或增量的方式执行。

这不仅要求算法在时间复杂度上具备良好的可扩展性，还要求其在模式定义上具有一定的灵活性，以适应动态变化的移动行为。然而，现有群体模式定义及其对应方法在同时兼顾模式表达能力与计算效率方面仍存在明显不足，难以在大规模数据环境中实现高效、稳定的持续挖掘。基于以上观察，本文提出并研究了城市计算中基于轨迹数据的群体联合移动模式挖掘。此外，本文系统梳理了城市计算中现有的关于接触查询的研究进展，观察到：尽管已有方法在静态接触检测或离线轨迹分析方面取得了一定成果，但其大多假设数据完整可得，缺乏对持续到达的轨迹位置流的支持，且难以同时兼顾空间邻近约束与时间持续性约束，从而难以满足实时暴露接触监测的需求。基于以上观察，提出并研究了城市计算中面向大规模移动对象的持续暴露接触发现问题。最后，针对城市计算应用中的轨迹相似度连接研究，可以观察到：在真实应用场景中，由于采样频率不一致、定位误差不可避免、以及相邻采样点之间运动过程不可观测等因素，轨迹数据往往具有不确定性与稀疏性特征。现有多数相似度连接方法主要基于离散采样点进行距离计算，难以有效反映对象在采样间隔内的连续运动状态，从而可能遗漏在现实中具有重要意义的潜在交互行为。基于以上观察，提出并研究了城市计算中一种运动范围感知的轨迹相似度连接问题。

设计面向上述一系列城市计算中的高性能算法是一个不可阻挡的趋势，具有十分重要的实际应用价值和理论研究价值。以上城市计算中面临的核心挑战包括大规模数据处理、动态环境适应以及对象间潜在交互的准确建模。围绕路径规划、联合移动模式挖掘、接触搜索与轨迹相似连接等任务，设计高效算法与索引结构不仅具有理论价值，也具有显著应用潜力：一方面可提升城市交通系统的调度效率与拥堵缓解能力；另一方面可支持公共卫生管理、生态监测及智能出行等多领域的决策优化。基于此，本文旨在系统研究城市计算中高效算法设计问题，通过统一的时空数据建模与高性能计算框架，实现对移动对象行为的实时、准确与可扩展分析，为智慧城市应用提供技术支撑。

1.2 研究目标

推动上述四类城市计算中的算法应用，本文定义具体目标分别如下。

- (1) 针对“面向大规模行程请求流的持续最优路线组合规划”这一研究内容，需要合理定义全局最优路径规划问题本身，以及建立对应的路线规划算法。该研究应达到以下具体目标：所定义的“全局最优路径规划问题”需要为当前时间窗口内的所有行程请求规划一条从其起点到终点的可行路线。每条路线需合理定义不同车流量下的通行时间，优化目标是使该批

所有规划路线的总通行时间最小。同时，刻画的通行时间函数应该尽量简单，应合理考虑车流量和路线特镇，符合现实场景。所建立的“路线规划算法”应合理考虑新规划的路线会影响未来的交通状态这一事实，即规划的路线会增加一些路段的车流量从而影响该路段的实际通行时间。同时，能够满足批处理的实时性需求。规划后的路线应能极大程度减少全局通行总时间，从而缓解可能的交通拥堵。

- (2) 针对“基于轨迹数据的群体联合移动模式挖掘”这一研究内容，需要合理定义一种更加宽松且更具实用性的伴随群体，且设计精确的搜索算法以保证结果的准确性，以及近似算法以保证算法的高效性和实用性。该研究应达到以下具体目标：所定义的“伴随群体发现问题”的约束更加宽松且更具实用性。相较于传统群体模式定义，该问题在保证表达能力的同时降低了计算复杂度，为大规模数据场景下的高效求解奠定了理论基础。所设计的“精确搜索算法”能够在严格遵循问题定义的前提下，对所有候选对象组合进行精确验证，从而保证结果的完备性与正确性。所设计的“近似搜索算法”能够在保证较高结果质量的同时，显著降低了计算复杂度。此外，应能有效减少冗余计算与无效候选，从而在保证较高命中率的前提下显著降低运行开销，体现出较强的鲁棒性。
- (3) 针对“大规模移动对象上的持续暴露接触发现”这一研究内容，需要合理定义暴露接触发现对象，且设计精确的搜索算法以保证结果的准确性，以及近似算法以保证算法的高效性和实用性。该研究应达到以下具体目标：所定义的“暴露接触发现问题”能够持续发现所有在空间上与至少一个查询对象保持近距离、且在时间上满足持续接触阈值约束的对象，并进一步识别其可能存在的直接或间接暴露接触关系。该问题同时涉及空间距离判定、时间连续性维护以及查询集合动态扩展等关键因素，具有数据规模大、更新频率高、实时性要求强等显著特点。所设计的“查询算法”能够兼具高效率与高精度。一方面，移动对象规模可能达到百万甚至更高量级，直接基于两两对象比对的朴素方法在计算复杂度上难以扩展；另一方面，轨迹数据具有持续更新特性，算法不仅需要支持高吞吐量处理，还需保证低延迟响应，以满足实时监测需求。所设计的“近似实时搜索算法”能够在保证不产生漏检的前提下，通过引入可控的近似机制进一步提升整体处理效率，从而在准确性与效率之间取得良好平衡。
- (4) 针对“运动范围感知的轨迹相似度连接”这一研究内容，本文应达到以下具体目标：所定义的“相似度函数”能够在存在位置不确定性的时空环境中，以区域重叠替代点距离作为相似度判定标准，将相似度度量从

点级邻近关系扩展至连续时间下的区域交集关系。该建模方式能够准确地刻画稀疏采样轨迹中潜在的运动交互行为，从而弥补现有方法在未观测时间区间内建模能力不足的缺陷。在保证查询语义合理性的同时，实现高效的大规模对象对筛选。所设计的“时空索引结构”能够实现对候选对象对的有效过滤。该框架在保证结果正确性与完整性的前提下，显著降低了计算开销。高效的索引方法，不仅能够减少索引存储开销，提升索引更新效率，并且具有良好的可扩展性，能够适应数据规模的变化。同时，该索引能够包含所有满足索引条件的对象，不存在遗漏或者错误的索引项。

1.3 研究内容

本研究聚焦于四类城市计算应用中的关键问题：持续最优路径规划、联合移动模式挖掘、接触搜索与轨迹相似连接。这些问题均涉及海量移动对象和复杂时空约束，典型研究内容如下。

1.3.1 面向大规模行程请求流的持续最优路径组合规划

交通拥堵是城市化进程中普遍存在的问题。传统路径规划往往基于单一行程或当前交通状态进行局部优化，忽略了新到达请求对未来路网负载的影响。面对持续到达的行程请求流，单次最优路径规划可能导致集中拥堵，降低整体通行效率。因此，需要设计面向持续请求流的全局协同路径规划算法，实现对时间窗口内行程请求的联合优化，从整体视角缓解交通拥堵，提高路网利用率。具体来说，给定一组行程请求流，持续的最优路线组合问题旨在不断地处理最近到达的一批行程请求，寻求一个最优的路线组合使得所有路线的总的通行时间最小。具体研究内容如下。

- 研究如何定义一种全局最优路径规划问题，以符合现实应用场景。每条路线需合理定义不同车流量下的通行时间，优化目标是使该批所有规划路线的总通行时间最小。在一个动态变化的路网中，旨在对一段时间窗口内密集到达的行程请求进行联合优化。持续最优路线组合问题的核心在于：在动态交通环境中，将时间窗口内到达的行程请求作为一个整体进行协同规划，在保证个体可行性的同时，实现整体通行效率的最优。在实际应用场景中，大量用户很有可能会在极短的时间间隔内密集地发布行程请求，从而形成了持续的行程请求流。
- 研究如何建立一种高效地路线规划算法，应合理考虑新规划的路线会影响

未来的交通状态这一事实，即规划的路线会增加一些路段的车流量从而影响该路段的实际通行时间。同时，能够满足批处理的实时性需求。规划后的路线应能极大程度减少全局通行总时间，从而缓解可能的交通拥堵。

1.3.2 基于轨迹数据的群体联合移动模式挖掘

在大规模移动对象环境中，识别群体运动模式可揭示潜在的协同行为与社会动态规律。这类模式在拼车推荐、交通拥堵预测、异常行为检测以及生态监测中具有广泛应用。高效挖掘联合移动模式的核心挑战在于对象轨迹在时间上可能暂时离群或不连续，导致候选群体数量指数级增长，需要设计高效的索引结构与过滤策略以保证算法可扩展性。具体研究内容如下。

- 研究如何定义一种更加宽松且更具实用性的伴随群体：相较于传统群体模式定义，该问题在保证表达能力的同时降低了计算复杂度，为大规模数据场景下的高效求解奠定了理论基础。与已有研究不同，该伴随群体不要求对群体规模设置下界约束，也不以识别稳定群体结构为主要目标。相反，该伴随群体关注的是在动态环境下持续检测潜在接触关系，即便仅涉及少量对象亦具有重要意义。更为重要的是，在本研究的定义框架下，群体之外的个体也可能因间接接触或传播链关系而被判定为感染对象。
- 研究如何建立一种精确搜索算法。能够在严格遵循问题定义的前提下，对所有候选对象组合进行精确验证，从而保证结果的完备性与正确性。
- 研究如何设计一种近似搜索算法。能够在保证较高结果质量的同时，显著降低了计算复杂度。此外，应能有效减少冗余计算与无效候选，从而在保证较高命中率的前提下显著降低运行开销，体现出较强的鲁棒性。

1.3.3 大规模移动对象上的持续密切接触发现

在公共卫生管理与疫情防控中，及时识别个体间潜在接触事件对于控制疾病传播具有关键意义。由于移动对象的轨迹数据通常存在采样间隔与定位误差，仅依赖离散采样点判断接触关系容易产生误判。面向大规模连续轨迹数据的接触搜索算法必须支持高并发、低延迟处理，并在动态更新环境下保证结果的准确性与一致性，为公共卫生决策提供数据支撑。具体研究内容如下。

- 研究如何定义一种暴露接触发现问题。该问题的目标为持续发现所有在空间上与至少一个查询对象保持近距离、且在时间上满足持续接触阈值约束的对象，并进一步识别其可能存在的直接或间接暴露接触关系。
- 研究如何建立一种高效的精确查询算法以及一种时间复杂度更低的近似实时搜索算法。能够在保证不产生漏检的前提下，通过引入可控的近似机制

进一步提升整体处理效率，从而在准确性与效率之间取得良好平衡。

1.3.4 面向时空运动范围的轨迹相似性连接方法

传统轨迹相似性连接方法多依赖离散采样点计算对象间距离，难以刻画未观测时间区间内的潜在交互。交集相似度连接通过引入运动范围的概念，将对象在给定时间段内可能经过的空间区域作为分析单元，采用交集相似性度量量化对象间潜在重叠关系。这一方法可应用于交通监控、野生动物迁徙分析、接触追踪等场景，在稀疏采样或不确定条件下仍能稳健地识别潜在关联。

- 研究如何定义一种基于移动范围的交集相似度：以区域重叠替代点距离作为相似度判定标准，将相似度度量从点级邻近关系扩展至连续时间下的区域交集关系。该建模方式能够准确地刻画稀疏采样轨迹中潜在的运动交互行为，从而弥补现有方法在未观测时间区间内建模能力不足的缺陷。
- 研究如何建立一种时空索引结构。在保证结果正确性与完整性的前提下，能够实现对于候选对象对的有效过滤，显著降低计算开销。
- 研究如何设计一种高性能连接算法。结合上述时空索引结构，能够返回所有满足索引条件的对象，不存在遗漏或者错误的索引项。

1.4 研究所面临的挑战

设计面向上述四种城市计算应用中的高效算法，主要面临主要挑战如下。

(1) 面向大规模行程请求流的持续最优路线组合规划

高效处理“面向大规模行程请求流的持续最优路线组合规划”面临三个主要的挑战。首先，行程请求以数据流形式持续到达，系统需要在不断扩展的查询集合上进行在线优化，而非一次性处理静态输入；其次，早期规划结果会改变未来时刻各路段的车流分布，从而影响后续行程的路径选择，使得决策之间具有时间上的传递性；最后，不同行程路径通过共享路段产生资源竞争关系，使得各路径之间形成复杂的耦合结构，从而显著扩大了组合搜索空间。上述特性使得该问题不仅是一个路径搜索问题，更是一个具有强依赖性的动态组合优化问题。

(2) 基于轨迹数据的群体联合移动模式挖掘

高效地处理基于轨迹数据的群体联合移动模式挖掘面临以下技术挑战。首先，如何构建一个既支持多种群体模式表达，又能够灵活调整参数（如距离阈值、持续时间阈值与最小群体规模）的统一算法框架，是实现方法广泛适用性的关键。同时，该框架在增强表达能力的同时，必须避免因

模式定义过于群体模式发现通常涉及复杂的时空约束匹配、多对象关系维护以及候选结构验证等操作。若直接采用精确枚举算法,往往需要在大量时间快照上反复执行聚类或连通性检测,计算代价极高。因此,如何在保证结果准确性的前提下,通过有效的剪枝策略、搜索空间压缩技术、分层索引机制或近似计算方法减少不必要的候选生成与验证,是提升系统性能的核心问题。最后,轨迹数据具有天然的动态特征,移动对象的位置与行为随时间持续变化。在流式或持续更新的数据环境中,新轨迹不断到达,历史数据逐渐失效,群体结构也随之演化。若每次更新都重新对全部历史数据进行计算,将带来难以接受的时间开销。因此,如何设计支持增量更新或滑动窗口处理机制的算法,在无需重复计算全部数据的前提下高效维护群体模式结果,是保证系统实时性、稳定性与可扩展性的关键技术挑战。

(3) 大规模移动对象上的持续暴露接触发现

高效地处理大规模移动对象上的持续暴露接触发现存在以下技术难题。该问题同时涉及空间距离判定、时间连续性维护以及查询集合动态扩展等关键因素,具有数据规模大、更新频率高、实时性要求强等显著特点。具体而言,在连续到达的位置流中,以离散时间戳为基本处理单位,在每个时间快照上维护对象之间的空间邻近关系,并在滑动时间窗口内追踪接触持续状态。对于每一个非查询对象,需要判断其是否在连续时间段内满足距离阈值约束,并据此更新查询对象集合。在解决方案上,持续暴露接触搜索问题的精确求解需要在对象规模与时间维度双重增长的空间中进行大量距离计算与状态维护,其计算复杂度随查询对象数量和总体对象规模呈乘积增长,难以直接应用于实时系统。

(4) 运动范围感知的轨迹相似度连接

高效地处理运动范围感知的轨迹相似度连接存在以下技术难题。首先,交集相似度在连续时间区间上定义,理论上涉及无限多个时间点,直接计算精确结果在实际中不可行。因此,需要构建可计算的等价离散化模型或可证明误差界的近似计算方法。此外,现有相似度连接算法主要针对基于点距离或静态区域重叠的查询而设计,无法直接支持连续时间下的运动范围交集计算。若直接改造现有方法,往往导致候选对数量急剧膨胀,从而产生显著的计算开销。

1.5 论文的主要贡献

本文面向大规模时空数据挖掘在城市计算中的应用进行了深入的研究，提出并分析了四个新颖的问题，即面向大规模行程请求流的持续最优路径组合规划问题，基于轨迹数据的群体联合移动模式挖掘问题，大规模移动对象上的持续密切接触发现问题以及面向时空运动范围的轨迹相似性连接，并研究了高效能的解决方案，现将主要贡献总结如下：

(1) 面向大规模行程请求流的持续最优路径组合规划

本研究观察到以个人最优为最终目标规划个人最优的路线，可能会导致潜在的交通拥堵。针对这一现象，提出了一种新颖的持续最优路线组合问题，面向多种应用场景，包括交通流管理、实时路径规划和拥堵预防。为了高效且有效地解决该问题，设计了一种自感知批处理算法以高效求解持续最优路线组合问题。该算法包含两个主要步骤：初始路径搜索和批量优化处理。具体而言，对于一批新到达的行程请求，首先执行初始路径搜索，为每个新到达的行程请求生成一组高质量的初始路径组合。该步骤有助于在初步阶段避免部分交通拥堵。使用路径组合来存储这些为最近发布的行程请求规划的初始路径。为了进一步减少所有行程请求的总出行时间并缓解交通拥堵，以批处理模式对最近发布的请求集合进行处理。随后，执行批量优化处理步骤，对初始路径组合中的路径进行进一步优化。此外，基于两个真实数据集进行了大量实验来评估所提方法的准确性和效率。实验结果表明，本章提出的方法优于基线方法，并且能够以实时方式处理行程查询流。

(2) 基于轨迹数据的群体联合移动模式挖掘

观察到现有群体模式定义及其对应方法在同时兼顾模式表达能力与计算效率方面仍存在明显不足，难以在大规模数据环境中实现高效、稳定的持续挖掘。因此，提出了伴随群体这一联合移动模式定义。该定义相较于传统模式更加宽松且具备更强的适应性，尤其适用于需要持续结果更新与快速响应的应用场景，例如传染病传播监测与公共安全紧急风险预警。针对该问题，设计并实现了一种精确搜索算法以及一种社区感知的近似搜索算法，系统性地解决了伴随群体发现问题在准确性与效率之间的权衡难题。此外，基于两个真实世界数据集开展了全面的实验评估。实验结果表明，所提出的社区感知算法能够在用户可交互时间范围内，从大规模轨迹数据中高效发现伴随群体，验证了方法在实际应用场景中的有效性与实用价值。

(3) 大规模移动对象上的持续密切接触发现

本研究观察到已有的接触检测方法要么仅适用于少量查询场景，要么假设输入数据为静态轨迹集合，难以适应持续更新的流式轨迹数据环境。因此，在面对大规模移动对象及大量并发查询时，这些方法在可扩展性与实时性方面均存在明显不足。此外，由于缺乏对全体对象状态的统一维护机制，现有方法也难以为每个移动对象持续提供准确且实时的接触检测结果，限制了其在实际动态场景中的应用价值。因此，本研究形式化定义了持续暴露接触搜索问题，用于在大规模动态移动对象环境下持续维护最新暴露接触病例集合，并刻画了其递归扩展与实时更新特性。该问题为流行病传播监测与控制提供了统一的数据处理框架。针对该问题，提出了两种实时处理算法，分别为精确实时搜索算法与近似实时搜索算法。同时，设计了分组过滤与预检查等优化策略，有效减少候选对象数量并降低计算开销。此外，在两个真实数据集上进行了系统性的实验评估。实验结果表明，所提出的方法在保证检测准确性的同时，能够在大规模移动对象场景下实现高效的实时暴露接触检测，展现出良好的可扩展性与稳定性。

(4) 面向时空运动范围的轨迹相似性连接

本研究注意到如何在存在观测不确定性的条件下有效刻画移动对象之间的潜在运动交集关系，并在海量移动对象数据环境下高效地发现所有满足交集条件的对象对，仍然是一个亟待解决的开放性问题。因此，提出了一种移动对象相似度连接问题——交集相似度连接。与传统基于离散位置采样点距离或静态区域重叠的相似度连接不同，交集相似度连接在语义层面显式引入“运动范围”概念，将相似度度量从点级邻近关系扩展至连续时间下的区域交集关系。该建模方式能够更准确地刻画稀疏采样轨迹中潜在的运动交互行为，从而弥补现有方法在未观测时间区间内建模能力不足的缺陷。针对交集相似度连接在连续时间区间下计算复杂、候选对象对规模庞大的挑战，本文提出了一种由时间区间运动范围引导的高效求解框架。具体而言，设计了混合球树索引结构，用于层次化组织对象在时间区间内的运动范围表示，并结合重划分策略与多级剪枝技术，实现对候选对象对的有效过滤。该框架在保证结果正确性与完整性的前提下，显著降低了计算开销。本文在两个真实世界数据集上开展了系统性的实验评估，从运行时间、可扩展性以及参数敏感性等多个维度对所提出方法进行了深入分析。实验结果表明，相较于精心设计的基线方法，本文方法在保持查询结果准确性的同时，在整体性能上取得了显著提升，

验证了所提出模型与算法的有效性与实用价值。

1.6 论文的组织结构

本文包括七个章节，行文框架如图 1-1所示，具体内容如下。

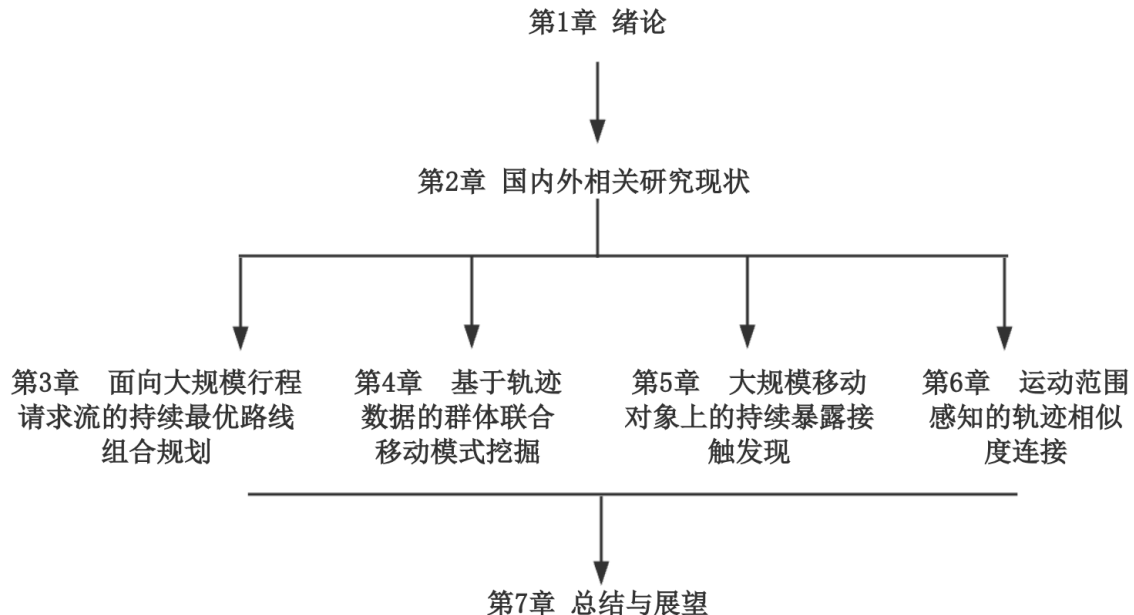


图 1-1 本文组织结构

第 1 章，绪论。首先分析了本论文的研究背景与意义，阐述了论文的主要研究目标，研究内容，所面临的挑战以及主要贡献，并给出了本论文的组织结构。

第 2 章，相关工作和研究综述。本章从时空数据建模和管理，路径搜索和推荐，以及轨迹数据挖掘和应用相关研究进行梳理，并分析了相关工作的贡献和不足之处。其中，在路径搜索和推荐中，本文主要总结了基于位置的路线搜索和推荐以及基于关键词的路径搜索和推荐；在轨迹数据挖掘和应用中，本文主要研究了轨迹相似度连接以及一些流行的轨迹模式挖掘。最后，总结了与本文密切相关的移动对象时空索引技术。

第 3 章，面向大规模行程请求流的持续最优路径组合规划。首先，给出了本章工作的研究背景和动机。然后，正式提出了所要解决的问题。针对该问题，本文分别提出了精确遍历算法和自感知的批处理算法。其中，批处理算法包含初始路线规划、批处理优化以及将两者相结合的总体解决方案。最后，基于真实数据集和合成数据集开展了实验研究，分别在变化行程请求数目、行程请求到达速率以及优化时间间隔下比较了本章提出的算法与基准算法的性能表现。实验结果验证了本文所提出算法的高效性和有效性。在两个真实数据集上的实验结果表明，本研

究提出的算法与个人最优路径规划算法相比，所提出方法能够降低超过 40% 的总通行时间。本章结尾对提出的问题和算法进行了总结，并阐述了本章研究所做出的具体贡献。

第 4 章，基于轨迹数据的群体联合移动模式挖掘。首先，给出了本章工作的研究背景和动机。然后，提出了本章要解决的联合移动模式挖掘问题。为了解决该问题，本文分别提出了一个精确搜索算法和一个近似搜索算法，并分析了对应的的时间复杂度。最后，基于两个真实数据集开展了实验研究，分别比较了在变化轨迹数量、协同行为建立的时间阈值、群体规模阈值等参数下的算法的性能表现。实验结果验证了本文所提出算法的高效性准确性和可扩展性。本章结尾对提出的问题和算法进行了总结，并阐述了本章研究所做出的具体贡献。

第 5 章，大规模移动对象上的持续暴露接触发现。首先，给出了本章工作的研究背景和动机。然后，提出了问题定义。为了解决该问题，分别讨论了精确遍历算法、精确分组算法和近似分组处理算法。随后，在两个真实数据集，分别变化查询对象数量、距离阈值和接触时长阈值等参数，对以上 3 个解决方案进行了综合比较。本章结尾对提出的问题和算法进行了总结，并阐述了本章研究所做出的具体贡献。

第 6 章，运动范围感知的轨迹相似度连接。首先，讨论了本章的研究背景与意义，研究成果与内容以及研究成果与贡献。随后，提出了一种移动对象相似度连接问题。为了解决该问题，先提出了一种基于 M 树变种的连接算法作为基准方法。为了实现更加高效的运算，接着提出了一种运动范围引导的匹配方法，包括 3 个主要的核心思想，即时间区域预检查、基于混合球树的交集预先检查以及基于混合球树的相似度连接算法。在实验研究中，在两个真实数据上进行了实验。分别变化移动对象基数、特征数量以及时间区间等参数下算法的实际表现。最后，本章进行了小结，阐述了主要研究成果。

第 7 章，全文总结。分别总结了以上 4 个研究问题、解决方案以及本文所作出的主要贡献。此外，还分析了进一步可以研究的问题及对应的拟解决思路。

第二章 相关技术及研究综述

本章相关技术及研究包含三大类：时空数据建模和管理，路径搜索和推荐，以及轨迹数据挖掘和应用。在时空数据建模和管理中，基于现有研究工作时空数据进行分类，并讨论了常见的空间索引技术；在路径搜索和推荐中，调研了基于位置的路径规划以及基于关键词的路径规划。在轨迹数据挖掘和应用中，对轨迹相似度连接、模式挖掘时空索引技术和相关应用和研究现状进行了调研。

2.1 时空数据建模和索引

随着智能交通系统和基于位置的社交媒体的持续普及，基于位置的服务提供商和用户能够生成海量的基于位置的时空数据 [1]。根据数据之间的依赖关系，时空数据通常可以划分为两类：独立数据（例如单个位置点）和关联数据（例如路径、轨迹）。二者的主要区别在于，后者由多个相互依赖的对象组成。一般而言，基于位置的数据被建模为由位置信息以及其他信息（包括时间、文本和属性值对）组成的对象。一些典型示例包括来自配备 GPS 设备的采样点、基于位置的社交媒体签到数据，以及带有位置信息的短文本数据如带地理标注的推文。不同于对象之间相互独立的基于位置数据，关联数据刻画了对象之间的相互连接关系。基于对象所包含的信息，本文研究的关联型路径数据包含两类：路径数据和轨迹数据。为了更有效地对位置点和路径数据在道路网络中进行分析处理，需要构建合理的数据模型对数据的时间特征，空间特征以及文本特征等进行建模，以便于精炼数据信息，挖掘数据各条目间的相关性。在本节中，定义若干时空数据中的基础术语，并讨论了常见的索引技术。

2.1.0.0.1 数据空间 本文主要考察两类具有代表性的数据空间：欧几里得空间和路网空间。在欧几里得空间中，两个对象之间的空间距离通常看作是两位置点间的直线距离，也可以视为地球表面的大圆距离 [1]。而在路网空间中，边的权重一般不满足三角不等式。一般考虑欧式距离作为轨迹位置点空间相似度的主要衡量标准，而路网距离作为路径搜索和推荐的主要影响因素。

2.1.0.0.2 道路网络 通常情况下，道路网络 (路网) 可建模为一个连通的有向图 $G(V, E, t_m)$ ，其中顶点集合 V 表示道路交叉口或道路端点，边集合 $E \subset V \times V$ 表示道路路段。每一条边 $e(v_i, v_j) \in E$ 连接两个端点 v_i 和 v_j ，其中 $v_i, v_j \in V$ 。每条道路路段 e 都关联一个权重函数，记为 t_m 。函数 $t_m : E \mapsto \mathbb{R}$ 为每条边 e 赋予

一个实值权重 $t_m(e)$ (例如通行时间、距离等)。在大多数情况下, 若 $i \neq j$, 则 $t_m(e_i) \neq t_m(e_j)$ 。根据权重函数的不同形式, 道路网络可以进一步划分为多种类型, 包括但不限于静态道路网络、动态道路网络 (例如 [2, 3]) 以及时间依赖道路网络 (例如 [4–6])。在静态道路网络中, 每条边的权重保持不变, 即为常数; 而在动态道路网络中, 边的权重会根据特定设置动态更新, 例如交通状况 [7–9] 或自感知规划 [2, 3, 10]。在时间依赖道路网络中, 边的权重随时间变化, 通常基于大规模历史轨迹数据进行建模。

2.1.0.0.3 地理位置点 一个地理位置点通常被定义为地理空间中的一个二元组 $\langle x, y \rangle$, 通常由经度和纬度构成。每一个二元组对应于由具备 GPS 功能的设备记录的一个唯一空间点。

示例 2.1 (地理位置点示例) 北京天安门广场的位置表示为 $\langle 39^\circ 54' 27'' \text{N}, 116^\circ 23' 17'' \text{E} \rangle$ 。

2.1.0.0.4 时空位置点 一个时空位置点由一个离散的位置点 $\langle x, y \rangle$ 和时间戳 t 组成, 表示为 $[\langle x, y \rangle, t]$ 。

示例 2.2 (时空位置点示例) 某人在 2023 年 10 月 1 日 10 时 10 分经过北京天安门广场的时空位置点表示为 $[\langle 39^\circ 54' 27'' \text{N}, 116^\circ 23' 17'' \text{E} \rangle, 2023/10/01, 10:10]$ 。

2.1.0.0.5 Dijkstra 算法 Dijkstra 算法由 Dijkstra 于 1959 年提出, 用于求解带非负权重图中单源最短路径问题 [11]。给定加权有向图 $G = (V, E, t_m)$ 及源点 $s \in V$, 算法目标是计算源点 s 到图中其余所有顶点的最短路径长度。该算法本质上是一种基于贪心策略的逐步扩展方法: 在每一步中选择当前未确定最短路径的顶点中距离源点最近的一个进行扩展, 从而保证全局最优性。形式化地, $t_m(u, v) \geq 0$ 表示边 (u, v) 的权重。算法维护一个距离数组 $dis[\cdot]$ 与一个已确定最短路径的顶点集合 T 。其核心思想可概括如下:

(1) **初始化阶段:** 对所有顶点 $v \in V$, 令

$$dis[v] = \begin{cases} 0, & v = s, \\ +\infty, & v \neq s. \end{cases} \quad (2-1)$$

初始时集合 $T = \emptyset$ 。

(2) **顶点选择:** 从 $V \setminus T$ 中选择满足 $u = \arg \min_{v \in V \setminus T} dis[v]$ 的顶点 u , 并将其加入集合 T 。

(3) **更新最短距离:** 对于所有与 u 相邻且尚未加入 T 的顶点 v , 若满足 $dis[u] + t_m(u, v) < dis[v]$, 则更新 $dis[v] = dis[u] + t_m(u, v)$ 。

(4) **终止条件**: 重复步骤 2-3, 直至 $T = V$ 或所有可达顶点均被确定。

需要指出的是, Dijkstra 算法的正确性建立在边权非负的前提之上。当图中存在负权边时, 该算法不再保证最优性。设图中顶点数为 $n = |V|$, 边数为 $m = |E|$ 。算法整体复杂度主要取决于“最小距离顶点选择”与“松弛操作”的实现方式: 若采用顺序数组实现优先选择操作, 则每次选择最小值需 $O(n)$ 时间, 总时间复杂度为 $O(n^2)$ 。若采用二叉堆或二项堆实现优先队列, 则时间复杂度为 $O((n+m) \log n)$ 。若采用斐波那契堆, 整体复杂度可优化为 $O(n \log n + m)$ 。

2.1.0.0.6 位置点间的路网距离计算 在城市路网环境中, 欧氏距离难以准确反映真实移动成本。因此, 本章采用基于道路网络的最短路径距离作为位置点之间的度量标准。设两个位置点 s_i 与 s_j 已经映射到路网中的顶点集合。定义其路网距离 $d(s_i, s_j)$ 为二者在路网中的最短路径长度, 如式 2-2。其中, $sd(s_i, s_j)$ 表示在路网中从 s_i 到 s_j 的最短路径长度, 可通过 Dijkstra 算法在边权非负条件下有效求解。

$$d(s_i, s_j) = sd(s_i, s_j) \quad (2-2)$$

2.1.0.0.7 路径 一条路径被定义为一个有限的顶点序列 $v_1, v_2, \dots, v_k$, 该序列 y 也可以表示为路网中的一组边序列 (即 $e(v_i, v_{i+1}) \in E$)。

2.1.0.0.8 精确轨迹 移动对象的精确轨迹是连接该对象所经过的每一个时空位置点的一条连续曲线。移动对象可以是人、动物, 或移动的位置追踪设备。现实应用中, 由于位置追踪设备无法连续记录位置, 精确轨迹在现实中是无法直接获取的。

2.1.0.0.9 采样轨迹 移动对象的一条采样轨迹是从精确轨迹中按照一定采样率, 由位置追踪设备采样得到的、按时间顺序排列的有限个离散时空位置点序列。轨迹中的每一个时空位置点均按照其时间戳排序。在不特殊说明情况下, 本文所说的轨迹一般是指采样轨迹而非精确轨迹。

与路径不同, 轨迹并非直接存在于道路网络之上, 而是通过对连续精确轨迹进行采样生成的。轨迹本身依赖于精确轨迹及其采样率: 即使移动对象行走的是同一路径, 不同的采样率也可能生成不同的轨迹。图 2-1 给出了一个示例, 用以说明独立路径数据 (位置点) 以及关联路径数据 (精确移动轨迹、轨迹和路径)。道路网络记为 $G(V, E)$, 其中顶点集 $V = \{v_1, v_2, v_3, v_4\}$, 边集 $E = \{v_1v_2, v_2v_3, v_3v_4\}$ 。假设一个移动对象从起点 v_1 移动到终点 v_4 , 并经过一系列时空位置点, 其可能的精确轨迹由一条黑色曲线表示。采集到的轨迹中的每一个时空位置点均已匹配到道路网络中的某一具体路段。基于这些采样得到的时空位置点即可生成相应的采

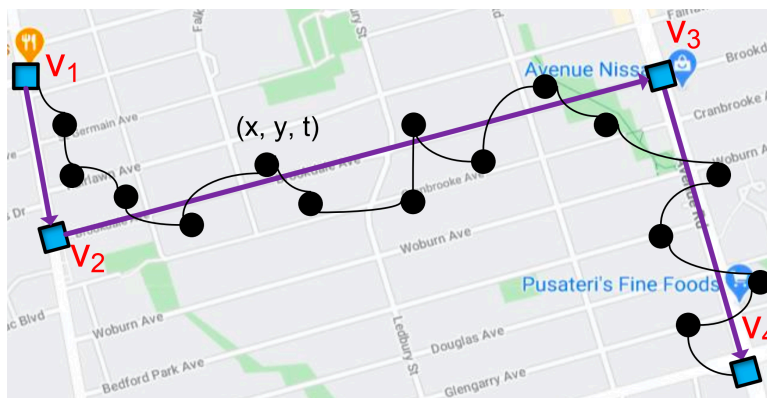


图 2-1 轨迹与路径示例

样轨迹，在图中标记为黑色的顶点序列。在该示例中，经过地图匹配后，移动对象所经过的路径为 $\langle v_1, v_2, v_3, v_4 \rangle$ 。需要注意的是，由于位置追踪设备无法连续采集位置，该示例中的精确移动轨迹在现实中是不存在的。

为降低系统负载并增强对未观测路径的刻画能力，一些研究引入基于速度约束的轨迹表示模型 [12–14]。此类方法通过构造可能到达区域或速度包络来限制对象的可达空间，从而在一定程度上缓解稀疏采样带来的不确定性。Ding 等人 [15] 进一步提出数据驱动方法，利用历史轨迹挖掘实际可达区域以刻画运动约束，提高区域建模的现实性。为了支持更加灵活的相似性定义并提高计算效率，Bakalov 等人 [16] 采用分段聚合近似 [17] 将轨迹映射为符号序列，并构建紧凑索引以支持高效近似连接。该方法通过后过滤步骤对候选结果进行精化，从而在减少 I/O 开销的同时保持结果质量。

2.1.0.0.10 常见时空索引 围绕上述定义的时空数据模型，大量索引结构被提出以支持高效的时空查询处理。其中，R 树及其变体 [18, 19] 是最具代表性的空间索引结构之一，其通过最小外接矩形对数据进行层次化组织。每个内部节点对应一个矩形区域，子树中的数据对象均位于该区域之内。类似地，KD 树 [20, 21] 通过递归空间划分实现高效的点查询与范围查询。这类方法通常适用于基于矩形区域或轴对齐分割的查询场景，在处理高维或动态更新数据时仍具备良好的可扩展性。为支持基于距离度量的查询，M 树 [22, 23] 与球树 [24] 被提出用于组织度量空间中的对象。不同于 R 树依赖矩形边界，球树采用超球体作为节点覆盖区域，更适用于欧氏距离或其他度量空间下的相似性搜索。Omohundro [25] 对多种球树构建策略进行了系统比较，分析了构建时间与树结构质量之间的权衡关系。随后，Dolatshah 等人 [26] 通过引入主成分分析优化划分方向，从而提升树结构的平衡性与查询效率。在移动对象查询领域，研究者进一步针对接触搜索与相似性检索设计了专用索引结构。例如，Chao 等人 [27] 提出了广义接触搜索查询模型，用于识

别满足特定接触条件的移动对象集合。Zhang 等人 [28] 研究了接触相似性查询，并设计了专用的 UTM-tree 索引结构以提升查询效率。这些方法通常基于离散采样点或基于距离阈值的接触判定机制。

2.2 路径搜索和推荐

随着位置服务的日益普及，大量路径规划应用（如谷歌地图，百度地图）以及网约车服务（如滴滴打车）在日常生活中发挥着不可或缺的作用。与仅返回一组位置点不同，路径搜索是一项计算任务，其目标是生成一条从给定起始位置出发、到达指定终点位置，并且途经若干由用户指定类型的地理实体的路径。具体而言，这些地理实体可以被视为用户偏好，并通过路径查询进行指定。路径搜索的目标是在满足约束条件的前提下，找到一条能够至少访问每一种指定类型中一个满足条件实体的路径。根据用户定义实体的数据类型，该领域的现有研究可进一步划分为两类：基于位置的路径搜索以及基于关键词的路径搜索。下面简要给出这两类问题的形式化定义以便对比，其详细内容分别在第 2.2.1 节和第 2.2.2 节中讨论。

通常，基于位置的路径搜索的输入包括：一个起始位置、一个终点位置、一组可指定的空间位置、一个可选的预算约束（例如行程时间、关键词访问顺序或偏好），以及一个用于计算路径目标得分的函数。其目标是在预算约束下，从起始位置到终点位置找到一条经过指定的空间位置的最优路径（即目标函数取最大值的路径）。相比之下，基于关键词的路径规划问题的输入并非具体的位置集合，而是一组查询关键词。该类查询通常返回一条从起始位置到终点位置的路径，使得在预算约束以及关键词相关约束（例如必须覆盖指定的所有关键词或兴趣点）的条件下，目标函数达到最大值。

路径搜索与路径规划在空间数据管理和基于位置的社会服务中一直发挥着重要作用。在此背景下，对现有关于路径搜索与路径规划的相关研究文献进行了系统综述，并对每一项具有代表性的路径搜索与路径规划研究工作进行了详细介绍。本章节总结了现有研究的主要成果，从而揭示了一些新的研究启示，可为空间数据管理与地理信息系统领域的研究人员和软件工程师提供参考与指导。

图 2-2 给出了一个示例，用以说明基于位置的路径查询与基于关键词的路径查询之间的差异。给定一组空间位置 $\{S, A, B, C, D, E, T\}$ ，每个位置都关联有类别信息（例如酒店、公园等）。位置 A, B, C, D, E 分别关联类别 $\langle \text{机场}, \text{体育场} \rangle$ 、 $\langle \text{公园}, \text{学校} \rangle$ 、 $\langle \text{厕所} \rangle$ 、 $\langle \text{酒店}, \text{公园} \rangle$ 和 $\langle \text{学校}, \text{公园} \rangle$ 。考虑一个基于位置的路径查询：“从 S 到 T ，途经 A 和 C ，寻找一条最优路径”，则两条候选路径 $\{SA, AC, CD, DT\}$ 和 $\{SA, AC, CE, ET\}$ 均满足条件，最终可根据具体优化

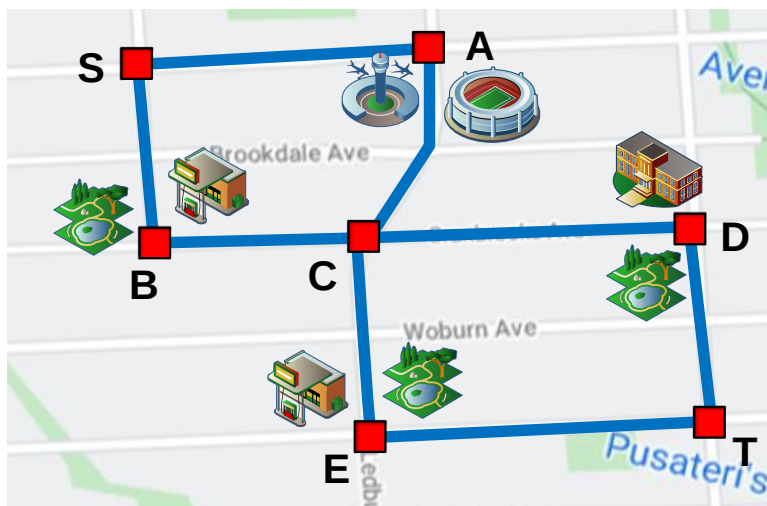


图 2-2 基于位置的路径搜索与基于关键词的路径搜索示例

目标（如最小行程时间）向用户推荐其中一条路径。与提供具体位置不同，基于关键词的路径规划考虑的是一组用户定义的关键词（例如兴趣点类别），其目标是找到一条从查询起点（如家或酒店）出发、并覆盖一组用户指定类别（如 {酒吧, 餐馆, 博物馆}）的最优路径。给定一个基于关键词的路径查询：“从 S 到 T ，途经一个体育场、一个厕所和一个公园”，同样可以得到两条满足条件的候选路径 $\{SA, AC, CD, DT\}$ 和 $\{SA, AC, CE, ET\}$ 。此外，用户还可以指定不同类别之间的部分顺序约束（例如必须先访问公园再访问体育场），这也是路径规划中一种常见的查询形式。

2.2.1 基于位置的路径规划

该领域的相关研究主要可以划分为两类：基于单个起点终点的路径搜索和推荐和基于多位置或多用户的路径搜索和推荐。

2.2.1.1 基于单个起点终点的路径搜索和推荐

该类研究主要在给定特定约束条件的情况下执行基本的路径查询。给定一个 $\langle \text{起点}, \text{终点} \rangle$ 元组，并可能附带一些时间相关约束，基于起点—终点的路径规划旨在用户特定约束（如出发时间）下，求解从起点位置到终点位置的最优路径（例如最短距离或最小拥堵概率）。在该方向上，大量研究致力于在静态图上解决基于距离的最短路径搜索问题。Dijkstra 算法 [11] 是经典的最短路径算法，可用于在静态图中寻找从节点 s 到节点 t 的最短路径。随后，A* 算法 [29] 通过引入启发式策略进一步提升了搜索效率。在实际应用中，由于交通网络具有动态特性，道路图往往会频繁更新，这些变化会显著影响路径规划的性能 [6–8]。下面分别介绍时间依赖最短路径问题、交通感知路径规划、自感知路径规划以及基于用户偏好

的路径规划。

2.2.1.1.1 时间依赖最短路径问题 给定一个路网以及用户给定的出发时间区间，TDSP 查询旨在寻找一个最优出发时间，使得从一个位置到另一个位置的总行程时间最小，其中交通状况会随时间动态变化 [4–6, 30, 31]。相关研究既包括基于离散时间方法的近似解 [30]，也包括基于连续时间方法的最优解 [5, 31]。Ding 等人 [4] 提出了基于 Dijkstra 的新算法以求解最优出发时间；后续研究 [6] 构建了一种新的高度平衡树索引结构 TD-G-tree，并基于该结构提出了支持 TDSP 查询的高效算法。在随机路网中，边的行程时间被建模为随机变量。给定源点 s 和终点 d ，随机路径规划旨在寻找期望代价最小的路径。Lim 等人 [32] 提出了随机运动规划算法，以最大化在给定时间预算内到达目的地的概率。Yang 等人 [33] 提出了随机天际线路径模型；Pedersen 等人 [34] 利用轨迹数据构建高分辨率不确定路网，并提出了高效的随机路径规划算法。时间依赖路网 [4] 与概率路网 [35] 是两类典型的动态空间网络。Hua 等人 [35] 提出了三种概率路径查询模型，通过将边权建模为随机变量并使用样本近似，设计了高效的概率计算方法。

2.2.1.1.2 交通感知路径规划 交通感知路径规划主要关注在考虑实时交通状况的前提下，寻找行程时间最短的路径。Xu 等人 [7] 通过分析路网中的交通状态并挖掘时空知识来指导路径规划，以避免在大规模图上的冗余计算和频繁重计算。Shang 等人 [8] 提出了交通感知空间网络中的无阻塞路径规划问题，通过分析移动对象的不确定轨迹构建交通感知路网，并设计了高效算法。另一项研究 [9] 利用人群轨迹数据预测人类移动行为，识别拥挤站点，并规划避开拥挤区域的路径。此外，Malviya 等人 [36] 提出了连续查询系统，在交通状态变化时动态更新用户的最短路径。

2.2.1.1.3 自感知路径规划 随着路径规划系统的广泛应用，规划出的路径会反过来影响未来的交通状况，即已有路径规划会对后续规划产生影响 [2, 3, 10]。为此，自感知路径规划被提出，用以避免由先前规划路径引发的潜在交通拥堵。早期工作 [10] 提出了自感知路径规划算法，利用当前规划路径预测未来交通状态；后续研究 [3] 进一步表明，当为较大比例的车辆生成路径时，可以有效估计整体交通模式，并提出了适用于城市级规模的解决方案。

2.2.1.1.4 基于用户偏好的路径规划 传统路径规划算法通常围绕单一优化目标（如最短路径）展开，但研究表明，现实中的驾驶员往往选择既非最短也非最快的路径 [37]。Guo 等人 [38] 发现，不同驾驶情境对应不同的路径偏好，并据此提出

了情境感知的偏好路径规划问题。现有研究多通过挖掘历史轨迹数据来学习用户偏好，并据此指导路径规划算法 [37, 39, 40]。

2.2.1.2 基于多位置或多用户的路径搜索和推荐

基于多位置路径搜索和推荐输入通常为的一组位置、位置序列或位置一时间对序列。研究目标是在覆盖用户指定位置的前提下，使某种代价最小化或收益最大化。其中，移动序列推荐问题 [41] 旨在为出租车司机推荐最优行驶路径，以最小化成功载客前的潜在行驶距离。后续研究 [42–45] 从不同目标函数和多车辆场景对该问题进行了扩展。基于位置的轨迹搜索问题以查询位置集、轨迹数据库和参数 k 为输入，返回在地理上最优连接这些位置的 k 条轨迹 [46, 47]。相关扩展还包括群体出行规划查询 [48]，旨在为多个出发点和一个目的地规划低成本路径，并允许通过有限数量的集合点。相关研究还包括最优多集合点路径查询 [49] 以及群体出行路径推荐问题 [50]。

基于多用户的路径搜索和推荐关注在多用户场景下生成多条不同路径或共享路径，以满足整体最优目标。现有研究主要包括基于群体的最优路径规划和全局最优路径规划。Hashem 等人 [51, 52] 提出了群体出行规划查询，旨在最小化群体总出行距离。全局路径规划为每个查询生成一条独立路径，使所有用户的总代价最小或总收益最大 [53]。相关研究还包括多出租车推荐 [45]、拼车服务路径规划 [54, 55] 以及全局最优路径问题 [2]。

2.2.2 基于关键词的路径规划

近年来，关键词感知路径规划问题得到了广泛研究。现有研究大体可以分为两类：基于精确匹配的关键词感知路径规划（例如 [56–58]），以及基于近似匹配的关键词感知路径规划（例如 [59–63]）。总体而言，近似匹配是对精确匹配的一种更为现实的扩展，用以处理拼写错误或用户定义条件模糊的情形。

在路径规划领域，Li 等人 [56] 是最早在空间数据库中提出行程规划查询（Trip Planning Query, TPQ）的研究之一。该模型中，每个空间对象同时具有位置信息和关键词，并使用 R 树进行索引。一个行程规划查询由起始位置、目标位置以及关键词集合构成，其目标是在满足距离、交通状况等优化目标的前提下，寻找一条从起始位置到目标位置的最优路径，并且该路径至少经过给定关键词集合中每个关键词对应的一个对象。文献 [56] 基于度量空间中的三角不等式性质，提出了贪心算法和整数规划算法，并设计了近似算法以应对搜索空间的指数级增长。随后的一些工作 [64, 65] 对 TPQ 进行了扩展。例如，[64] 在约束条件下最大化用户满意度，其约束包括所有关键词必须被访问，且路径在给定时间预算内以不低于用

户设定阈值的概率完成。该方法通过离线与在线相结合的方式剪枝无效候选路径。Yawalkar 等人 [65] 进一步引入多种偏好函数, 以平衡关键词覆盖与路径长度, 并通过目标导向搜索显著缩小搜索空间。Sharifzadeh 等人 [57] 提出了 TPQ 的一种变体, 称为最优序列路径查询。其中, 对关键词集合 Ψ 施加一个全序约束, 仅给定起始位置, 目标是寻找一条从起始位置出发、按照给定顺序访问不同类型位置的最短路径。为此, 该研究提出了一个基于阈值的向量空间算法和一个结合 R 树的高效扩展算法。在实际应用中, 用户更倾向于指定部分顺序或任意顺序 [66–68]。Chen 等人 [66, 67] 提出了多规则部分序列路径查询, 并设计了三种启发式算法。Li 等人 [68] 进一步提出前向搜索与后向搜索方法。后续研究 [69–71] 引入了时间约束和用户偏好等附加条件。

Cao 等人 [58] 提出了关键词感知最优路径 (Keyword-aware Optimal Route, KOR) 查询, 其目标是在满足预算约束的前提下, 寻找覆盖用户指定关键词集合且目标函数最优的路径。与 TPQ 查询不同, 该查询同时考虑用户偏好和预算约束, 并不限制关键词访问顺序。该研究提出了两种近似算法和一种贪心算法来高效求解 KOR。

交互式路径搜索 [72–76] 并非一次性返回完整路径, 而是逐步向用户推荐下一个访问位置, 并根据用户反馈动态调整路径。相关研究提出了贪心算法、数据挖掘方法以及近似算法, 以在路径长度、关键词相关性和用户偏好之间取得平衡。群体行程规划关注多个用户共同出行的路径规划问题。每个用户提供起始位置、目标位置、关键词集合及约束条件, 目标是在满足所有用户需求的同时, 最小化总距离或最大化群体偏好。Hashem 等人 [51] 首次提出 GTP 查询, 并设计剪枝技术以提高效率; 后续工作 [52] 进一步提出精确与近似算法。Fan 等人 [77, 78] 则在考虑个体偏好和约束的基础上, 提出了最大化群体满意度的算法。此外, 交通感知路径搜索 (TARS) [79] 考虑交通状态变化对行程速度的影响; 室内路径规划研究 [80, 81] 将关键词感知路径规划扩展到室内空间场景。

基于近似匹配的关键词感知路径规划问题的输入通常包括起始位置、目标位置以及若干 (关键词, 相似度阈值) 对。研究重点在于利用字符串相似度度量判断对象关键词是否与查询关键词匹配。代表性工作包括多近似关键词路径查询 [59], 其目标是在满足编辑距离阈值的条件下, 寻找覆盖每个关键词的最短路径。Zheng 等人 [60] 提出了基于线索的路径搜索, 允许用户指定关键词及空间关系线索, 并设计了多种高效算法。

2.3 轨迹数据挖掘和应用

大规模轨迹数据的管理、挖掘与分析问题已受到数据科学领域的广泛关注。与本文密切相关的代表性研究主要包括轨迹相似度连接、轨迹模式挖掘、轨迹聚类等等。

2.3.1 轨迹相似度连接

轨迹相似度搜索与连接问题是时空数据管理领域的重要研究方向，相关工作已从距离度量设计、索引优化以及语义扩展等多个维度展开深入探索。总体而言，现有研究可从相似性建模方式与查询语义两个方面进行归纳。因此，在介绍具体的轨迹相似度连接问题之前，首先回顾已有关于轨迹相似度度量的研究工作。

2.3.1.1 轨迹相似度度量

2.3.1.1.1 传统相似性度量方法 与简单数据类型（如点或向量）的距离计算不同，由于轨迹具有复杂的时空属性，其相似性度量的设计具有较高难度。此外，轨迹数据往往由不同的采样策略或非均匀采样率生成，这进一步增加了相似性计算的复杂性。为此，研究者提出了大量经典的轨迹相似度度量方法，其中最具代表性的包括基于欧氏距离的方法（EDB）[82]、动态时间规整（DTW）[83]、最长公共子序列（LCSS）[84]、带实值惩罚的编辑距离（ERP）[85]、实值序列编辑距离（EDR）[86]以及带投影的编辑距离（EDwP）[87]。早期方法的基本思想是通过 L_p -范数对两条轨迹中的采样点进行一一对齐。例如，文献 [82] 尝试将欧氏距离直接用于轨迹相似度度量。然而，该方法对局部时间偏移非常敏感。DTW 方法通过允许多对一的点映射，利用动态规划寻找最优对齐方式，从而有效解决了局部时间偏移问题 [83]。尽管 DTW 具有较好的匹配效果，但其计算复杂度较高，不适用于大规模轨迹数据库，因此后续研究主要集中在其效率优化上。为了提升对噪声轨迹的鲁棒性，LCSS [84] 通过引入阈值机制，允许一定程度的空间偏移和未匹配点。相比 DTW，LCSS 对噪声更加鲁棒，但由于忽略了未匹配的子序列，可能会导致相似性计算精度下降。ERP [85] 基于编辑距离并引入实值惩罚，同时满足度量性质，使其能够支持高效剪枝。然而，由于直接使用实值差异作为距离度量，ERP 对噪声仍较为敏感。EDR [86] 通过根据间隙长度分配惩罚，在一定程度上弥补了 LCSS 的不足。上述方法大多依赖于点级对齐，因此对非均匀采样率较为敏感。为解决该问题，研究者提出了 APM [88] 和 EDwP [87]。其中，EDwP 通过线性插值和投影操作，有效处理了不一致的采样率问题。此外，文献 [89] 提出了基于地标的轨迹相似度模型，其结果更符合人类直觉和情景记忆；后续研究 [90] 进一步分析了传

统时间规整距离的不足，并提出了新的改进方法以应对轨迹的平移和尺度变化。

2.3.1.1.2 基于深度学习的方法 近年来，研究者开始探索基于深度学习的轨迹相似度计算方法，以克服传统基于对齐方法的局限性。Li 等人 [91] 提出了首个用于轨迹表示学习的深度学习模型 **t2vec**，通过学习轨迹的向量表示，将轨迹相似度计算转化为向量空间中的距离计算。与传统基于动态规划的方法通常具有 $O(n^2)$ 的计算复杂度不同，**t2vec** 的相似性计算复杂度为 $O(n + |v|)$ ，其中 n 为轨迹长度， $|v|$ 为嵌入向量维度。因此，**t2vec** 不仅能够有效处理低质量轨迹（如噪声轨迹或非均匀采样轨迹），还具备良好的可扩展性，能够支持大规模轨迹数据分析。随后，Yao 等人 [92] 在此基础上进一步提出了一种通用的种子引导神经学习方法，以提升深度学习轨迹相似度计算的效率。轨迹表示学习被学者们广泛研究，旨在将轨迹数据映射为向量表示，从而支持多种下游任务，例如轨迹相似度计算、轨迹分类以及行程时间估计。周等人 [93] 揭露了现有 TRL 方法在下游任务中的表现往往不够理想，主要原因在于这些方法未能充分利用轨迹所包含的多维信息。为此，提出了一种自监督的轨迹表示学习框架 **RED**。该框架以 **Transformer** 为核心模型，通过对轨迹中的路径进行掩码来训练一个掩码自编码器。具体而言，**RED** 通过道路感知掩码策略保留轨迹中的关键路径，从而更好地捕捉轨迹的运动模式并避免关键信息丢失。同时，**RED** 设计了空间-时间-用户联合嵌入，在输入阶段编码更加全面的轨迹信息。实验表明，**RED** 在轨迹相似度计算、轨迹分类以及行程时间估计等下游任务中达到了业界最好表现。

表 2-1 相似度度量方法比较

方法	局部时间偏移	噪声	非均匀采样率	阈值限制
EDB [82]	✗	✗	✗	✗
DTW [83]	✓	✗	✗	✓
LCSS [84]	✓	✓	✗	✗
ERP [85]	✓	✗	✗	✗
EDR [86]	✓	✓	✗	✗
EDwP [87]	✓	✓	✓	✓
t2vec [91]	✓	✓	✓	✗
RED [93]	✓	✓	✓	✗

表 2-1 总结了现有研究中提出的一些主流轨迹相似度度量方法。为了比较它们在处理低质量轨迹数据时的鲁棒性能力，从能否处理一下四个问题对能力进行评估：局部时间偏移、噪声、非均匀采样率以及是否为无阈值方法。值得注意的是，仅有 EDB（即基于欧氏距离的度量方法）无法处理局部时间偏移问题。在处

理含噪轨迹时, EDwP 和 t2vec 均能够取得较为理想的性能。此外, 只有 DTW 和 EDwP 属于无阈值方法, 即不需要额外设定参数阈值。

2.3.1.2 轨迹相似度搜索与连接

轨迹相似搜索旨在给定查询轨迹的情况下, 检索所有相似度不低于阈值的轨迹对象; 而轨迹相似连接则要求从两个轨迹集合中枚举所有满足相似度约束的轨迹对。该类问题的关键挑战在于: (1) 构建能够准确刻画时空行为差异的相似性度量函数; (2) 设计高效的候选过滤与精确验证机制, 以支撑大规模数据处理。在传统基于欧氏距离或时空同步距离的基础上, Shang 等人 [94] 通过线性组合的方式融合空间相似性与时间相似性, 从而增强对轨迹匹配语义的刻画能力。这类方法通常假设轨迹采样点在时间维度上可对齐或可通过插值实现同步, 并基于离散时间点进行相似性计算。Chen 等人 [95] 提出了通用轨迹连接框架, 统一支持距离连接与 k 最近邻连接。Tampakis 等人 [96] 则进一步提出子轨迹连接查询, 用于识别“最大匹配”子轨迹片段, 从而捕获局部相似行为模式。Yuan 等 [97] 从路网约束出发, 定义了基于道路网络的轨迹相似函数, 并结合过滤—精化框架与分布式内存系统, 实现负载均衡与高效分区策略。Chen 等 [98] 则引入文本语义信息, 将语义维度融入两阶段并行匹配框架, 扩展了传统轨迹相似连接至语义轨迹场景。

尽管上述研究在算法设计与系统实现方面取得显著进展, 但其共同假设是轨迹数据集为静态且预先完整存储的, 即所有轨迹在查询前均已生成且不再更新。因此, 这类方法通常采用批处理模式进行统一索引构建与连接计算。当面对持续到达的轨迹流数据或动态演化的移动对象集合时, 原有索引结构与候选集合难以高效维护, 需频繁重构或重新计算, 导致计算与存储开销显著增加。因而, 这类离线方法难以直接适用于需要实时响应与持续更新的动态轨迹分析场景。近年来, 随着移动设备与传感技术的普及, 在线轨迹相似性分析逐渐成为研究热点。与离线方法不同, 在线分析强调在轨迹数据持续到达的环境下进行增量计算与实时响应。Pan 等 [99] 采用经典空间距离度量, 如 Hausdorff 距离 [100]、动态时间规整 (DTW) [83] 以及最长公共子序列 (LCSS) [84], 并提出基于矩阵划分的处理方法以实现轨迹点的均匀分区。其算法通过滑动时间窗口识别候选轨迹对并执行相似连接。然而, 该方法在窗口滑动时需对窗口内轨迹重新计算相似度, 未能有效复用历史计算结果, 因而存在明显的冗余计算问题。Fang 等 [101] 进一步提出支持多种空间距离度量的统一框架, 通过对不同相似函数进行优化设计, 实现增量式相似度计算, 从而避免重复计算带来的性能损耗。同时, 其统一相似性定义与剪枝策略显著降低了时间开销, 提高了在线处理效率。然而, 现有在线方法大多基于固定大小的时间窗口进行计算。该机制虽然能够控制计算规模, 却在理论上引入两个潜在问

题。首先，窗口边界可能截断真实的时空交互过程，从而影响结果的完整性；其次，窗口机制通常以离散时间片为单位更新，难以支持连续语义下的细粒度实时反馈。在需要保证结果完备性与连续响应能力的应用场景中，上述方法存在明显局限。近期，丁等人 [102] 针对传统轨迹相似连接方法仅适用于离线静态数据、难以支持动态轨迹流实时处理的问题，提出了一个时间感知的在线轨迹相似连接新模型。通过引入指数衰减相似度函数，有效消除过时相似结果，从而保证连接结果的时效性与准确性。基于此，构建了一种并行在线轨迹相似连接框架，结合负载均衡、剪枝与近似优化技术，实现了在大规模轨迹流环境下的实时更新、完整评估与良好可扩展性。

2.3.2 轨迹模式挖掘

2.3.2.1 协同行为模式挖掘

与本文研究问题密切相关的方向是协同行为模式挖掘。已有研究围绕不同应用需求提出了多种具有代表性的协同行为模式 [103–106]，并在交通分析、行为建模与公共安全等应用场景中得到广泛应用。其中，flock 模式 [107, 108] 是最早被提出的协同行为模式之一，用于刻画在连续 k 个时间区间内共同移动的 m 个移动对象所构成的群体，其核心思想是在时空连续性约束下识别稳定的群体结构。Jeung 等人 [103] 提出了支持任意形状聚类的 convoy 模式。然而，flock 与 convoy 均对时间连续性提出了严格要求，即对象必须在连续时间段内保持群体结构，这在部分应用场景下显得过于苛刻。在此基础上，文献 [106] 从交通拥堵检测的角度出发，提出了 gathering 模式，用于交通拥堵的早期发现，该模式允许成员在任意时间离开群体，从而增强了模式表达的灵活性。类似地，Naserian 等人 [109] 在轨迹数据流环境下提出了一种宽松的伴随群体发现方法，以支持在线场景下的模式挖掘。然而，Shein 等人 [110] 指出上述方法普遍存在较高的计算代价，并提出了一种增量式算法以降低重复计算开销，但其整体效率仍难以满足严格的实时发现需求。为适应对持续时间约束更加宽松的需求，研究者进一步提出了 group、swarm、platoon 等多种扩展模式 [104, 111, 112]，在时间连续性与成员稳定性方面进行了不同程度的放松。

为进一步支持轨迹数据流处理，Fang 等人 [113] 构建了一个实时联合移动模式挖掘系统，在给定时空约束下实现在线检测；Chen 等人 [114] 则在分布式环境中研究实时联合移动模式检测问题，并通过基于双层索引的范围连接操作加速聚类过程，从而减少不必要的验证计算。尽管这些方法在系统层面提升了处理能力，但在模式灵活性与效率权衡方面仍存在改进空间。近年来，已有大量研究尝试利

用深度学习技术加速轨迹挖掘任务，包括轨迹聚类、异常轨迹检测 [115]、轨迹相似度计算 [91] 以及风险检测 [116] 等。然而，这些方法主要侧重于表示学习或相似性建模，并未针对联合移动模式中的组合结构约束与时间-空间共现判定机制进行专门设计。

2.3.2.2 接触模式搜索

为有效遏制 SARS、埃博拉以及 COVID-19 等传染病的传播，学术界和工业界已开展了大量相关研究 [117–119]。其中，文献 [118] 首次形式化提出了接触追踪问题，并对疾病传播过程进行了建模与模拟分析，但其主要关注传播链分析与风险评估，未能支持大规模实时查询需求。从查询处理角度出发，文献 [117] 提出了接触追踪查询，用于识别与给定查询对象存在直接或间接接触关系的个体。进一步地，文献 [27] 提出了广义轨迹接触搜索查询，并设计了一种基于迭代扩展的求解算法。另一项相关研究 [120] 则提出了一种基于轨迹片段的接触检测技术，用于追踪与患者接触的所有对象并搜索潜在感染者，但该方法主要面向室内定位场景，且对数据组织形式具有特定假设。

2.3.2.3 轨迹聚类模式挖掘

近年来，轨迹聚类模式挖掘已经演变为采用深度表示学习方法以加速轨迹相似度计算 [91, 92, 121]，其核心思想是将变长轨迹映射为固定长度的低维表示向量，从而在嵌入空间中高效完成相似性度量与近邻搜索。Yao 等人 [122] 通过滑动窗口提取轨迹的局部特征序列，并利用自编码器学习紧凑的深度表示；Boyle 等人 [123] 将训练后自编码器最小隐藏层的输出作为轨迹的表达向量，用于后续相似性分析；Fang 等人 [124] 则提出了一种基于自训练机制的端到端聚类框架，在无需人工特征工程的前提下实现轨迹表示学习与聚类优化的联合训练。这类方法在提升相似性计算效率与聚类质量方面取得了显著进展。

2.4 本章小结

在本章中，系统地调研并总结了城市计算应用中时空数据建模及索引、路径搜索与规划以及轨迹挖掘与应用相关的研究工作，并在该部分中定义了一些基本术语。基于对已有研究的调研，将有助于更好地发展新研究方向和解决方案。

尽管路径搜索与轨迹挖掘已经得到了广泛研究，认为该领域仍然存在大量亟待解决的问题以及有待改进的方法。对于基于位置的路径规划，目前针对大规模出行查询的全局路径规划研究仍然较为有限，该类问题旨在为所有用户寻找一组最优路径组合，以最小化整体交通拥堵和总出行成本。因此，如何支持面向海量

出行查询或出行查询流的高效路径规划，是一个亟需深入研究的重要问题。此外，在大规模移动对象环境中，识别群体运动模式可揭示潜在的协同行为与社会动态规律。这类模式在拼车推荐、交通拥堵预测、异常行为检测以及生态监测中具有广泛应用。高效挖掘联合移动模式的核心挑战在于对象轨迹在时间上可能暂时离群或不连续，导致候选群体数量指数级增长，需要设计高效的索引结构与过滤策略以保证算法可扩展性。再者，在公共卫生管理与疫情防控中，及时识别个体间潜在接触事件对于控制疾病传播具有关键意义。由于移动对象的轨迹数据通常存在采样间隔与定位误差，仅依赖离散采样点判断接触关系容易产生误判。面向大规模连续轨迹数据的接触搜索算法必须支持高并发、低延迟处理，并在动态更新环境下保证结果的准确性与一致性，为公共卫生决策提供数据支撑。最后，传统轨迹相似性连接方法多依赖离散采样点计算对象间距离，难以刻画未观测时间区间内的潜在交互。交集相似度连接通过引入运动范围的概念，将对象在给定时间段内可能经过的空间区域作为分析单元，采用交集相似性度量量化对象间潜在重叠关系。这一方法可应用于交通监控、野生动物迁徙分析、接触追踪等场景，在稀疏采样或不确定条件下仍能稳健地识别潜在关联。

第三章 全局持续最优路线组合规划

3.1 引言

本章研究如何针对面向大规模行程请求下的全局持续最优路线规划这一城市计算应用设计高性能算法，以缓解动态路网下的全局交通拥堵。在研究背景上，调研了现有工作在路线规划问题取得的研究进展。随后，通过分析现有工作的不足进而引出本章研究工作的研究内容和面临的研究挑战。在问题描述上，一一定义了动态路网及其构成、车辆通行时间函数、路线及其通行时间、行程请求流以及所研究的全局持续最优路线规划问题。在解决方案上，该问题的精确求解需要在指数级的路线组合空间中进行搜索，其时间复杂度随查询数量呈指数级增长，难以满足实时应用的需求。因此，如何在保证整体优化效果的前提下设计高效的近似算法，是解决全局持续最优路线组合问题的核心挑战。为解决该问题，本章提出了自感知的全局最优路线规划算法以及对应的修剪枝策略。最后，基于真实数据集开展了广泛的实验研究，实验结果验证了这些算法的准确性、高效性以及可扩展性。

3.1.1 研究背景与意义

随着汽车产业、智能移动设备以及基于位置服务的快速发展，路线规划已经成为现代社会运行的重要基础功能之一，并深刻影响着人们的日常出行方式。随着地理定位服务的不断普及，各类路线规划应用（如谷歌地图^①，高德地图^②，百度地图^③）和拼车平台（如滴滴出行^④、曹操出行^⑤等）已成为日常生活中不可或缺的重要工具。随着网络购物的兴起，物流运输的高效运行同样需要路线规划算法。不同于仅返回一组实体结果（如兴趣点或位置点）的查询方式，路线搜索关注的是满足特定约束条件的完整路线：该路线通常以给定起点为出发点，在满足多种优化目标（如时间、距离或成本）的前提下，最终到达指定终点。无论是通勤导航、网约车调度，还是物流配送与智慧交通管理，高效且智能的路线规划算法都在其中发挥着核心作用。据新华社北京 2026 年 1 月 26 日报道^⑥，公安部发布的最新统计数据显示，2025 年中国机动车保有量已经达 4.69 亿辆。如此庞大的车辆规模

① <https://www.google.com/maps>

② <https://www.amap.com/>

③ <https://map.baidu.com/>

④ <https://www.didiglobal.com/>

⑤ <https://www.caocao.com.cn/>

⑥ https://paper.people.com.cn/rmrhwb/pc/content/202601/27/content_30135944.html

对城市路网的承载能力构成了前所未有的挑战，由此带来的交通拥堵、出行效率下降以及能源浪费等问题日益凸显。在此背景下，如何通过智能路线规划技术缓解交通压力、提升整体出行效率，成为学术界与工业界共同关注的重要课题。

实际上，路线规划和行程推荐在很早以前就引起了学者们的广泛研究 [57, 68, 70, 75, 125]。上述早期路线规划的研究目标是基于当前的交通状态，为单个行程制定个人最优路线。个体最优路线规划在不同设定下已被广泛研究 [126–130]。部分研究旨在在用户指定约束（例如指定出发时间）下，根据特定优化目标（例如最短距离、最小拥堵概率）生成一条推荐路线 [46, 58, 131]。然而，这些研究提出的查询大多基于单次行程请求。此外，也有研究从全局角度出发，将问题建模为流量分配问题，在预定义路线集合上寻找最优流量分布 [53, 132, 133]。然而，这类方法本质上并非面向实时路线规划场景，其路线集合通常是静态给定的，难以适应动态变化的行程请求流。

近年来，自动驾驶技术在人工智能、传感器融合与高精度地图等关键技术的推动下实现了快速发展，并逐步渗透到实际交通场景中。这种技术演进不仅改变了车辆的控制方式，也深刻影响了人类的驾驶行为与出行模式，使传统以人为主导的驾驶方式逐步向智能化、自动化方向演进。在通勤时间等高峰时期，大量用户很有可能会在极短的时间间隔内密集地发布行程请求，从而形成了持续的行程请求流。在这种场景下，实现针对行程请求流的路线规划这一需求变得更加迫切。已经存在的相关研究旨在为行程请求中的单个行程依次制定个人最优路线 [7, 36]。尽管已有研究 [2] 提出了最小化全局出行时间的方法，但其解决方案无法用于解决持续最优化。在为行程请求流规划路线时，仅仅基于当前的交通状态以个人最优为最终目标规划静态路线，可能会导致交通拥堵。更合理的路线规划应该考虑到之前已经规划的行程会对未来的交通状态产生影响，因为它们也会增加路网中各路段的车流量。图 3-1 展示了面向个人最优路线规划下潜在的交通拥堵示例。

示例 3.1（个人最优下潜在交通拥堵示例） 如图 3-1，共有 4 个行程请求，分别记为行程 1 至行程 4；共有 6 条路线，分别记为其中每个行程均包含一个起点和一个终点，分别以圆形和菱形标记。顶点 v_1 至 v_4 分别对应行程 1 至 4 的起点，而顶点 v_5 至 v_8 则分别对应行程 1 至 4 的终点。假设行程请求 1 至 3 的发布时刻十分相近，分别为 8:00:01、8:00:01 和 8:00:05，几乎为同时发布。基于各自发布时刻的实时交通状态，为这 3 个行程请求规划得到的最优路线分别为路线 1、路线 2 和路线 3，在图中以实线表示。根据预测，这三条路线将在时间区间 [8:02:00, 8:05:00]、[8:01:40, 8:04:40] 和 [8:01:30, 8:04:30] 经过路段 $e(v_9, v_{10})$ 。可以观察到，在时间区间 [8:02:00, 8:04:30] 内，三条路线将同时占用该路段。由于路线规划过程彼此独立，未考虑其他已规划行程的未来占用情况，因此这种时间上的重叠可能导致路段



图 3-1 个人最优下潜在交通拥堵示例

$e(v_9, v_{10})$ 在未来产生交通拥堵，进而增加该路段上所有车辆的实际通行时间。随后，行程 4 于 8:00:20 发布。基于该时刻的实时交通状态，为其规划得到的最优路线为路线 4。然而，该规划过程同样未考虑先前规划路线 1 至路线 3 在未来时段对路段 $e(v_9, v_{10})$ 的潜在影响。因此，路线 4 将再次经过该路段，可能进一步加剧拥堵问题。从该示例可以观察到，在面对一组密集发布的行程请求流时，如果仍然采用逐个独立规划的策略，容易在局部最优决策的叠加下产生全局拥堵。为缓解交通压力，有必要将这些行程请求视为一个整体进行协同规划。例如，在本例中，可以首先将行程 1 至 3 作为一个整体进行联合优化，在预测未来路段负载的基础上，将行程 3 的路线由路线 3 调整为路线 5，其中路线 5 以虚线表示，以降低对路段 $e(v_9, v_{10})$ 的集中占用。进一步地，对于随后到达的行程 4，同样将行程 1 至 4 作为一个整体进行优化，可将其路线由路线 4 调整为路线 6。尽管调整后的路线对单个请求而言可能不再是各自的最短路线，即在个体层面上是“次优”的，但由于有效分散了路段 $e(v_9, v_{10})$ 的交通负载，整体交通拥堵得到缓解，从而显著降低了行程 1 至 4 的总通行时间。通过采用路线组合 {路线1, 路线2, 路线5, 路线6}，行程 1-4 的总体通行时间减少。将能够使全局总的通行时间最少的这种路线组合称为全局最优路线组合。

综上，当多个行程在相近时间内发布并分别进行独立规划时，可能在未来某一时间区间内集中占用同一路段，从而导致拥堵加剧。若后续行程仍然基于当前状态进行规划而不考虑已有规划结果的潜在影响，则会进一步恶化交通状况。因此，在面对持续到达的行程请求流时，有必要将新到达的请求与已规划路线进行协同考虑，从整体角度优化路线组合，而非仅追求单个请求的即时最优。基于上

述背景，本章围绕持续行程请求流场景下的协同路线规划问题展开研究，旨在动态路网环境中实现更具前瞻性的整体优化。

3.1.2 研究内容与挑战

基于以上观察，本章的研究目标为：开发适应城市动态变化的智能路线规划算法，以实现交通流量平衡，减少拥堵，提高效率。主要目标是开发一种高效的算法，为大规模交通通行提供最优路线规划，以减轻城市交通拥堵。次要目标包括提高路网利用效率，减少交通通行时间，以及提高交通系统的整体性能。为实现该目标，本章创新性地提出并研究了一种持续的面向行程请求流的全局最优的路线规划问题：在一个动态路网上，行程请求流中的行程请求数是不断增加的，需要找到一个路线组合使得所有在该路线组合中的路线的总的通行时间最小。其中，每一条路线是为每一个具体的行程请求对应规划的。在本研究的设定中，每条边上的实通行时间与当前位于该边上的车辆数量成正比。该问题具有十分广泛的应用场景，包括但不限于城市交通流量管理，实时的路线规划以及持续的拥堵预防上。

与传统单次最优路线规划问题不同，全局持续最优路线组合问题具有显著的持续性、动态性与全局耦合性等特征。首先，行程请求以数据流形式持续到达，系统需要在不断扩展的查询集合上进行在线优化，而非一次性处理静态输入；其次，早期规划结果会改变未来时刻各路段的车流分布，从而影响后续行程的路线选择，使得决策之间具有时间上的传递性；最后，不同行程路线通过共享路段产生资源竞争关系，使得各路线之间形成复杂的耦合结构，从而显著扩大了组合搜索空间。上述特性使得该问题不仅是一个路线搜索问题，更是一个具有强依赖性的动态组合优化问题。高效且有效地解决该问题面临以下关键挑战：

- **大规模性：**在实际应用中，行程请求可能达到百万级甚至更高规模。系统需要在大规模请求集合与复杂路网结构下进行统一规划，同时考虑车辆数量变化对通行时间的动态影响。
- **实时性：**动态路网中的交通状态（如车流密度、拥堵程度）随时间持续变化。算法必须能够在极短时间内响应路网状态更新，并为新到达的行程请求生成可行路线，以满足实时交互需求。
- **准确性与效率的平衡：**单纯追求全局最优往往需要高昂的计算代价，而过度简化模型又可能导致严重的拥堵问题。因此，需要在整体通行时间优化效果与算法运行效率之间取得合理平衡。
- **连续查询流的处理：**行程请求以流式形式持续到达，系统需要在已有规划结果的基础上进行增量优化，而非每次从头计算。这要求算法具备良好的在线处理能力与历史信息复用机制。面对持续增长的查询规模与复杂动态

约束，算法应具有良好的可扩展性，能够随着数据规模扩大而保持稳定性能表现。

- **动态路网的适应性：**路段通行时间随车辆数量动态变化，算法需具备对未来交通负载的预测与感知能力，从而在规划阶段主动规避潜在拥堵，而非被动响应。

更为重要的是，从理论角度看，全局持续最优路线组合问题本质上需要在指数级规模的路线组合空间中进行搜索。随着行程请求数量的增加，可行的路线组合候选数目呈指数级增长，其精确求解的时间复杂度难以控制在可接受范围内，因而无法满足实际应用中的实时性需求。已有研究 [2] 尝试在全局层面最小化一组行程的总出行时间成本，但其假设输入为静态行程集合，并未考虑在当前时间之前已经规划的路线对未来交通状态的持续影响。因此，该方法难以直接扩展到持续到达的行程请求流场景。在高频行程发布场景下，若忽略历史路线的累积影响，系统容易在局部最优决策的叠加下形成新的拥堵。因此，如何在保证整体优化效果的前提下，设计具有可扩展性和实时性的高效近似算法，在动态路网环境中持续优化路线组合，是解决全局持续最优路线组合问题的核心挑战，也是本文研究的重点。

3.1.3 研究成果与贡献

该问题由于计算开销高而极具挑战性。精确算法的时间复杂度随查询数量呈指数级增长，因此无法在用户可接受时间内找到最优的路线组合。为实现高效率，本章提出了一种自感知的批处理算法及相应的剪枝技术。除当前时刻之前由规划路线所引起的与查询相关的交通流，以及与查询无关的交通流之外，还考虑了由初始路线规划所带来的潜在交通流。本文的主要贡献总结如下。

- 提出了全局持续最优路线组合规划问题，面向多种应用场景，包括交通流管理、实时路线规划和拥堵预防。
- 设计了一种自感知的批处理算法以高效求解。该算法包含两个主要步骤：初始路线搜索和批量优化处理。对于一批新到达的行程请求，首先执行初始路线搜索为每个新到达的行程请求生成一组高质量的初始路线组合。该步骤有助于在初步阶段避免部分交通拥堵。为了进一步减少所有行程请求的总出行时间并缓解交通拥堵，以批处理模式对最近发布的请求集合执行批量优化处理，对初始路线组合中的路线进行进一步优化。
- 在两个真实数据集进行了大量实验来评估所提方法的准确性和效率。实验结果表明，提出的方法优于基线方法，能够有效处理行程查询流。与个人最优路线规划算法相比，所提出方法能够降低超过 40% 的总通行时间。

3.2 问题描述

本章定义了动态路网及其构成、车辆通行时间函数、路线及其通行时间、行程请求流以及所研究的全局持续最优路线规划问题。本章所使用的主要符号及其解释如表 3-1 所示。

表 3-1 主要符号解释

符号	解释
t	某个时刻
v	路网上的一个顶点
V	路网上的顶点集合
e	路网上的一条边
E	路网上的边集合
$G = (V, E)$	动态路网及其构成
C_e	边 e 的车容量限制
$f(e, t)$	在时刻 t 经过路段 e 的车流量函数
$T_m(e)$	车辆在边 e 的最短通行时间
$T(e, t)$	车辆在时刻 t 经过路段 e 的通行时间函数
π	一条由有限的顶点序列构成的路线
Π	一个包含多条路线的路线集合
$ET(\pi, t_0)$	一条路线 π 的预计通行时间, 即路线 π 中每个路段的预计通行时间之和
$q = (v_s, v_d, t_s)$	一个行程请求, 其中 v_s 是起点位置, v_d 是终点位置, t_s 是出发时间
Q	包含多个行程请求的一组行程请求流
T	批处理时间间隔
$TT(\Pi)$	所有已规划路线的总的通行时间

定义 3.1 (动态路网) 一个动态路网 $G = (V, E)$ 是由顶点集 V 和边集 E 构成的, 其中 $E \subseteq V \times V$, 顶点代表路段的连接点, 交叉点; 连边则代表具体的路段。每条边 $e(v_i, v_j) \in E$ 连接着两个顶点 v_i 和 v_j , 这里 $v_i, v_j \in V$ 。为了便于描述, 假定所有行程请求的起点和终点位置都已经被映射到离它们最近的顶点上。针对每个路段 e , 用 C_e 来代表该路段的车容量, 用 $T_m(e)$ 代表在路段 e 的最小通行时间, 通常为该路段为空车状态时的通行时间。在本研究的设定中, 每一个路段的通行时间是动态变化的。一个路段的最小通行时间是 $T_m(e)$, 而实际的通行时间是和该路段上的动态车流量成比例的。

定义 3.2 (车辆通行时间函数) 车辆在时刻 t 经过路段 e 的通行时间和当前时刻路段 e 上的车流量 $f(e, t)$ 相关。参照已有研究 [2, 53], 基于当前时刻路段 e 上的车流量, 使用式 3-1 来计算车辆在时刻 t 进入路段 e 的通行时间, 用 $T(e, t)$ 表示。其中, C_e 代表路段 e 的车容量限制, α 和 β 是由路段的不同性质 (例如, 限速, 路

宽和道路属性) 决定的可变参数。由于本章的解决方案是和这些参数独立的, 因此如何为这些参数设置合适的值并不在研究范围内。由式 3-1 可以知道, 随着车辆数目的动态变化, 每条路段的通行时间也会随之发生改变。

$$T(e, t) = T_m(e) \times (1 + \alpha \times (\frac{f(e, t)}{C_e})^\beta) \quad (3-1)$$

定义 3.3 (路线及其通行时间) 一条路线 π 是一个有限的路网顶点序列, 形如 $\langle v_0, v_1, \dots, v_{|\pi|-1} \rangle$ 。假定路线 π 出发时间为 t_0 , t_i 是其经过的顶点 v_i 的预计到达时间, 即 $t_i = t_{i-1} + T(e(v_{i-1}, v_i), t_{i-1})$ 。结合式 3-1, 用式 3-2 来计算一条路线 π 的预计通行时间, 即路线 π 中每个路段的预计通行时间之和, 用 $ET(\pi, t_0)$ 来表示。本章接下来将使用 $\pi.v_0$ 和 $\pi.v_{|\pi|-1}$ 来分别代表一条路线 π 对应的起点和终点。

$$ET(\pi, t_0) = \sum_{i=0}^{|\pi|-2} T(e(v_i, v_{i+1}), t_i) \quad (3-2)$$

定义 3.4 (行程请求流) 一个行程请求表示为 $q = (v_s, v_d, t_s)$, 其中 v_s 是该行程的起点位置, v_d 是该行程的终点位置, 而 t_s 是该行程的出发时间。而一组行程请求流 $Q = \{q_1, q_2, \dots\}$ 是由多个行程请求组成, 即 $\forall q_i, q_j \in Q, i > j: q_i.t \geq q_j.t$ 。需要注意的是, Q 是以流数据的形式到达的行程请求集合并且 Q 是动态更新的。

定义 3.5 (全局持续最优路线组合问题) 在一个动态路网 $G = (V, E)$ 上, 给定一组行程请求流 $Q = \{q_1, q_2, \dots\}$ 和时间间隔 T , 全局持续最优路线组合问题旨在不断地处理最近到达的一批行程请求 $Q_n = \{q_k, q_{k+1}, \dots, q_{|Q|}\}$ ($Q_n \subset Q$ 且 $q_{|Q|}.t - q_k.t = T$), 找到一个最优的路线组合 $\Pi = \{\pi_1, \pi_2, \dots\}$ 使得: (1) $|Q| = |\Pi|$; (2) $(\forall \pi_i \in \Pi) (\pi_i.v_0 = q_i.s \wedge \pi_i.v_{|\pi_i|-1} = q_i.d)$; (3) 在 Π 中规划的所有路线的总的通行时间最小。路线组合 Π 中所有路线的总通行时间由公式 3-3 计算, 表示为 $TT(\Pi)$ 。

$$TT(\Pi) = \sum_{i=1}^{|\Pi|} ET(\pi_i, q_i.t) \quad (3-3)$$

3.3 精确遍历算法

针对全局持续最优路线组合问题, 一个直接办法是对每一个行程请求, 搜索起点和终点间可能存在的所有路线, 然后对所有的路线组合进行遍历评估, 比较各个路线组合所有路线总的通行时间, 最终筛选出具有最小通行时间的路线组合。具体来说, 对于每一个行程请求 $q_i \in Q$, 可以采取深度优先算法来查找从起点 s_i 到终点 d_i 可能存在的所有路线。其中, 可以应用一些剪枝策略来加速深度优先搜索。随后, 对于每一个路线组合计算它所有路线总的通行时间。最终, 返回具有最

小通行时间的路线组合作为返回结果。精确算法比较简单直接，便于理解。当行程请求数量较小时，精确算法可以在有限时间内得到最优解。但是，当行程请求数量很大时，精确算法会消耗大量的计算时间，不能运用到实际的应用场景中。

3.4 自感知的批处理算法

为了高效解决全局持续最优路线组合问题，提出了一个自感知的批处理算法，该算法主要包含两大组成部分：初始路线搜索和批量优化处理。具体来说，给定一组行程请求流，首先采用初始路线搜索生成一个高质量的初始路线组合，该组合包含了为该行程请求流中的当前处理批次中每个请求指定的初始路线。接下来，执行批量优化处理对初始路线组合中的每个初始路线进行优化。最终，将优化后的局部路线组合加入到全局路线组合中去。每当处理完一批行程请求后，返回全局路线组合作为实时的规划结果。自感知思想合理考虑了新规划的路线会影响未来的交通状态这一事实，即规划的路线会增加一些路段的车流量从而影响该路段的实际通行时间。

3.4.1 初始路线搜索

初始路线搜索的目标是为新一批到达的行程请求流中的每个行程请求规划一条高质量的个人路线。每个路段的动态车流量由两部分组成的，一部分是在规划系统中的请求车辆所造成的交通流量，即规划的路线在各路段上造成的车流量，称之为系统内的车流量，一部分是系统外的车流量即在各路段上没有使用规划系统的车辆数目，把系统外的车流量直接作为算法输入。

3.4.1.1 路段标签

为了实现对系统内车流量的快速计算，在每个路段上维持一系列的路段标签 $L_e = \{l_1, l_2, \dots\}$ 。这些路段标签记录了经过该路段的路线在该路段的时间信息。每个路段标签 $l_i = \{t_a, t_b\}$ 记录了一条特定的路线进入路段 e 的时间 t_a 以及离开该路段的时间 t_b 。刚开始没有行程请求的时候，每个路段维持的路段标签是一个空集。之后每当处理完一批新的行程请求集合的时候，对应路段的路段标签会进行动态地更新。

3.4.1.2 系统内的车流量计算

基于路段标签，可以快速地计算在各个路段上由规划系统规划的路线所产生的车流量。具体来说，由规划系统规划的路线在路段 e ，时刻 t 所额外造成的车流量，也就是满足 $t \in [l_i.t_a, l_i.t_b]$ 的路段标签 l_i 的数量，其中 $l_i \in L_e$ 。

3.4.1.3 通行时间下界

从一个顶点 v_i 到另一个顶点 v_j 的通行时间下界用一个启发值 $H(v_i, v_j)$ 来表示。给定最小的系统外的车流量作为静态的车流量输入，该下界是由 Dijkstra 算法提前计算并存储备用的。将非请求车辆的车流量直接作为算法输入。

3.4.1.4 算法描述

算法 3-1: 初始路线搜索

```

输入: 1) 动态路网  $G = (V, E)$ ;
        2) 行程请求  $q = (v_s, v_d, t_s)$ ;
        3) 所有两点间的通行时间下界  $H(v_i, v_j)$ ;
        4) 规划路线的路段标签集合  $\mathcal{L}$ 。
输出: 为行程请求  $q$  制定的一条初始化路线  $\pi$ 。
1 初始化:  $\forall v \in V: v.t_s = \infty, v.t_d = H(v, v_d), v.et = t_s$ ;
    $v_s.pred = null, v_s.t_s = 0; \pi = \emptyset$ ;
2  $PQ \leftarrow \{v_s\}$ ;
3 while  $PQ \neq \emptyset$  do                                     /* 主搜索循环 */
4      $v \leftarrow PQ.pop()$ ;
5     if  $v = v_d$  then                                       /* 到达目的节点 */
6         while  $v_d.pred \neq null$  do
7              $\pi \leftarrow \{v_d, \pi\}$ ;
8              $v_d \leftarrow v_d.pred$ ;
9         return  $\pi$ ;
10    for  $e(v, v') \in G$  do                                   /* 遍历邻接路段 */
11        计算当前时刻  $v.et$  下路段  $e$  的通行时间  $T(e, v.et)$ ;
12        if  $v.t_s + T(e, v.et) \leq v'.t_s$  then
13             $v'.t_s \leftarrow v.t_s + T(e, v.et)$ ;
14             $v'.et \leftarrow t + v'.t_s$ ;
15             $v'.pred \leftarrow v$ ;
16            if  $v' \notin PQ$  then
17                 $PQ.push(v')$ ;
18 return  $\pi$ ;
    
```

算法 3-1展示了初始路线搜索的伪代码。算法输入包含一个动态路网 $G = (V, E)$ ，一个行程请求 $q = (v_s, v_d, t_s)$ ，所有顶点间提前计算好的通行时间下界查询表 $H(v_i, v_j)$ 以及所有路段的路段标签集合 $\mathcal{L} = \{L_{e_1}, L_{e_2}, \dots\}$ 。算法输出是为该行程请求制定的一条初始路线。

每个顶点包含以下记录信息：从起点 v_s 到达该顶点的精确通行时间 t_s ；从该

点到达终点的通行时间下界 t_d ；从起点到达该点的最小时刻 et 以及一个前驱节点记录 $pred$ 。首先，初始化每一个顶点 $v \in V$ 的信息记录：从起点 v_s 到达该点的精确通行时间 t_s 为无穷大；从该点到达终点的通行时间下界 t_d 由提前计算的表 $H(v_i, v_j)$ 中直接索引；从起点到达该点的最小时刻 et 为出发时刻。接下来，初始化起点的前驱节点为空，从起点到达起点的精确通行时间为 0。初始化路线 π 是一个空集 (第 1 行)。

算法正式开始前，把起点 v_s 加入一个优先队列 PQ 中。该优先队列 PQ 内部对顶点对象按照该点的 $v.t_s + v.t_d$ 的值从小到大排序，队头元素为具有最小 $v.t_s + v.t_d$ 的顶点 (第 2 行)。在搜索过程中，每一次从 PQ 选出队头元素 v 并探索它的邻接顶点。具体来说，每一次选出的 v 都具有最小的 $v.t_s + v.t_d$ ，之所以这样选择，是基于这样的顶点扩张可能会产生最小的通行时间 (第 4 行)。如果选择的 v 是行程请求的终点 v_d ，就利用相应节点的前驱节点记录生成一条路线 π ，并将 π 作为结果返回 (第 5–9 行)。如果选择的 v 不是行程请求的终点 v_d ，对于每一个从顶点 v 出发的路段 e ，计算当前时刻该路段的车流量 (第 11 行)。对于路段 $e(v, v')$ 的结束点 v' ，需要检查该点能否通过顶点 v 到达，并且所花的时间比从其他点到达该点更小 (第 12 行)。如果满足，将分别更新顶点 v' 对应的 t_s, et 和 $pred$ 记录 (第 13–15 行)。接下来，如果顶点 v' 不在优先队列 PQ 中，将其加入 (第 17 行)。当优先队列 PQ 为空或者已经扩张到行程请求对应的终点时，初始路线搜索算法终止。最终，返回 π 作为初始化路线 (第 18 行)。

3.4.2 批量优化处理

初始规划路线能够在一定程度上优化个人的通行时间，但并没有考虑短时间内后续行程的影响。为了提高自感知能力，提出了一个批量优化处理方法对算法 3-1 产生的初始路线进行优化。一个初始路线组合 Π_n 包含了为最近到达的行程请求 Q_n 制定的初始路线。不同于 Π_n 的是，全局的路线组合 Π 包含了所有在当前时间之前已经规划的路线。优化处理过程重新利用了初始路线组合 Π_n ，并通过一种自感知的批处理交换策略对 Π_n 中的每一条初始路线进行优化。批量优化处理的精髓在于：在每一个优化过程中，选择一条路线 $\pi_i \in \Pi_n$ 。将除了该路线的其他正在规划的初始路线作为已知的路线规划输入到交通流量信息中，重新预测未来的交通状态并尝试重新分配一条新的路线 π'_i 给行程 q_i 。用 π' 对原始地初始路线 $\pi \in \Pi_n$ 进行交换，该交换操作可以用 $\Pi_n.swap(\pi, \pi')$ 表示。反复执行提升程度较高的有效路线交换。当路线组合 Q_n 中不存在有效交换操作，不能再产生一个更好的规划结果，批量优化处理算法终止。在批量优化处理中，本章也提出了一些预检查策略来进一步提高效率。

3.4.2.1 交换操作

在批量优化处理过程中，除了考虑系统外车流量和系统内已经规划的路线产生的车流量外，进一步考虑了当前正在规划的初始路线可能对未来交通产生的潜在影响。一个新的路段标签集合 $\mathcal{L}' = \{\mathcal{L}, \mathcal{L}(\Pi_n \setminus \pi_i)\}$ 是由原有的路段标签 $\mathcal{L}$ 以及预估的正在规划的初始路线的时间信息联合组成的 (参照算法 3-1)。把 $\mathcal{L}'$ 作为已知的交通信息输入，再次执行算法3-1为行程 q_i 分配一条新的路线 π'_i 。尝试使用新的路线 π'_i 交换原有的路线，如果该交换操作能够使所有行程的通行时间之和减少至少 $1 + \epsilon$ 的比例，即 $\frac{TT(\{\Pi, \Pi_n\})}{TT(\{\Pi, \Pi_n.swap(\pi, \pi')\})} > 1 + \epsilon$ ，认为该操作是有效的并采用该操作去更新原有的初始路线。其中，采用一个极小的常数值 ϵ 排除一些“无效”的操作，这里无效操作指通过该操作只能减少很小的通行时间。此外， ϵ 的使用还保证了交换操作的次数是有限的。实际上，有效的交换操作的次数是和 ϵ 成反比的。具体来说，最大的有效交换操作的次数是 $\log_{(1+\epsilon)} \frac{TT}{TT_m}$ 。其中 TT 是当前规划的初始路线组合的总的通行时间，而 TT_m 是理想的最小通行时间。

3.4.2.2 预检查策略

为了检查一个交换操作的有效性，必须更新所有路段的路段标签去计算所有行程总的通行时间的降低率。因此，提出了一个预检查策略进一步提高算法效率。具体来说，定义了一个所有行程总的通行时间的降低率上界 UB ，计算如下式3-4。注意 $TT(\{\Pi, \Pi_n \setminus \pi\}) \neq TT(\{\Pi, \Pi_n\}) - ET(\pi, t)$ ，在这里 $TT(\{\Pi, \Pi_n \setminus \pi\})$ 是将路线 π 剔除，假设该路线不存在下的所有路线的总的通行时间。不仅要剔除路线 π 本身的通行时间，也要消除路线 π 对其他行程的影响，因为路线 π 造成了额外的交通流量，会对其他路线的通行时间造成一定的影响。

$$UB(\pi) = \frac{TT(\{\Pi, \Pi_n\})}{TT(\{\Pi, \Pi_n \setminus \pi\})} \quad (3-4)$$

3.4.2.3 算法描述

算法3-2提出了批量优化处理算法的伪代码。算法输入包含一个动态路网 $G = (V, E)$ ，全局路线组合 Π ，初始路线组合 Π_n ，路段标签集合 $\mathcal{L}$ 以及提升参数 ϵ 。算法输出是优化后的路线组合 Π_n 。初始地，使用一个标志 **flag** 来标记在上一次“while”循环中是否存在有效的操作，算法开始设置其为 **true** (第 1 行)。在优化过程中，对于每一条初始路线 $\pi \in \Pi_n$ ，检查它能否被一条新的路线 π' 替换 (第 4-10 行)。具体来说，首先更改标志 **flag** 为 **false**，假设在本轮循环中不存在有效的交换操作 (第 3 行)。对于每一条路线 $\pi \in \Pi_n$ ，通过计算提升上界 $UB(\pi)$ 来预检查该交换操作的

算法 3-2: 批量优化处理

```

输入: 1) 动态路网  $G = (V, E)$ ;
        2) 全局路线组合  $\Pi$ ;
        3) 初始路线组合  $\Pi_n$ ;
        4) 路段标签集合  $\mathcal{L}$ ;
        5) 提升参数  $\epsilon$ 。
输出: 优化后的路线组合  $\Pi_n$ 。
1  初始化:  $flag \leftarrow true$ ;
2  while  $flag$  do                                     /* 批量迭代优化 */
3       $flag \leftarrow false$ ;
4      for  $\pi_i \in \Pi_n$  do                             /* 遍历当前路线集合 */
5          if  $UB(\pi_i) > (1 + \epsilon)$  then
6               $\mathcal{L}' \leftarrow \{\mathcal{L}, \mathcal{L}(\Pi_n \setminus \pi_i)\}$ ;
7               $\pi' \leftarrow \text{初始路线搜索}(G, q_i, H, \mathcal{L}')$ ;
8              if  $\Pi_n.swap(\pi_i, \pi')$  then             /* 检查交换操作是否有效 */
9                   $\Pi_n \leftarrow \Pi_n.swap(\pi_i, \pi')$ ;
10                  $flag \leftarrow true$ ;
11 return  $\Pi_n$ ;

```

有效性。如果满足上界满足, 生成一个新的路段标签 $\mathcal{L}'$ (第 6 行)。将 $\mathcal{L}'$ 作为输入, 基于更新的车流量再次执行初始路线规划算法 3-1 产生一条新的路线 π' (第 7 行)。如果该交换操作有效, 将采用这次交换操作并设置标志 $flag$ 为 $true$ 来记录当前循环存在有效的交换操作 (第 8–10 行)。当一轮循环, 如果对所有的 $\pi \in \Pi_n$ 都没有检测到有效的交换操作, 算法终止。这表明已经没有能够产生更好结果的有效操作存在。最后, 批优化算法返回当前优化后的路线组合 Π_n 作为结果 (第 11 行)。

3.4.3 总体解决方案

通过整合上文提出算法, 本章提出针对全局持续最优路线组合问题的总体解决方案即持续最优路线组合算法。持续最优路线组合算法执行流程如下: 对于每一个新到达的行程请求, 先执行初始路线搜索, 即算法 3-1 为其分配一条初始个人路线。接着执行算法 3-2 周期性的批优化最近规划的初始路线, 这些初始路线是为最近一段时间发布的行程请求制定的。一旦有新的行程请求发布, 持续最优路线组合算法将不断执行上述操作。

3.4.3.1 批优化处理间隔

在本研究的设定中, 批优化算法是以固定的时间间隔周期性执行的 (例如, 10 秒)。用 T 表示每两次批量优化处理之间的时间间隔。通过利用这样一个时间间隔,

合理考虑了更多的正在规划的初始路线对未来交通的潜在影响，在这种情况下能更准确地预测未来的交通状态，从而通过一次交换操作也许可以减少更多的通行时间，整个优化过程中总的交换次数也会减少，算法效率也因此得到了提高。

算法 3-3: 持续最优路线组合

```

输入: 1) 动态路网  $G = (V, E)$ ;
        2) 行程请求流  $Q$ ;
        3) 优化时间间隔  $T$ ;
        4) 提升参数  $\epsilon$ 。
输出: 实时的最优路线组合  $\Pi$ 。
1  初始化:  $\mathcal{L} \leftarrow \emptyset$ ;  $\Pi \leftarrow \emptyset$ ;  $\Pi_n \leftarrow \emptyset$ ;
2  while true do /* 系统持续运行 */
3      for  $q \in Q$  do /* 处理到达的行程请求 */
4           $\pi \leftarrow$  初始路线搜索( $G, q, H, \mathcal{L}$ );
5           $\Pi_n \leftarrow \{\Pi_n, \pi\}$ ;
6      if 当前时刻 mod  $T = 0$  then /* 周期性批量优化 */
7           $\Pi_n \leftarrow$  批量优化处理( $G, \Pi, \Pi_n, \mathcal{L}, \epsilon$ );
8           $\Pi \leftarrow \{\Pi, \Pi_n\}$ ;
9           $\mathcal{L} \leftarrow \{\mathcal{L}, \mathcal{L}(\Pi_n)\}$ ;
10          $\Pi_n \leftarrow \emptyset$ ;
11         return  $\Pi$  /* 实时输出结果 */
    
```

3.4.3.2 算法描述

算法 3-3 提出了针对全局持续最优路线组合问题的总体解决方案，即持续最优路线组合算法的伪代码。算法输入包含一个动态路网 $G = (V, E)$ ，行程请求流 Q ，优化时间间隔 T 以及提升参数 ϵ 。算法输出实时更新的路线组合 Π_n 。算法开始，规划系统中没有接收到任何行程请求，设置路段标签 $\mathcal{L}$ 和全局的路线组合 Π 都是空集。全局的路线组合 Π 记录了所有已经规划完毕的行程的路线。也使用了一个路线组合 Π_n 来记录为新的一批行程 Q_n 规划的初始路线，初始的 Π_n 也是空集 (第 1 行)。首先执行初始路线规划算法 3-1 为每一个新到达的行程 $q \in Q$ 请求分配一条初始化路线 (第 3-5 行)。接着持续地为新到达的请求生成初始路线直到下一次批量优化处理。在批量优化处理过程中，执行批优化算法 3-2 去生成一个优化后的路线组合 Π_n (第 7 行)。接下来，将这些优化后的路线加入全局的路线组合 Π 中，并根据新规划的路线对所有路段的路段标签记录进行更新 (第 8-10 行)。每一个优化处理完成后，由于所有的初始路线已经被优化了，设置 Π_n 为空并继续用于记录下一批到达的行程的初始路线。每当处理完一批新的行程请求流，输出全局最优的路线组合 Π 作为实时的规划结果 (第 11 行)。

3.5 实验研究

在本章中，首先介绍实验设定，包括实验所使用的数据集，如何实时计算车流量，算法评估以及运行环境等，然后给出实验结果与分析。实验结果主要关注所提出的搜索算法的运行时间，即高效性方面的表现；以及全局通行时间的比较，即算法有效性方面的表现。实验的目的在于，通过设置不同的参数(路线长度，位置点密度，空间-文本相似度阈值等)，对比不同的算法的运行时间和规划结果并进行分析。

3.5.1 实验设定

3.5.1.1 数据集

使用两个路网数据集用于实验研究：San Joaquin County Road Network (TG 路网)^①和 the New York Road Network (NY 路网)^②。用邻接表进行路网图的存储。对于每个路段连边，基于该路段的长度分配一个数值为 20–100 间的车容量 C_e 。每个路段的最小通行时间 $T_m(e)$ 是随机生成的，位于 5–10 分钟之间。

为了模拟系统外非请求车辆，假设每个路段 e 上的非请求车辆的平均车流量 $\bar{x}$ 为 $0.4 \times C_e$ ，实时的非请求车辆在平均车流量附近周期性地动态变化 $0.8\bar{x}$ 到 $1.2\bar{x}$ 。给定最小的非请求车流量 $0.8\bar{x}$ 作为所有路段的静态车流量，提前计算好所有顶点两两间的通行时间下界，该下界即算法3-1中的启发值 $H(v_i, v_j)$ 。在路网较为密集的区域，随机采样起点-终点对来模拟行程请求。为了模拟持续到达的行程请求流，给定第一个行程的出发时间 $q_1.t$ ，接下来的第 i 个行程的出发时间 $q_i.t$ 在上一个行程请求的出发时间 $q_{i-1}.t$ 的基础上小幅度增加或保持不变。

表3-2列出了评估参数的范围设定。

表 3-2 评估参数设定

参数	TG 路网	NY 路网
行程请求数目	4,000-20,000 (默认 4,000)	2,000-10,000 (默认 2,000)
提升参数 ϵ	0.0002-0.0010 (默认 0.001)	0.0002-0.0010 (默认 0.001)
优化时间间隔 T	1s-5s (默认 2s)	1s-5s (默认 2s)
行程到达速率	20/s-100/s (默认 50/s)	20/s-100/s (默认 50/s)
α (参照式 3-1)	1-5 (默认 2)	1-5 (默认 2)
β (参照式 3-1)	1-3 (默认 2)	1-3 (默认 2)

① <https://www.cs.utah.edu/~lifeifei/SpatialDataset.htm>

② <https://publish.illinois.edu/dbwork/open-data>

3.5.1.2 车流量计算

在时刻 t 路段 e 实时的车流量 $f(e, t)$ 是由式 3-5 计算的。其中, $f'(e, t)$ 代表系统外的非请求车辆的车流量, 而 $f''(e, t)$ 代表系统内请求车辆的车流量。为了模拟非请求车辆, 假设 $f'(e, t)$ 是随时间周期性动态变化的。如何计算系统内请求车辆的车流量已经在算法 3-1 中给出。

$$f(e, t) = f'(e, t) + f''(e, t) \quad (3-5)$$

3.5.1.3 算法评估

表 3-3 比较算法

算法标识	算法描述
SBP	持续最优路线组合算法
Ind	基于个人最优的搜索算法
SBP*SA	初始路线搜索阶段未采用自感知思想的持续最优路线组合算法
SBP*DA	初始路线搜索阶段未考虑路网动态性的持续最优路线组合算法

表 3-3 列举了实验研究中的比较算法。主要评估了自感知的批处理算法, 即持续最优路线组合算法的算法表现, 表示为 SBP。为了评估该算法的结果质量, 额外实施了一个基于个人的搜索算法, 表示为 Ind 算法。具体来说, 给定一组行程请求流 Q , 对于每一个行程请求 $q \in Q$, Ind 算法基于单个行程的出发时刻的交通状态生成一条最优的个人最优路线, 这和一些已经存在的研究相符 [7, 36]。由于精确算法在默认设定下需要至少一天的时间, 因此不研究精确算法的表现。持续最优路线组合算法由两部分组成: 初始路线搜索和批量优化处理。其中初始路线搜索合理考虑了自感知的思想和路网动态性, 为了验证其有效性, 分别实施了 SBP*SA 算法和 SBP*DA 算法。其中, SBP*SA 算法是初始路线搜索没有采用自感知思想的持续最优路线组合算法, 即它没有考虑由规划系统规划的路线对交通流量造成的额外影响。SBP*DA 算法是没有考虑网络动态性的持续最优路线组合算法, 它在初始路线搜索过程中面向的是一个静态的交通状态快照即一个行程的出发时刻所对应的静态交通状态。使用 CPU 运行时间和总通行时间作为算法效率和结果有效性的评估指标。除非特别申明, 实验结果是超过 20 次独立实验的平均值结果。

3.5.1.4 运行环境

所有的算法都是在 Windows 10 平台上运行的, 采用 Java 编程, 使用 Intel(R) Core(TM) i5-9300H Processor (2.40 GHz) 的处理器和 16GB 的内存。

3.5.2 实验结果与分析

3.5.2.1 行程请求数目的影响

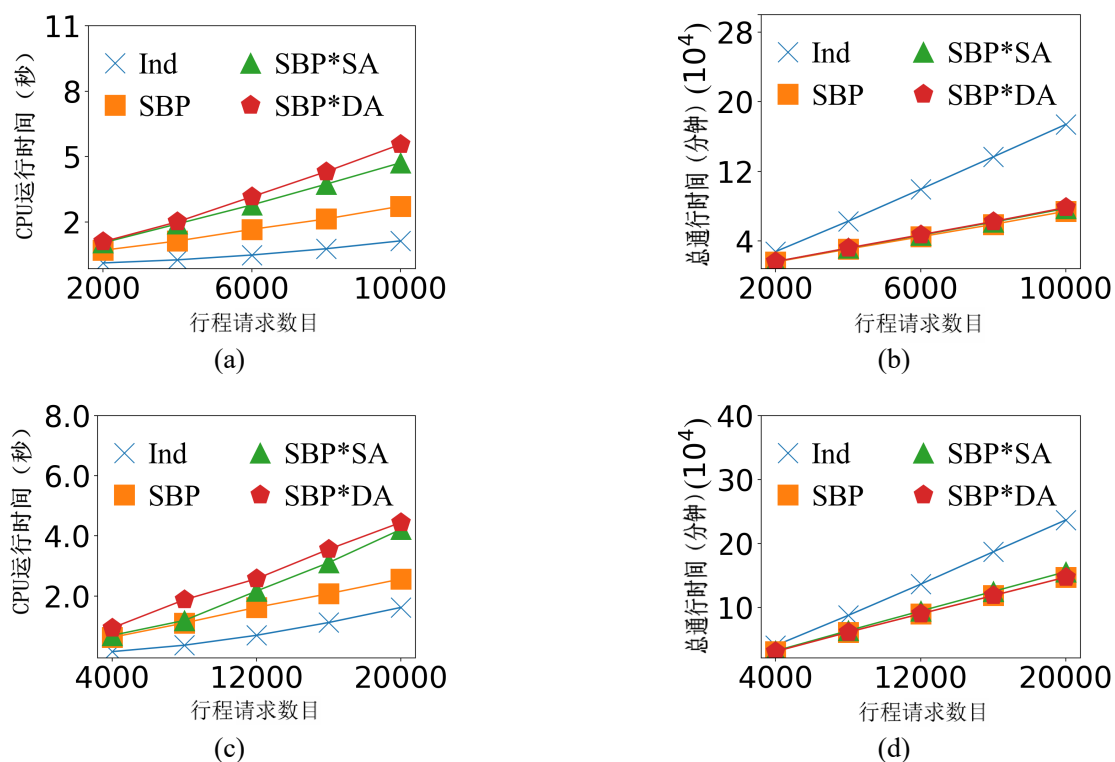


图 3-2 行程请求数目对算法运行时间和全局通行时间的影响。(a) NY 路网下的算法运行时间；(b) NY 路网下的总通行时间；(c) TG 路网下的算法运行时间；(d) TG 路网下的总通行时间

首先，在默认参数设定下，考察了变化行程请求数目对算法性能的影响。随着系统中需要同时处理的请求数量上升，扩大了批优化的搜索空间，也增强了路线之间的耦合效应，从而增大计算开销并降低整体通行效率。图3-2展示了提出的算法在两个路网上的表现。所有算法的运行时间均随着行程请求数目的增加而单调上升，且整体呈近似线性增长趋势。这表明各方法在请求规模扩展时具有良好的可扩展性。在运行开销方面，Ind 算法始终具有最低的运行时间，这是因为该方法对每个请求独立进行路线规划，不涉及跨请求的批量协调优化，因此避免了额外的组合计算开销。相比之下，SBP、SBP*SA 与 SBP*DA 由于引入批处理与持续优化机制，计算成本略有增加，但增长趋势平稳，整体仍处于可接受范围内。

然而，在总通行时间指标上，Ind 算法明显劣于其他批优化方法，且随着行程请求数目的增加，这种差距进一步扩大。当请求规模较小时，不同路线之间的冲突有限，个体最优与全局最优之间的差异相对较小；而当行程请求数目增大时，大量

行程共享关键路段，局部独立规划会加剧拥堵，导致系统整体效率下降。相比之下，持续最优路线组合方法能够在批级别进行协调分配，缓解热点路段的负载压力，从而获得更优的全局通行效果。在高负载场景下，这种优势尤为明显。例如，在 NY 路网 $|Q| = 10,000$ 以及 TG 路网 $|Q| = 20,000$ 时，与 Ind 算法相比，所提出方法能够降低超过 40% 的总通行时间。这表明，在动态且高密度的交通环境中，全局协同优化机制的收益会随着请求规模的扩大而进一步放大。

和 SBP 相比，SBP*SA 和 SBP*DA 表现较差。因为它们会在优化过程中进行较多次数的交换操作，这是非常耗时的。具体来说，SBP*SA 没有考虑到已经规划的路线也会对路段交通增加额外的车流量，因此它在初始化路线搜索阶段进行的交通预测是不可靠的。对于 SBP*DA 算法，它以一个行程出发时刻的交通状态快照进行路线规划。因此，SBP*SA 算法和 SBP*DA 算法所指定的初始化路线往往是低质量的，需要在批优化过程中采取大量的交换操作来对这些低质量的初始路线进行调整，从而产生了更多的计算消耗去提高结果质量。从图3-2(b)和图3-2(d)也观察到这三种算法产生的全局路线组合的所有路线的总的通行时间是接近的，而且的 SBP 总能产生较小的通行时间。这些实验结果证实了通过采用自感知的思想和考虑到网络动态性，初始路线搜索能够有效地产生高质量的处理路线，可以减少后续优化处理过程中的交换操作次数。

3.5.2.2 行程请求到达速率的影响

图3-3展示了在不同行程请求到达速率下，持续最优路线组合算法在不同路网上的表现。行程请求到达速率反映了单位时间内进入系统的请求数量。当到达速率增大时，意味着在相同时间窗口内将有更多车辆被分配到路网中，从而加剧各路段的交通负载与路线之间的耦合程度。可以观察到，在 NY 与 TG 两个路网中，算法运行时间均随着到达速率的增加而单调上升。这是因为更高的请求到达速率会在每个优化周期内产生更多待处理请求，从而扩大批处理规模并增加协调计算开销。

同样可以观察到，在两个路网上规划路线的总通行时间均呈现明显上升趋势。这是由于随着单位时间内车辆数量增加，各路段的流量迅速累积，路段通行时间随之增长，进而导致全局路线组合的总通行时间增加。这一现象符合交通流理论中关于拥堵随流量增长而加剧的基本规律。此外，在相同到达速率下，TG 路网的总通行时间始终显著高于 NY 路网。这主要是由于在默认实验设定中，TG 路网对应的总体请求规模更大，导致系统整体负载更高，从而产生更大的累计通行时间。同时，TG 路网规模与结构特性的差异也会进一步放大流量增长带来的拥堵效应。总体而言，实验结果表明，随着行程请求到达速率的提升，算法的计算开销与系统

总通行时间均呈现稳定上升趋势，但增长过程平滑且可控，验证了持续最优路线组合方法在高频动态请求环境下的适应能力。

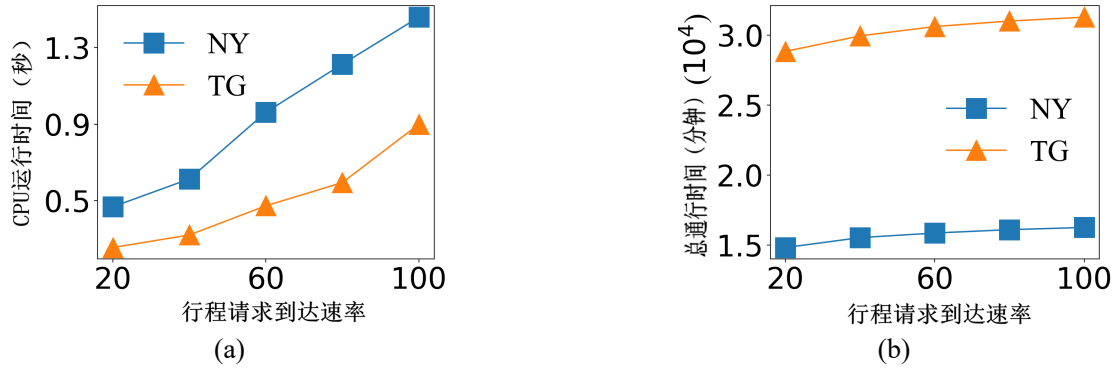


图 3-3 行程请求到达速率的影响。(a) 对算法运行时间的影响；(b) 对总通行时间的影响

3.5.2.3 通行时间函数各参数的影响

图3-4展示了当通行时间函数中的参数 α 和 β （参见式 3-1）发生变化时，持续最优路线组合算法的性能变化情况。根据通行时间函数的定义， α 控制流量对路段通行时间的放大程度，而 β 则调节流量影响的增长形态。因此，从直觉上看，较大的 α 或较小的 β 都会使路段通行时间对流量变化更加敏感，从而可能加剧拥堵效应。在图3-4(a)中，可以观察到，当 α 从1增加到5时，无论是在NY还是TG路网中，算法运行时间与总通行时间均呈现明显上升趋势。这是因为更大的 α 会放大流量对路段通行时间的影响，使得拥堵效应更加显著，进而导致整体路线组合的累计通行时间增加。同时，由于路段代价差异被进一步拉大，算法在协调不同路线分配时需要处理更复杂的权衡关系，从而带来一定的计算开销增长。

相比之下，当 β 从1增加到3时，总通行时间在两个路网中均呈现下降趋势。这表明较大的 β 在当前函数设定下缓解了通行时间对流量增长的敏感程度，从而减弱了拥堵的累积效应，使得整体路线组合更加高效。值得注意的是，当改变 β 的取值时，算法运行时间并未表现出显著的单调变化趋势。其主要原因在于，算法运行时间主要取决于请求规模与批优化过程的计算复杂度，而并不直接依赖于通行时间函数参数本身。参数 β 的变化虽然会影响路段代价的数值分布，但不会显著改变算法的搜索空间规模或计算流程，因此对计算开销的影响较为有限。该实验结果表明，通行时间函数参数对系统总通行时间具有显著影响，而对算法计算开销的影响相对有限。这进一步验证了持续最优路线组合方法在不同交通流敏感度设定下的稳定性与适应能力。

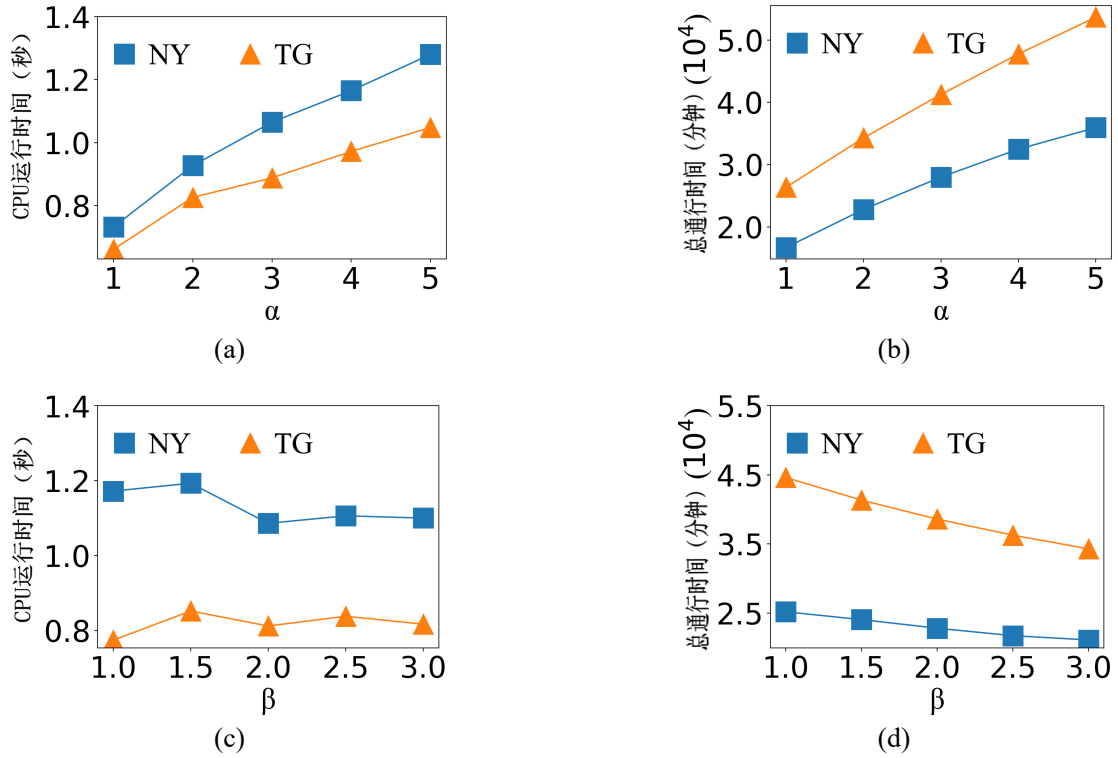


图 3-4 路段通行时间函数中两个参数的影响

3.5.2.4 优化时间间隔的影响

图3-5展示了当批优化时间间隔 T 发生变化时,持续最优路线组合算法的性能表现。参数 T 控制系统进行一次批量重优化的时间跨度。较大的 T 意味着请求在进入优化队列后需要等待更长时间,但同时也能够一次优化过程中纳入更多尚未完成的行程请求,从而提升全局协调能力。在这种情况下,通过交换操作大大减少所有行程总的通行时间。在图3-5(a),可以观察到,在 NY 与 TG 两个路网中,算法运行时间均随着 T 的增加而近似线性上升。这是因为更大的时间间隔会在单次优化过程中累积更多请求,使得批处理规模扩大,从而增加路线重分配与交换操作的计算开销。因此,计算成本与 T 呈正相关关系。

在图3-5(b)中,显示总通行时间 $TT(II)$ 随 T 的增加呈现轻微下降趋势。这表明,当优化间隔适度增大时,算法能够在更大的请求集合上进行全局协调,通过路线交换更有效地缓解热点路段的拥堵,从而提升整体通行效率。换言之,较大的 T 提供了更充分的全局信息,使得优化结果更加接近系统层面的最优解。需要注意的是, $TT(II)$ 的下降幅度相对平缓,说明在当前实验设置下,算法在较小 T 时已经能够获得较为合理的解,而进一步扩大时间间隔仅带来边际收益的提升。这也体现出算法在不同优化频率下的稳定性。该实验揭示了优化时间间隔 T 在计算开销与优化效果之间所起到的调节作用。较小的 T 能够降低等待时间与计算成本,

而较大的 T 则有助于提升全局协调能力并略微改善总通行时间。因此，在实际应用中，应根据实时性需求与系统负载情况选择合适的优化时间间隔，以在效率与效果之间取得平衡。

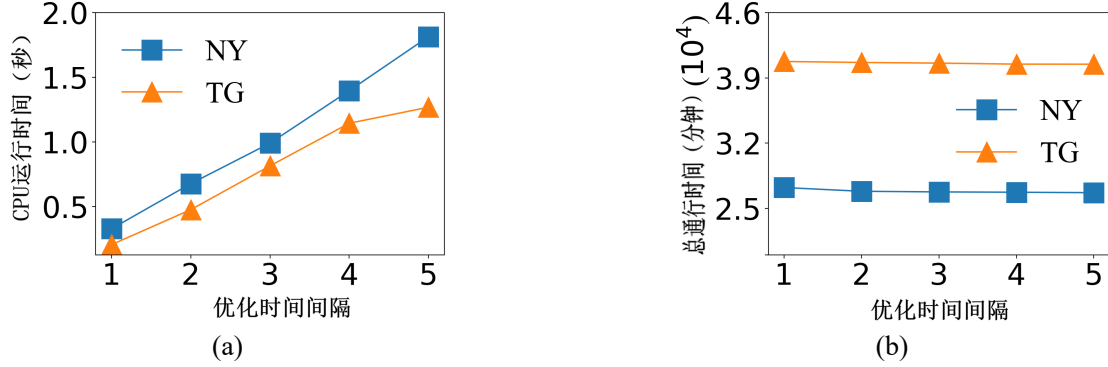


图 3-5 批优化时间间隔的影响

3.5.2.5 提升参数和预检查策略的影响

最后，研究了提升参数 ϵ 以及预检查策略对持续最优路线组合算法性能的影响。将 ϵ 从 0.0002 逐步增加至 0.001，分析其对计算开销与总通行时间的影响。根据算法 3-3，在优化过程中，有效交换操作的触发条件与参数 ϵ 成反比关系。 ϵ 越大，意味着判定一次交换为“有效”的门槛越高，从而可执行的有效交换次数减少。因此，随着 ϵ 的增大，算法在单次优化过程中的搜索深度降低，算法运行时间随之减少。然而，一个更大的 ϵ 也会产生具有更大通行时间的全局路线组合 Π 。在图3-6(a)中，可以观察到，在 NY 与 TG 两个路网中，算法运行时间均随着 ϵ 的增大呈现下降趋势。尤其当 ϵ 从 0.0002 增加至 0.0004 时，算法运行时间降幅最为显著。例如，在 NY 路网与 TG 路网中，算法运行时间分别减少超过 36% 与 19%，表明在较小阈值区间内适当放宽有效交换标准能够显著降低计算开销。

与此同时，图3-6(b)显示总通行时间随 ϵ 的增加呈现轻微上升趋势。这是因为较大的 ϵ 会减少可执行的优化操作次数，从而限制了路线重分配的充分性，使得最终得到的全局路线组合在通行效率上略有下降。然而，该增长幅度相对平缓，说明算法对 ϵ 具有一定鲁棒性。综合考虑计算效率与优化效果，可以发现当 $\epsilon = 0.001$ 时，算法在算法运行时间与总通行时间之间取得了较好的平衡，因此该取值在当前实验设置下是一个较为合适的选择。这表明，通过合理设定参数 ϵ ，可以在保证通行效率基本稳定的前提下显著降低计算成本，从而提升算法整体性能。

此外，进一步评估了预检查策略对算法效率的影响。记未使用预检查策略的持续最优路线组合算法为 SBP*PC。从图3-6(c)可以看出，在不同请求规模行程请

求数目下，引入预检查策略的算法始终具有更低的算法运行时间。该结果说明，预检查机制能够在正式执行交换操作前提前过滤无效或低收益的候选操作，从而有效减少不必要的计算开销，进一步提升算法效率。上述实验结果表明，参数 ϵ 与预检查策略均对算法性能具有显著影响。通过合理选择 ϵ 并结合预检查机制，持续最优路线组合算法能够在计算效率与全局通行效率之间实现更加理想的权衡。

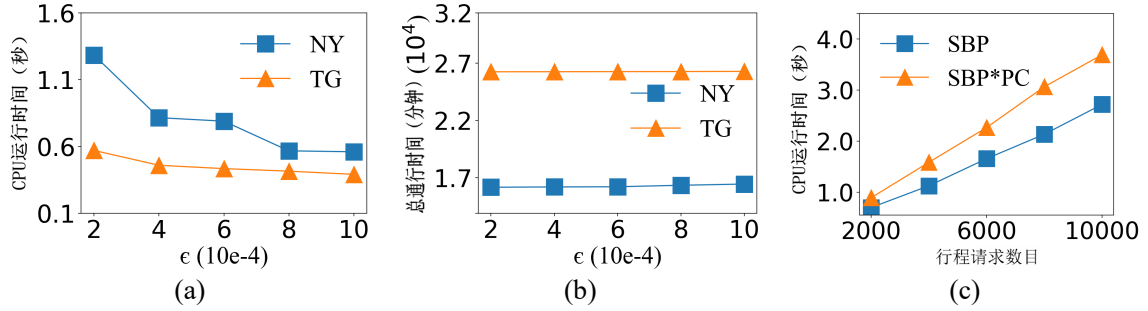


图 3-6 提升参数和预检查策略的影响。(a) 运行时间；(b) 通行时间；(c) 预检查策略的影响评估

3.6 本章小结

近年来，随着自动驾驶技术与车联网技术的迅速发展，大量车辆可以在极短时间内向路线规划系统提交行程请求。尤其在早晚高峰等特定时段，行程请求呈现出密集、持续到达的特点，从而形成行程请求流。在这一新兴应用场景下，传统“逐个独立规划”的策略逐渐暴露出局限性。若系统仅基于当前时刻的交通状态为每个请求独立生成个人最优路线，则可能忽略已规划路线对未来交通状态的影响，从而在局部最优决策的叠加下诱发全局拥堵。基于此，本文旨在为持续的形成请求搜寻最佳的路线组合，提出了一个自感知的批处理算法以及修剪枝技术来高效解决全局持续最优路线组合问题。

该解决方案包含初始路线规划和批量优化处理两个步骤。初始路线搜索过程从起点开始，对与起点相连的邻接顶点做网络扩张，接着选择一个新的顶点继续扩张，直到扩张到终点。每次选择一个最可能最快到达终点的顶点作为下一个扩张顶点，从起点到该点的精确耗时加上从该点到终点的预计耗时最小的顶点。从起点到该点的精确耗时是根据每个路段的实时车流量准确计算出来的，而从该点到终点的预计耗时是预处理中计算的。在预处理中，取每个路段的最小车流量作为静态的车流量，利用 *dijkstra* 算法计算顶点两两之间的最短路线，并将顶点两两之间的最小耗时存储起来作为初始路线搜索的算法输入。在批量优化处理过程中，将在同一时间间隔内发出的行程请求作为一个整体，对之前为它们规划的初始路

线集合进行批量优化处理。

本章主要贡献总结如下。首先，本章研究了全局持续最优路线组合问题。该问题具有十分广泛的应用场景，比如城市交通规划、智能驾驶、物流规划等。针对该问题，本章设计了一个持续最优路线组合框架，包含初始路线规划和批量优化处理两个步骤。在初始路线搜索中，对于每一个新到达的行程请求，寻找一条在出发时刻出发，从起点能最快到达终点的路线。在批量优化处理过程中，将在同一时间间隔内发出的行程请求作为一个整体，对之前为它们规划的初始路线集合进行批量优化处理。最后，在真实数据集上进行了广泛的实验。在两个真实数据集上的实验结果表明，提出的算法与个人最优路线规划算法相比，所提出方法能够降低超过 40% 的总通行时间。

第四章 群体联合移动模式挖掘

4.1 引言

本章研究如何针对联合移动模式挖掘这一城市计算应用设计高性能算法。通过对大规模移动轨迹的系统分析，旨在设计高效且可扩展的算法框架，用于识别在特定时间区间内频繁共同出现在相邻空间位置，且在多个时间点存在联合移动的对象集合，即特定群体联合移动模式。该问题有助于为城市规划、交通管理以及社会行为研究等应用场景提供更加深入和精细化的决策支持。

为实现上述目标，本章从数据建模、索引结构设计与算法优化等多个层面展开研究。首先，对特定群体模式检测问题进行形式化定义，明确问题输入、输出及约束条件，并系统分析其在大规模数据环境下面临的计算复杂性与存储开销挑战；其次，结合时空数据挖掘技术，构建适用于大规模轨迹数据的高效索引结构，以支持高性能的候选模式生成与验证过程。在研究背景方面，调研了联合移动模式搜索与轨迹聚类领域的代表性工作，总结其在模式表达能力、计算效率与扩展性方面的优势与不足，从而引出本章研究的核心问题与技术挑战。在问题描述层面，本章依次给出了移动对象及其轨迹的形式化定义，刻画了联合移动行为的时间持续约束以及空间共现条件，并在此基础上提出了一个更加简化且具有实际意义的伴随群体发现问题定义。

总体而言，大规模性、模式定义的灵活性、算法效率要求以及数据动态变化特征共同构成了该问题的技术难点，也对算法框架设计与系统实现提出了更高要求。针对伴随群体发现问题，首先设计了一种精确求解算法，随后提出了一种新颖的社区感知聚类模型，以有效缩减搜索空间并提升候选群体的质量。最后，本章在真实世界数据集开展了全面的实验评估。

4.1.1 研究背景与意义

作为时空数据挖掘中的一项核心任务，联合移动模式挖掘问题已受到广泛关注。一般而言，联合移动模式描述一组在一段时间内在空间上保持接近的对象集合。围绕这一问题，研究者提出了多种具有代表性的模式定义，包括 flock [107, 108]、convoy [103]、swarm [104]、group [111]、platoon [112] 以及 gathering [106, 134] 等。这些模式在时间连续性约束、空间密度约束以及成员稳定性等方面各有侧重，共同构成了联合移动模式研究的理论基础。

在诸多实际应用中，高效且准确地从移动对象轨迹中发现联合移动模式具有

重要意义。例如，在拼车推荐与出行规划中，可以通过识别稳定的同行群体，使其共同出发或考虑彼此对交通的影响等，从而提升资源利用率；在交通拥堵缓解中，可通过及早发现道路上的高密度移动群体来规避潜在拥堵；在新冠病毒等传染病的暴露与风险分析中，可辅助识别潜在高风险接触群体 [118, 120]；此外，在移动行为分析与异常轨迹检测 [115] 等任务中，群体模式挖掘同样发挥着关键作用。

然而，如何在保证模式表达能力的同时提升计算效率，成为该领域亟需解决的重要问题。现有研究大多基于移动对象随时间演化的完整轨迹设计搜索算法。这类方法通常在每一个时间快照上执行聚类或密度计算，以判断对象之间的空间邻近关系。当联合移动模式在时间上不稳定，即对象可能暂时离开群体后又重新加入时，算法需要反复进行候选生成与验证，计算代价显著增加。尤其是在实时或近实时应用场景中，对所有时间快照进行全量聚类将带来极高的时间开销。此外，主流方法多采用先过滤后精炼的处理框架，在过滤阶段通过建立时空索引排除不可能的数据对象，仅保留可能的候选群体，再在精炼阶段逐一验证其有效性。随着轨迹数据规模的持续增长，候选数量呈指数级膨胀，导致算法性能显著下降，难以满足大规模与在线分析的需求。

4.1.2 研究内容与挑战

在现实应用中，尤其是在公共安全紧急预警与传染病防控等对时效性要求极高的场景下，及时获得最新的联合移动结果至关重要。因此，联合移动模式挖掘任务往往需要以连续或增量的方式执行。这不仅要求算法在时间复杂度上具备良好的可扩展性，还要求其在模式定义上具有一定的灵活性，以适应动态变化的移动行为。然而，现有群体模式定义及其对应方法在同时兼顾模式表达能力与计算效率方面仍存在明显不足，难以在大规模数据环境中实现高效、稳定的持续挖掘。

为了解决上述问题，本文提出了一种联合移动模式概念——伴随群体。相较于已经存在的传统的联合移动模式定义，伴随群体的约束更加宽松且更具实用性。具体而言，一个伴随群体由至少 m 个移动对象组成，这些对象需要在至少 k 个时间段内在空间上保持接近（例如，两两距离不超过预定义的距离阈值 ϵ ），并允许对象在时间维度上暂时离开群体而不破坏整体模式的有效性。基于该模式定义，进一步形式化提出了伴随群体发现问题。给定一组移动对象及其轨迹数据，伴随群体发现问题旨在用户可交互时间范围内高效发现所有满足条件的伴随群体。此外，在问题设定中，若某一对象能够同时参与多个伴随群体，则将其归属于成员规模最大的伴随群体，以保证群体划分结果的确定性与可解释性。伴随群体发现问题具有广泛的应用前景，可服务于交通监测与管理、疫情防控、公共安全预警以及群体行为分析等多种实际场景，为大规模时空数据环境下的群体行为挖掘

提供一种高效且可扩展的解决方案。

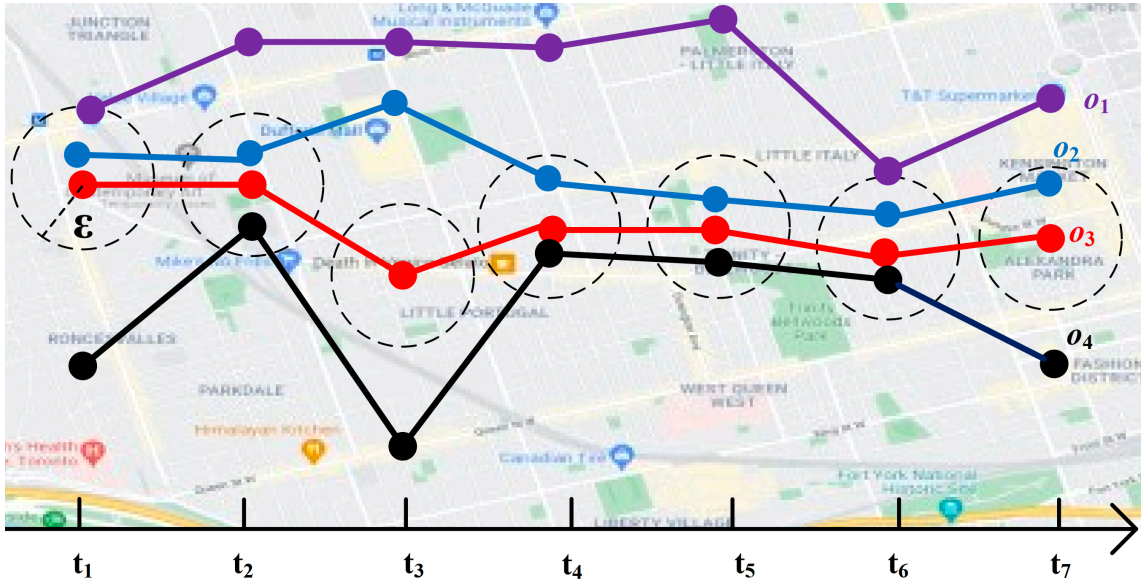


图 4-1 伴随群体发现问题示例 (空间距离阈值为 10 米, 联合移动的最小联合移动时长阈值为 3 分钟, 伴随群体规模阈值为 3)

示例 4.1 (伴随群体发现问题示例) 图 4-1 给出了一个具体示例以直观说明伴随群体的判定过程。设 o_1, o_2, o_3, o_4 为四个移动对象, 且所有对象以固定采样频率每分钟上报一次位置信息。图 4-1 展示了这四个移动对象在时间区间 $[t_1, t_7]$ 内的轨迹变化情况。设空间距离阈值 ϵ 为 10 米, 联合移动的最小联合移动时长阈值为 3 分钟, 伴随群体规模阈值为 3。首先, 从对象两两关系的角度进行分析。可以观察到, 对象 o_2 与 o_3 在时间集合 $T = \{t_1, t_2, t_4, t_5, t_6, t_7\}$ 内始终满足两两距离不超过空间距离的约束, 即在这些时间点保持空间接近关系。由于 $|T| = 6$, 显著超过联合移动时长阈值 k , 因此 o_2 与 o_3 构成一个满足条件的联合移动对。同理, o_3 与 o_4 在不少于 k 个时间点内保持空间邻近关系, 也可以构成一个联合移动对。在此基础上, 进一步考察多对象群体关系。可以发现, 在时间点 $\{t_2, t_4, t_5, t_6\}$ 上, 对象 o_2, o_3, o_4 的两两距离均不超过 ϵ , 即它们在这些时间点形成一个空间上连通且满足密度约束的三元对象集合。由于该时间集合的规模不少于联合移动时长阈值 3, 且群体规模 $|g| = 3$ 满足最小规模阈值, 因此将 $g = \{o_2, o_3, o_4\}$ 判定为一个有效的伴随群体。需要注意的是, 对象 o_1 仅在 t_1 和 t_6 与 o_2 短暂接近, 其满足距离约束的时间点数量不足以达到联合移动时长阈值 k , 且未能与其他对象形成稳定的多对象联合移动模式。因此, o_1 既无法与其他对象构成有效的联合移动对, 也不能参与任何满足条件的伴随群体。该示例说明, 伴随群体的判定不仅依赖于空间邻近关系, 还同时受到时间持续性与伴随群体规模约束的共同影响, 从而能够更全面地刻画移动对象之间的联合移动行为特征。

高效解决伴随群体发现问题面临以下关键技术挑战：

- **大规模数据处理：**在真实应用场景中，轨迹数据通常来源于海量移动对象，例如车辆、移动终端或各类传感设备，其数据规模可能达到千万级甚至更高。轨迹数据不仅记录频繁，而且具有高维度与强相关性特征，使得存储与计算开销显著增加。如何在如此庞大的数据规模下实现高效的数据组织、索引构建与查询处理，尤其是在需要实时或近实时发现群体模式的场景中，成为首要挑战。
- **模式定义的灵活性：**不同应用场景对群体模式的语义需求存在差异。例如，有的应用更关注空间邻近性，有的强调时间持续性，还有的关注成员规模或运动一致性等约束条件。因此，如何构建一个既支持多种群体模式表达，又能够灵活调整参数（如距离阈值、联合移动时长阈值与最小群体规模）的统一算法框架，是实现方法广泛适用性的关键。同时，该框架在增强表达能力的同时，必须避免因模式定义过于复杂而导致搜索空间急剧膨胀，从而影响整体计算效率。
- **准确性与效率的平衡：**群体模式发现通常涉及复杂的时空约束匹配、多对象关系维护以及候选结构验证等操作。若直接采用精确枚举算法，往往需要在大量时间快照上反复执行聚类或连通性检测，计算代价极高。因此，如何在保证结果准确性的前提下，通过有效的剪枝策略、搜索空间压缩技术、分层索引机制或近似计算方法减少不必要的候选生成与验证，是提升系统性能的核心问题。尤其在数据规模持续增长的背景下，算法必须在结果质量与运行效率之间取得合理且可控的权衡。

4.1.3 研究成果与贡献

综上所述，大规模数据处理需求、模式定义的灵活性要求、算法效率约束以及数据动态演化特征，共同构成了伴随群体发现问题的核心技术难点，也对整体算法框架设计提出了更高要求。针对伴随群体发现问题，本文从精确算法设计与高效近似算法设计两个层面展开研究。

首先，设计了一种精确搜索算法。该算法基于严格的时空约束判定机制，对候选对象对及候选群体进行精确验证，能够保证结果的完备性与准确性。然而，由于需要在多个时间维度上进行精细化匹配与验证，其计算代价相对较高，更适用于离线分析或数据规模适中的场景。为进一步提升算法效率并满足实时或近实时应用需求，本文额外提出了一种社区感知的近似搜索算法。该算法在保证结果质量可控的前提下，通过近似计算与结构化聚类方法显著所见缩减了搜索空间，从而在严格的效率约束下有效解决伴随群体发现问题，实现准确性与效率之间的平衡。

近似算法主要包括两个阶段：近似联合移动对搜索阶段与社区感知群体发现阶段。在近似联合移动对搜索阶段，首先采用基于网格划分的空间离散化方法，将每个轨迹位置映射为对应的网格单元编号，从而将连续轨迹转换为离散的网格单元编号序列表示。随后，通过计算不同轨迹之间的公共编号数量来快速估计其潜在的联合移动程度，从而生成候选的近似联合移动对。该过程有效避免了在原始连续空间中进行高代价的距离计算。在社区感知群体发现阶段，将生成的联合移动对视为图结构中的边，并利用 SCAN 算法 [135] 进行结构化聚类，以识别具有高内聚性和清晰社区边界的对象集合。通过引入社区结构约束，算法能够在保证群体结构合理性的同时，进一步降低无效候选的数量，从而提升整体运行效率。

本文的主要贡献总结如下：

- 提出了伴随群体这一联合移动模式定义。该定义相较于传统模式更加宽松且具备更强的适应性，尤其适用于需要持续结果更新与快速响应的应用场景，例如传染病传播监测与公共安全紧急风险预警。
- 设计并实现了一种精确搜索算法以及一种社区感知的近似搜索算法，系统性地解决了伴随群体发现问题在准确性与效率之间的权衡难题。
- 基于两个真实世界数据集开展了全面的实验评估。实验结果表明，所提出的社区感知算法能够在用户可交互时间范围内，从大规模轨迹数据中高效发现伴随群体，验证了方法在实际应用场景中的有效性与实用价值。

本文其余部分组织结构如下。第 4.2 节首先形式化给出问题定义，并介绍相关的符号说明与必要的预备知识，为后续算法设计奠定理论基础。第 4.3 节在此基础上提出一种精确搜索算法，详细阐述其核心思想、实现流程以及时间复杂度分析。第 4.4 节进一步给出一种社区感知的近似算法，通过近似处理与候选剪枝策略提升算法的可扩展性，并分析其性能优势与适用场景。第 4.5 节基于真实世界轨迹数据开展系统实验，从有效性与效率两个方面对所提方法进行全方面评估与对比分析。最后，第 4.6 节对全文进行总结，并展望未来的研究方向。

4.2 问题描述

本节介绍所需的基础概念，并给出问题的形式化定义。给定一组移动对象，依次定义移动对象及其轨迹、联合移动时长、联合移动对、伴随群体以及伴随群体发现问题。本章所使用的主要符号及其解释如表 4-1 所示。

定义 4.1（移动对象及其轨迹） 一个移动对象 o 由其轨迹 $\tau = \{s_1, \dots, s_n\}$ 表示，其中轨迹是一个有限的、按时间顺序排列的离散时空点序列。该序列由位置追踪设备从对象 o 的真实运动轨迹中采样获得。为简化描述，使用 $s_i = ([x, y], t)$ 表

表 4-1 主要符号解释

符号	解释
o/O	一个移动对象/一组移动对象集合
s	一个采样位置点
$\tau/\mathcal{T}$	一条轨迹/一组轨迹集合
t	某个时刻
ϵ_s/ϵ_t	联合移动空间距离阈值/时间距离阈值
k	联合移动时长阈值
m	最小伴随群体规模阈值
g	一个伴随群体
$\mathcal{G}$	一组伴随群体

示二维空间中的一个离散采样位置点，其中 $[x, y]$ 表示空间坐标， t 表示该位置被记录时对应的时间戳。对于一组移动对象 O ，记其轨迹集合为 $\mathcal{T} = \{\tau_1, \dots, \tau_{|O|}\}$ ，其中 τ_i 为对象 o_i 的轨迹。本文对移动对象及轨迹的建模方式与已有研究保持一致 [87, 88, 136]。

定义 4.2（联合移动时长） 设 o_i 与 o_j 为两个移动对象。当 o_i 与 o_j 之间的空间距离小于等于给定的空间距离阈值 ϵ_s ，且对应时间戳之差不超过给定的时间距离阈值 ϵ_t 时，称二者在该时刻处于联合移动行为状态。对象 o_i 与 o_j 的联合移动时长定义为所有满足上述条件的时间长度之和。

定义 4.3（联合移动对） 设 τ_i 与 τ_j 分别为对象 o_i 与 o_j 的轨迹。给定 k 为联合移动时长阈值，若 τ_i 与 τ_j 之间的联合移动时长不少于给定阈值 k ，则称对象 o_i 与 o_j 构成一个联合移动对。

定义 4.4（伴随群体） 给定一组移动对象 $\mathcal{O}$ 以及最小伴随群体规模阈值 m ，若对象子集 $g \subseteq \mathcal{O}$ 满足 $|g| \geq m$ 且对任意两个对象 $o_i, o_j \in g$ ， o_i 与 o_j 均构成联合移动对，则称 g 为一个伴随群体。

定义 4.5（伴随群体发现问题） 给定一组移动对象 $\mathcal{O}$ ，伴随群体发现问题旨在从 $\mathcal{O}$ 中发现一组伴随群体 $\mathcal{G}$ ，并满足对任意两个群体 $g_i, g_j \in \mathcal{G}$ ，均有 $g_i \cap g_j = \emptyset$ 。

4.3 精确搜索算法

本节提出一种用于发现伴随群体的精确搜索方法。该算法对所有候选对象组合进行精确验证，从而保证结果的完备性与正确性。精确搜索算法整体上包含三个核心步骤。首先，给定一组移动对象轨迹集合，通过精确的时空约束计算枚举并识别所有满足联合移动时长阈值的联合移动对；其次，以这些联合移动对为基础，构建对象之间的关联结构，并采用增量扩展策略逐步生成满足条件的伴随群体候

选；最后，在所有生成的候选集合中，依据成员规模优先的原则筛选出不可扩展且规模最大的伴随群体作为最终输出结果。该三阶段流程分别对应“联合移动对发现—伴随群体候选集合扩展—精确结果筛选”三个层次，层层递进，从而实现对搜索空间的系统化遍历与剪枝。在介绍具体流程之前，首先给出伴随群体候选的形式化定义，以便后续算法描述与性质分析。

定义 4.6(伴随群体候选) 一个伴随群体候选 g' 是一个移动对象集合，满足其中任意两个对象均可以构成一个联合移动对。若存在至少一个集合外的对象 $o \notin g'$ ，使得扩展后的集合 $g' \cup \{o\}$ 仍然满足上述条件，则称 g' 可以通过加入对象 o 进行扩展；否则，称 g' 为不可扩展的伴随群体候选。

算法 4-1: 精确搜索算法

```

输入: 1) 轨迹集  $\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_{|\mathcal{O}|}\}$ ;
        2) 空间距离阈值  $\epsilon_s$ ;
        3) 时间距离阈值  $\epsilon_t$ ;
        4) 伴随群体规模阈值  $m$ ;
        5) 联合移动时长阈值  $k$ 。

输出: 一组满足定义约束的精确的伴随群体集合  $G$ 。

1  初始化:  $P \leftarrow \emptyset$ ;                                // 有效轨迹对集合
2  初始化:  $G_c \leftarrow \emptyset$ ;                          // 候选群体集合
3  初始化:  $G \leftarrow \emptyset$ ;                          // 最终结果集合

4  for  $i = 1$  to  $|\mathcal{T}|$  do                                /* 枚举联合移动对 */
5      for  $j = i + 1$  to  $|\mathcal{T}|$  do
6          if  $|\tau_i \cap \tau_j| \geq k$  then
7               $P \leftarrow P \cup \{(\tau_i, \tau_j)\}$ ;

8  foreach  $(\tau_i, \tau_j) \in P$  do                        /* 生成伴随群体候选 */
9       $g_i \leftarrow \{\tau_i, \tau_j\}$ ;
10     foreach  $(\tau_k, \tau_p) \in P$  do
11         if  $\forall \tau_p \in g_i, (\tau_k, \tau_p) \in P$  then      /* 两两有效且所有成员均在  $g_i$  中 */
12              $g_i \leftarrow g_i \cup \{\tau_k\}$ ;
13     if  $|g_i| \geq m$  then                                /* 伴随群体规模约束 */
14          $G_c \leftarrow G_c \cup \{g_i\}$ ;

15  $G \leftarrow \text{select\_max}(G_c)$ ;                        /* 选择伴随群体 */
16 return  $G$ ;
    
```

算法 4-1 给出了精确搜索的完整伪代码描述。该算法以一组轨迹数据 $\mathcal{T}$ 作为输入，同时给定空间距离阈值 ϵ_s 、时间距离阈值 ϵ_t 、最小伴随群体规模阈值 m 以及联合移动时长阈值 k 。算法的输出为一组满足定义约束的伴随群体集合 $G =$

$\{g_1, g_2, \dots\}$, 其中每个 $g_i \subseteq \mathcal{T}$, 表示由若干移动对象轨迹构成的一个伴随群体。在实现层面, 使用一个集合 P 存储所有满足联合移动时长阈值 k 的联合移动对, 用以刻画对象之间的两两联合移动关系 (第 1 行); 同时, 使用一个集合 G_c 存储在搜索过程中生成的伴随群体候选 (第 2 行)。通过先构建联合移动对集合 P , 再基于 P 进行增量式候选扩展与验证, 算法能够系统地遍历可能的对象组合, 并通过可扩展性判定与规模约束进行有效剪枝, 从而最终得到满足条件的伴随群体集合 G (第 3 行)。该算法包含 3 个主要步骤: 枚举联合移动对, 生成伴随群体候选, 以及选择伴随群体。

步骤 1: (枚举联合移动对)

对于任意一对移动对象 $o_i, o_j \in \mathcal{O}$, 计算其轨迹中任意两个采样位置 $s \in \tau_i$ 与 $s' \in \tau_j$ 之间的精确距离。若空间距离与时间距离分别不超过阈值 ϵ_s 与 ϵ_t , 则将其联合移动持续时间 $|\tau_i \wedge \tau_j|$ 累加 1。具体而言, 对于每一对对象 $\langle o_i, o_j \rangle$, 当其累计的联合移动持续时间不少于阈值 k 时, 将其视为一个联合移动对^①。当所有对象对均被评估完成后, 该步骤结束 (第 4–7 行)。

步骤 2: (生成伴随群体候选)

在获得所有联合移动对之后, 基于这些联合移动对以增量方式生成伴随群体候选。具体而言, 首先选择一个以轨迹 τ_i 为起点的联合移动对, 例如 $\langle \tau_i, \tau_j \rangle$, 然后考察所有与其共享对象 o_i 的联合移动对 $\langle \tau_i, \tau_k \rangle$ 。初始群体记为 $g_i = \{\tau_i, \tau_j\}$ 。若 τ_k 与 g_i 中的所有成员均构成联合移动对, 则将 τ_k 加入 g_i 。当 g_i 仍可扩展时, 重复上述扩展过程。若 $|g_i| \geq m$, 则将其加入候选集合 G_c 。随后选择新的联合移动对作为初始群体并继续扩展。当所有联合移动对均被访问并作为初始群体尝试扩展后, 该步骤结束 (第 8–14 行)。

步骤 3: (选择伴随群体)

在获得所有候选群体之后, 从中选取成员规模最大的伴随群体作为最终结果。具体而言, 若某个对象同时属于多个候选群体, 则仅保留其中成员规模最大的一个。该步骤在遍历完所有候选群体后结束 (第 15 行)。

示例 4.2 (精确搜索算法示例) 再次考虑图 4-1 中的示例。假设首先选择联合移动对 $\langle o_2, o_3 \rangle$ 进行扩展。显然, 联合移动对 $\langle o_2, o_4 \rangle$ 与 $\langle o_2, o_3 \rangle$ 共享对象 o_2 , 且对象 o_2, o_3, o_4 两两之间均可形成联合移动对。因此, 将 $g' = \{o_2, o_3, o_4\}$ 视为一个伴随群体候选。由于对象 o_1 无法与 g' 中的任一对象形成联合移动对, g' 不可进一步扩展。接着, 由于 $|g'|$ 不小于预设的伴随群体规模阈值 3, 将 g' 加入候选集合 G_c 。最终, 由于 g' 具有最大的成员规模, 从 G_c 中选取 g' 并将其加入结果集合 G 。

定理 4.1 (精确搜索算法时间复杂度) 精确搜索算法的时间复杂度为 $O(|\mathcal{T}|^2 \times$

^① 在上下文清晰的情况下, 使用 (o_i, o_j) 或 (τ_i, τ_j) 表示一个联合移动对。

$|\tau|^2$), 其中 $|\mathcal{T}|$ 表示移动对象或轨迹的数量, $|\tau|$ 表示单条轨迹的平均长度, 即采样位置数。

证明: 精确搜索的主要时间开销来自三个阶段。

首先, 在枚举所有联合移动对时, 需要对任意一对轨迹 $\tau_i, \tau_j \in \mathcal{T}$ 进行逐点比较。对于每一对轨迹, 需计算其所有采样点对之间的空间距离与时间距离, 时间复杂度为 $O(|\tau|^2)$ 。由于轨迹对的数量为 $O(|\mathcal{T}|^2)$, 因此该阶段的总体时间复杂度为

$$O(|\mathcal{T}|^2 \times |\tau|^2)。$$

其次, 在生成伴随群体候选阶段, 对于共享同一对象的联合移动对 (例如 $\langle o_i, o_j \rangle$ 与 $\langle o_i, o_k \rangle$), 需要判断其是否能够构成伴随群体候选, 并在满足条件时进行扩展。该过程本质上是在联合移动对构成的图结构上进行增量式团扩展。最少情况下 (例如不存在任何共享对象的联合移动对), 该阶段仅需线性扫描联合移动对集合, 时间复杂度为 $O(|\mathcal{T}|)$; 在最坏情况下, 若任意两个对象之间均可形成联合移动对, 则候选生成过程可能涉及 $O(|\mathcal{T}|^2)$ 次组合与验证操作。然而, 该阶段的复杂度上界仍然不超过前一阶段的 $O(|\mathcal{T}|^2 \times |\tau|^2)$ 。

最后, 在候选集合 G_c 中选择最终伴随群体时, 仅需遍历所有候选并进行规模比较, 时间复杂度为 $O(|G_c|)$ 。由于 $|G_c| < |\mathcal{T}|^2$, 且通常远小于前两阶段的计算规模, 因此该阶段不会改变整体复杂度阶。综上所述, 精确搜索算法的总体时间复杂度由联合移动对枚举阶段主导, 为

$$O(|\mathcal{T}|^2 \times |\tau|^2)。$$

■

4.4 社区感知的近似搜索算法

尽管精确搜索直观且能够保证结果的精确性, 但其较高的计算开销使其难以直接应用于大规模实际场景, 尤其是在移动对象数量极大的情况下。其主要性能瓶颈在于需要通过精确计算联合移动时长来枚举所有联合移动对, 该过程涉及大量轨迹间的逐点比较操作, 时间代价高昂。为提升算法效率, 本文提出一种基于社区感知的搜索算法, 并结合近似联合移动行为计算策略。该方法在保证较高结果质量的同时, 显著降低了计算复杂度。主要包含两个阶段: (1) 近似联合移动对搜索阶段, 通过构建轨迹的紧凑表示并进行快速相似性筛选, 以高效识别潜在的联合移动对; (2) 基于社区感知的聚类阶段, 在所构建的对象关系图上挖掘结构紧密的对象子集, 从而获得伴随群体。接下来, 将依次介绍近似联合移动对搜索策略以及社区感知聚类算法的具体设计。

4.4.1 近似联合移动对搜索

在该阶段中，提出了一种高效的联合移动对搜索方法，用于计算两个对象轨迹之间联合移动时长的近似值。所生成的近似联合移动对将作为后续社区感知聚类阶段的输入，从而高效地发现伴随群体。该阶段包含两个子步骤：生成轨迹网格编号序列，以及搜索近似联合移动对。

4.4.1.1 轨迹网格编号序列生成

该方法受到群体（flock）圆形结构思想的启发 [103]。首先将底层时空域进行离散化处理：在空间维度上，将二维欧氏空间划分为边长相等的网格单元；在时间维度上，将时间轴划分为等长的时间片。因此，对于对象轨迹中的每一个数据点（位置） $s = ([x, y], t)$ ，均可映射为一个网格编号，记为 $o(s) = f(x, y, t)$ ，其中 $f(\cdot)$ 为单射函数，用于将三维离散坐标唯一映射为一个整数标识。由此，轨迹 τ 可表示为对应的网格编号序列： $O(\tau) = \{o(s_1), \dots, o(s_{|\tau|})\}$ 。图 4-2 展示了空间与时间维度的划分方式以及轨迹网格编号序列的生成过程。假设空间被划分为 5×4 个网格单元，每个单元边长为 10 米，时间维度被划分为 3 个时间片，每个时间片长度为 10 秒。给定两条轨迹 τ_1 和 τ_2 ，其位置分别落入单元 $\{([0, 0], 0), ([3, 1], 1), ([2, 1], 2)\}$ 。基于上述划分方式，定义单射函数为 $o(s) = f(x, y, t) = t \times (5 \times 4) + y \times 5 + x$ ，其中 $x \in \{0, 1, 2, 3, 4\}$ ， $y \in \{0, 1, 2, 3\}$ ， $t \in \{0, 1, 2\}$ 。

示例 4.3 对于位置 $([3, 1], 1)$ ，其网格编号值为 $1 \times (5 \times 4) + 1 \times 5 + 3 = 28$ 。如图 4-2 所示，尽管 τ_1 与 τ_2 在各时间点的精确空间位置不同，但由于它们落入相同的空间网格单元与时间片，因此可被表示为相同的网格编号序列 $\{0, 28, 47\}$ 。

定理 4.2 若将空间划分为边长为 ϵ 的等大小网格单元，其中 ϵ 等于联合移动行为距离阈值，则若两个对象 o_i 与 o_j 的轨迹网格编号序列共享 p 个相同网格编号，则它们的联合移动时长至少为 p 个时间段。

证明：位置 s 的网格编号值由单射函数 $o(s) = f(x, y, t)$ 唯一确定。若两个位置具有相同的网格编号值，则说明它们位于同一个空间网格单元，并且处于同一时间片内。由于网格单元的边长设置为联合移动行为距离阈值 ϵ ，因此同一单元内任意两点之间的空间距离不超过 ϵ ，同时其时间差不超过一个时间片长度。若两条轨迹的网格编号序列共享一个网格编号，则表示对应对象在该时间段内保持不超过 ϵ 的空间距离共同移动。因此，共享 p 个网格编号意味着至少存在 p 个满足距离约束的联合移动行为时间段，从而联合移动时长不少于 p 。 ■

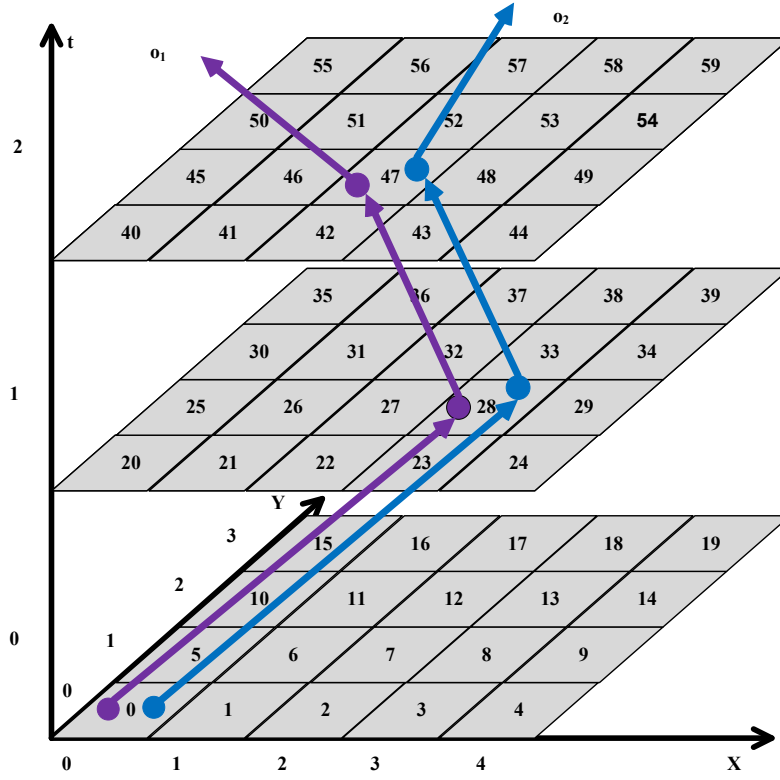


图 4-2 轨迹网格编号及网格编号序列示例

4.4.1.2 近似联合移动对搜索

基于轨迹网格编号序列，通过统计两个轨迹之间的公共网格编号数量来近似计算其联合移动时长。对于两个轨迹网格编号序列 $O(\tau_i)$ 和 $O(\tau_j)$ ，若其公共网格编号数 $|O(\tau_i) \cap O(\tau_j)|$ 不小于联合移动时长阈值，则将 $\langle \tau_i, \tau_j \rangle$ 记录为一个近似联合移动对。该过程对所有轨迹对执行一次，当全部轨迹网格编号序列均被评估完成后终止。

4.4.2 社区感知聚类

对于轨迹集合 $\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_{|\mathcal{T}|}\}$ ，通过第一阶段近似联合移动对搜索后可获得轨迹网格编号序列集合 $O(\mathcal{T}) = \{O(\tau_1), O(\tau_2), \dots, O(\tau_{|\mathcal{T}|})\}$ ，以及一组近似联合移动对。在第二阶段中，目标是发现所有满足以下条件的子集 $\mathcal{T}' \subseteq \mathcal{T}$ ：(1) $|\mathcal{T}'| \geq m$ ；(2) 对任意 $\tau_i, \tau_j \in \mathcal{T}'$ ，均有 $|O(\tau_i) \cap O(\tau_j)| \geq k$ ，

即满足伴随群体定义的对象集合。为此，采用聚类技术以实现高效检测。现有轨迹聚类方法主要包括基于划分的聚类 [137, 138]、基于密度的聚类 [139, 140] 以及基于深度学习的聚类 [91, 92, 121]。然而，这些方法通常侧重于整体轨迹相似性建模，难以刻画对象之间的显式联合移动行为连接关系；同时，部分基于密度的方法在大规模数据上具有较高的计算开销。因此，上述针对轨迹聚类或表示学习

提出的解决方案，均无法直接应用于本文所研究的伴随群体发现问题。其根本原因在于，聚类方法关注的是整体轨迹相似度或嵌入空间中的距离关系，而处于同一聚类中的移动对象并不一定在具体时间段内真实地满足时空共现约束。换言之，轨迹在全局层面上的相似性并不能保证其在局部时间区间内存在持续的联合移动行为。因此，伴随群体发现问题仍需基于严格的时空约束建模与结构化搜索策略进行专门设计，而不能简单依赖现有的轨迹表示学习或聚类方法。

本章巧妙地将联合移动行为关系建模为图结构，其中节点表示移动对象，边表示近似联合移动对。一个伴随群体对应于图中由至少 m 个节点构成的高度结构化子图，且任意两节点之间均满足联合移动行为约束。受同质性假设的启发——即共享大量联合移动行为邻居的对象更可能属于同一群体——采用基于共享邻居结构的社区感知聚类方法进行检测。SCAN 算法 [135] 是一种典型的社区感知聚类方法，其基于顶点结构相似性与连通性进行社区划分，时间复杂度为 $O(|P|)$ ，其中 $|P|$ 为联合移动对的数量（即图中的边数）。因此，SCAN 能够高效地应用于伴随群体检测任务。

定理 4.3 CS-ACMC 算法的时间复杂度为 $O(|\mathcal{T}|^2 \times |\tau|)$ 。

证明：在近似联合移动对搜索阶段，需要对每一对轨迹计算其公共网格编号数。若每条轨迹的网格编号序列长度为 $|\tau|$ ，则通过线性扫描或哈希统计可在 $O(|\tau|)$ 时间内完成一对轨迹的公共网格编号计算。因此，该阶段的时间复杂度为 $O(|\mathcal{T}|^2 \times |\tau|)$ 。在社区感知聚类阶段，SCAN 算法的时间复杂度为 $O(|P|)$ ，其中 $|P|$ 为近似联合移动对数量，且 $|P| \leq |\mathcal{T}|^2$ 。因此，整体时间复杂度为 $O(|\mathcal{T}|^2 \times |\tau| + |P|) = O(|\mathcal{T}|^2 \times |\tau|)$ 。综上，CS-ACMC 算法的总体时间复杂度为 $O(|\mathcal{T}|^2 \times |\tau|)$ 。 ■

4.5 实验评估

在本节中，首先介绍实验设定，包括所使用的数据集与运行环境；随后给出实验结果与分析。实验主要关注所提出搜索算法在运行时间方面的表现，即高效性。通过设置不同参数，如轨迹长度、位置点密度、联合移动时长阈值等，对比不同算法的运行时间并进行分析。

4.5.1 实验设置

4.5.1.1 数据准备

本文实验基于两个真实世界的出租车轨迹数据集：北京出租车轨迹数据集 [141, 142]^①以及波尔图出租车轨迹数据集^②。北京出租车轨迹数据集记录了

① <https://www.microsoft.com/en-us/research/publication/t-drive-trajectory-data-sample/>

② <https://www.kaggle.com/c/pkdd-15-predict-taxi-service-trajectory-i/data>

2008 年 2 月 2 日至 2008 年 2 月 8 日期间,北京市 10357 辆出租车的 GPS 轨迹数据。该数据集包含了约 1500 万条轨迹点,总距离达到 900 万公里,平均采样间隔约为 177 秒,距离约为 623 米。每个文件以出租车 ID 命名,详细记录了时间、经度和纬度信息,为城市交通分析、出租车轨迹预测、路径规划以及地理信息系统研究提供了丰富的数据支持。波尔图出租车轨迹数据集来源于葡萄牙波尔图市的出租车调度系统。时间跨度为 19 个月,包含超过 170 万条出租车轨迹。所有车辆均通过安装在车载终端中的移动数据设备与调度中心通信,因此能够连续采集高精度 GPS 轨迹数据。该数据集具有时间跨度长、采样频率稳定、轨迹覆盖范围广等特点,能够真实反映城市尺度下出租车群体的动态出行行为,为联合移动行为模式挖掘与群体结构分析提供了可靠的数据基础。

4.5.1.1.1 数据预处理 为统一实验设定,所有时间戳均被映射到 24 小时范围内。对于波尔图数据集,每条轨迹的出发时间被随机分配至 24 小时区间内,以避免时间分布偏置。所有轨迹数据在实验前均被预处理并转换为轨迹网格编号序列。生成网格编号序列及伴随群体发现所使用的默认参数设置如表 4-2 所示。同时,对轨迹的经纬度范围以及轨迹长度进行了筛选与限制,以保证实验规模可控且具有代表性,具体参数同样列于表 4-2 中。

表 4-2 评估参数设定

	北京数据集	波尔图数据集
经度范围	[116.250, 116.550]	[-8.735, -8.156]
维度范围	[39.830, 40.030]	[40.953, 41.307]
轨迹长度范围	[20, 100]	[20, 100]
ϵ_s	200 米	200 米
ϵ_t	200 秒	200 秒
$ \mathcal{T} $	2K–10K / 默认 6K	20K–100K / 默认 20K
k	2–6 / 默认 4	2–6 / 默认 4
m	2–6 / 默认 4	2–6 / 默认 4
μ	0.42–0.50 / 默认 0.50	0.42–0.50 / 默认 0.50

4.5.1.2 比较算法

表4-3列举了实验研究中的比较算法。本文对以下两种算法进行性能比较: ES-ECMC 算法, 以及 CS-ACMC 算法。此外, 在候选对象筛选阶段引入 R 树索引结构 [18], 以对空间上显然不满足距离约束的对象对进行早期剪枝, 从而减少无效距离计算并提升整体运行效率。

表 4-3 比较算法

算法标识	算法描述
ES-ECMC	精确搜索算法
CS-ACMC	基于近似联合移动行为计算的社区感知搜索算法

4.5.1.3 评估指标

为了评估 CS-ACMC 算法的有效性,采用 ES-ECMC 算法所发现的精确联合移动对作为基准,并使用联合移动对的平均命中率作为主要质量评估指标。具体而言,对于 ES-ECMC 算法发现的每一个精确伴随群体 g ,若其中任意一个联合移动对 $\langle o_i, o_j \rangle$ (其中 $o_i \in g, o_j \in g$) 同时存在于 CS-ACMC 算法所发现的任意一个伴随群体中,则认为该联合移动对被命中,命中计数加一;否则认为未被命中。最终统计所有精确联合移动对的命中比例,作为整体命中率。该指标能够有效反映近似算法在保持联合移动行为结构方面的结果质量。在效率评估方面,采用平均 CPU 运行时间作为主要性能指标。需要说明的是,在默认参数设置下,ES-ECMC 算法在北京数据集上采用单线程运行时至少需要 1 天时间,因此实验中为 ES-ECMC 启用了 40 个线程以保证实验可行性;相比之下,CS-ACMC 算法仅使用单线程运行。

4.5.1.4 实验环境

所有算法均采用 Python 实现,并运行于 Ubuntu 操作系统平台。实验硬件环境为:2 颗 Intel(R) Xeon(R) Silver 4214R CPU (主频 2.40GHz) 以及 128GB 内存。除非特别说明,所有实验结果均为在 20 组不同测试轨迹输入下重复实验所得的平均值,以减少偶然因素对结果的影响。

4.5.2 实验结果与分析

4.5.2.1 轨迹数目的影响

首先,在默认参数设置下,研究轨迹数目变化对算法性能的影响。具体而言,在波尔图数据集中,轨迹数目从 20,000 增加至 100,000;在北京数据集中,轨迹数目从 2,000 增加至 10,000。之所以采用该设置,是因为北京数据集的轨迹空间分布更加密集、采样间隔更小,因此任意两条轨迹更容易满足距离约束并形成联合移动对,从而带来更高的计算压力。从命中率结果来看,波尔图数据集上的命中率随轨迹数目增加呈现明显上升趋势(见图 4-3(c))。当轨迹数目由 20K 增加至 100K 时,命中率由约 0.70 提升至接近 0.88。这一现象说明,在轨迹规模较小时,CS-ACMC 可能因联合移动对数量较少而遗漏部分结构;随着轨迹数目增加,对象之间的连

接关系更加充分，社区结构更加清晰，基于共享邻居的聚类策略能够更准确地恢复真实伴随群体结构。相比之下，北京数据集上的命中率始终稳定在 0.91–0.93 之间（见图 4-3(a)），整体波动较小。这是因为北京数据集本身轨迹密度较高，在较小规模下已能形成相对稳定的联合移动行为图结构，因此随着规模增加，近似算法的结果质量提升空间有限，但整体保持较高水平。

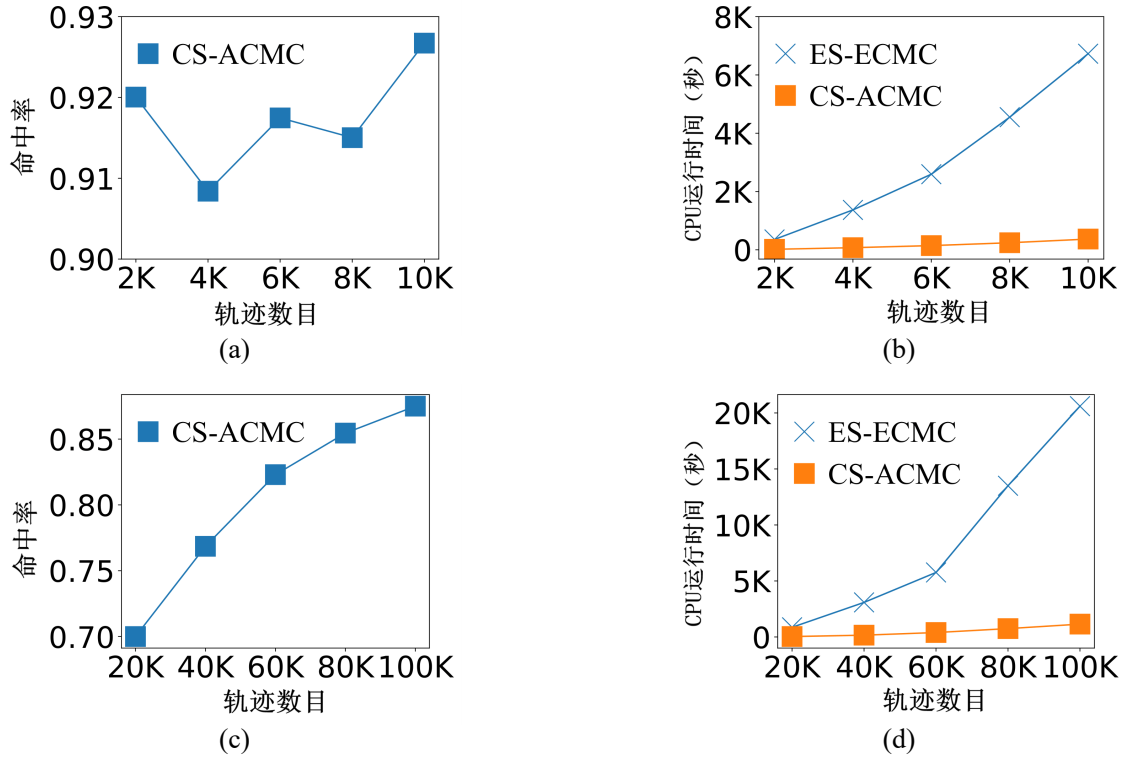


图 4-3 轨迹数目的影响。(a) 北京数据集上的命中率 (b) 北京数据集上的运行时间 (c) 波尔图数据集上的命中率 (d) 波尔图数据集上的运行时间

在算法效率方面，见图 4-3(b) 和图 4-3(d)，两种算法的运行时间均随轨迹数目增加而增长，但增长趋势存在显著差异。ES-ECMC 的运行时间随规模增加呈现近似二次增长趋势；而 CS-ACMC 的增长速度明显更缓。即便 ES-ECMC 在实验中采用了 40 线程并行计算，当轨迹数目达到 10,000（北京）和 100,000（波尔图数据集）时，其运行时间仍分别超过 8,000 秒和 20,000 秒；相比之下，CS-ACMC 在单线程条件下的运行时间始终控制在千秒以内。值得注意的是，波尔图数据集中捕获的联合移动对数量明显多于北京数据集，导致图结构更加稠密，从而增加了精确算法的计算负担。这也进一步放大了两种方法在大规模场景下的性能差距。综上所述，随着轨迹数目的增加，CS-ACMC 在保证较高命中率的同时，始终保持至少一个数量级的时间优势。这表明该方法在大规模轨迹数据分析场景下具有良好的可扩展性和实际应用价值。

4.5.2.2 联合移动时长阈值的影响

图 4-4 展示了当联合移动时长阈值 k 从 2 增加到 6 时两种算法的性能变化趋势。从命中率角度来看, 北京数据集上的命中率随 k 增大呈现缓慢上升趋势 (见图 4-4(a))。当 $k = 2$ 时命中率约为 0.89, 随后逐步提升并稳定在 0.91 以上。这表明在北京这种高密度轨迹环境下, 提高联合移动时长阈值能够有效过滤掉短暂、偶然的联合移动行为, 使近似方法更聚焦于结构稳定的联合移动对, 从而提升结果质量。相比之下, 波尔图数据集上的命中率随着 k 的增大整体呈下降趋势 (见图 4-4(c)), 从约 0.75 下降至 0.69 左右。这是因为波尔图数据集采样频率更高、时间跨度更长, 较大的 k 会显著减少满足条件的联合移动对数量, 使得图结构变得稀疏, 进而影响基于共享邻居的社区识别效果。因此, 近似方法在较大 k 下更容易遗漏部分真实联合移动行为结构。

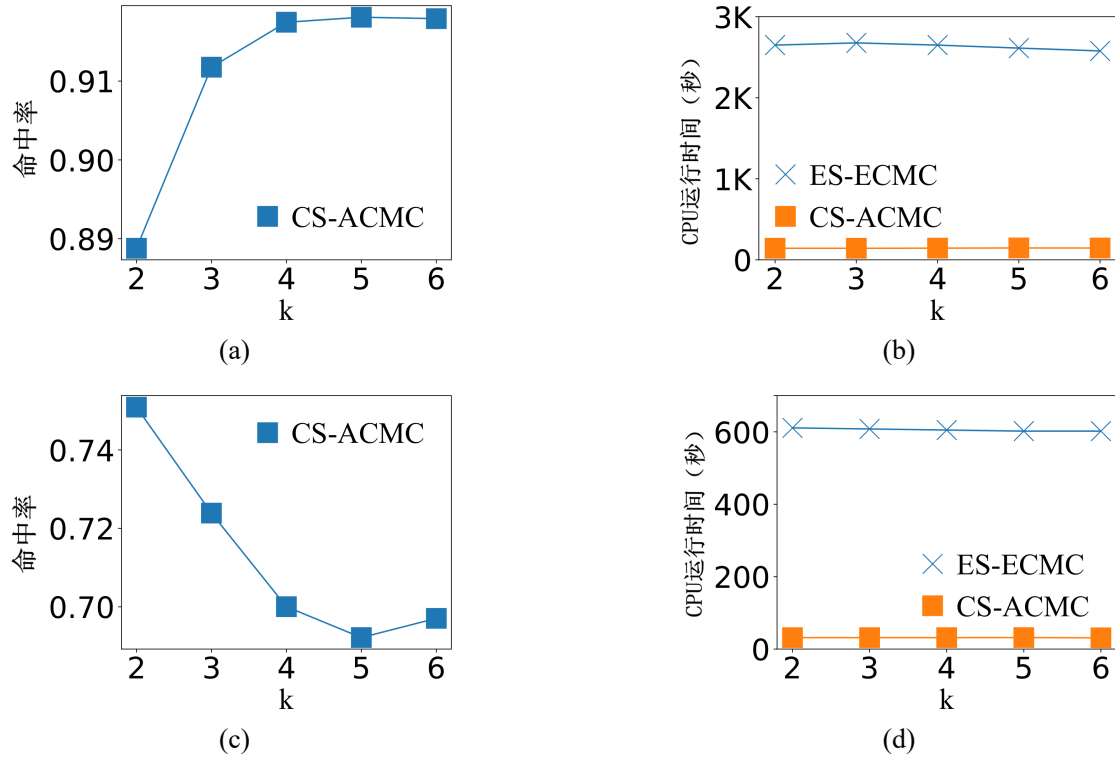


图 4-4 联合移动时长阈值的影响。(a) 北京数据集上的命中率 (b) 北京数据集上的运行时间 (c) 波尔图数据集上的命中率 (d) 波尔图数据集上的运行时间

在运行时间方面, 如图 4-4(b) 和图 4-4(d), 两种算法均未随着 k 的增大而出现明显下降趋势。尽管理论上更大的 k 会减少联合移动对及伴随群体数量, 从而降低后续聚类成本, 但实验结果表明, 总体运行时间基本保持稳定。其主要原因在于, 算法的核心计算开销集中在联合移动对的枚举或近似计算阶段, 而该阶段

需要对所有轨迹对进行比较，其复杂度与 k 无关；从联合移动对中生成伴随群体的代价相对较低，因此 k 的变化对总时间影响有限。同时可以观察到，在所有 k 取值下，CS-ACMC 的运行时间始终显著低于 ES-ECMC。即使在较小 k 的情况下，CS-ACMC 仍保持明显的效率优势，进一步验证了近似计算策略在降低计算成本方面的有效性。

4.5.2.3 伴随群体规模阈值的影响

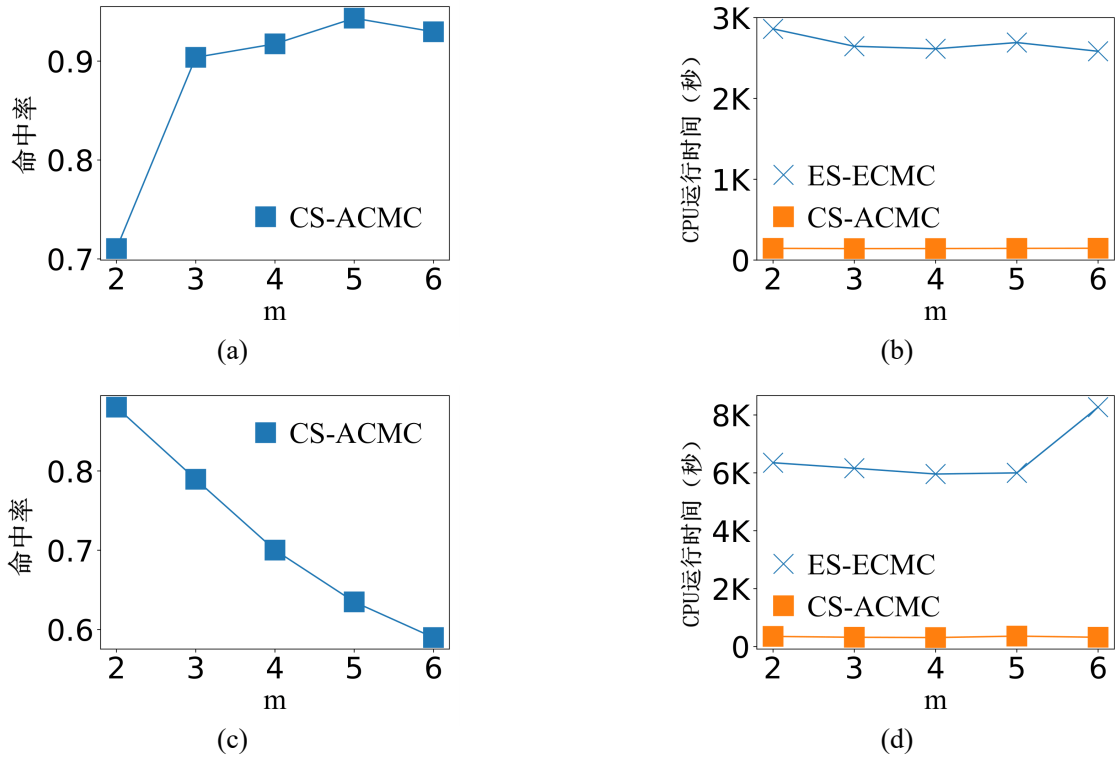


图 4-5 伴随群体规模阈值的影响。(a) 北京数据集上的命中率 (b) 北京数据集上的运行时间 (c) 波尔图数据集上的命中率 (d) 波尔图数据集上的运行时间

图 4-5 展示了当伴随群体规模阈值 m 从 2 变化到 6 时两种算法性能的变化趋势。直觉上，随着 m 的增大，满足条件的伴随群体数量逐渐减少，因为只有结构更加紧密、连接更加稳定的对象集合才能满足更严格的规模约束。这一变化对结果质量与运行时间均产生一定影响。如图 4-5(a)，北京数据集上的命中率随着 m 的增加呈显著上升趋势。当 $m = 2$ 时命中率约为 0.71，而当 $m = 5$ 或 6 时可提升至 0.93 左右。这说明在高密度轨迹环境下，小规模群体中包含较多边界性或弱连接结构，近似方法可能存在一定偏差；而当 m 增大后，仅保留结构更加紧密的核心群体，使得近似方法与精确方法在识别结果上更加一致，从而显著提升命中率。

相比之下, 波尔图数据集上的命中率随着 m 的增加呈下降趋势, 从约 0.88 下降至 0.59 左右。这主要是由于波尔图数据集数据的空间分布更为分散、轨迹时间跨度更长。当 m 较大时, 满足条件的群体数量显著减少, 图结构趋于稀疏, 近似联合移动对可能无法完整保留所有真实连接关系, 导致部分精确群体未被成功恢复, 从而降低命中率。

在效率方面, 见图 4-5(b) 和图 4-5(d), 伴随群体规模阈值的变化对总体运行时间影响较小。两种算法的运行时间曲线基本保持平稳, 仅在个别取值处存在轻微波动。这进一步验证了前文分析: 算法的主要时间开销集中在联合移动对的构建或近似计算阶段, 而最终伴随群体的筛选与过滤成本相对较低, 因此 m 的变化不会显著改变总体时间复杂度。同时可以观察到, 在所有 m 取值下, CS-ACMC 的运行时间始终远低于 ES-ECMC。即使在波尔图数据集中, 当 $m = 6$ 时 ES-ECMC 的运行时间超过 8,000 秒, 而 CS-ACMC 仍保持在数百秒以内, 体现出明显的规模优势。综上所述, 在不同 m 参数设置下, CS-ACMC 在保持较高命中率的同时, 显著降低了运行时间, 表现出良好的稳定性与参数鲁棒性。

4.5.2.4 聚类参数的影响

图 4-6 展示了当结构相似度阈值 μ 从 0.42 增加到 0.50 时算法在两个数据集上的命中率与运行时间变化情况。结构相似度阈值用于控制对象之间是否建立连接: μ 越小, 两个对象越容易满足相似性条件, 从而在联合移动行为图中形成更多边; μ 越大, 则连接关系更加严格, 图结构趋于稀疏。

从命中率变化趋势来看, 见图 4-6(c) 在波尔图数据集中, 随着 μ 的增大, 命中率呈现明显下降趋势。当 $\mu = 0.42$ 时取得最佳效果, 而当 $\mu = 0.50$ 时命中率下降至约 0.70。这表明在空间分布较为分散的数据环境下, 适当放宽结构相似度约束有助于保留潜在的真实连接关系, 而过高的阈值会导致部分真实联合移动候选集被过滤, 进而影响整体命中率。相比之下, 北京数据集上的命中率始终稳定在 0.91 以上, 且随 μ 的变化波动较小 (见图 4-6(a))。这一现象主要归因于北京数据集更高的轨迹采样密度和更强的空间聚集性。在高密度环境中, 即使提高相似度阈值, 真实联合移动行为之间仍能保持较高的结构一致性, 因此算法结果对 μ 的敏感度较低, 表现出更强的稳定性。

在效率方面 (见图 4-6(b) 和图 4-6(d)), 两种算法的运行时间均未随 μ 的变化出现显著波动, 说明结构相似度阈值主要影响图的连接密度, 而对整体计算流程的复杂度影响有限。然而, 即便 ES-ECMC 采用多线程实现, 其运行时间仍显著高于 CS-ACMC, 在两个数据集上均至少相差一个数量级。这表明本文提出的近似联合移动行为构建策略在降低候选连接数量的同时, 有效控制了计算开销。

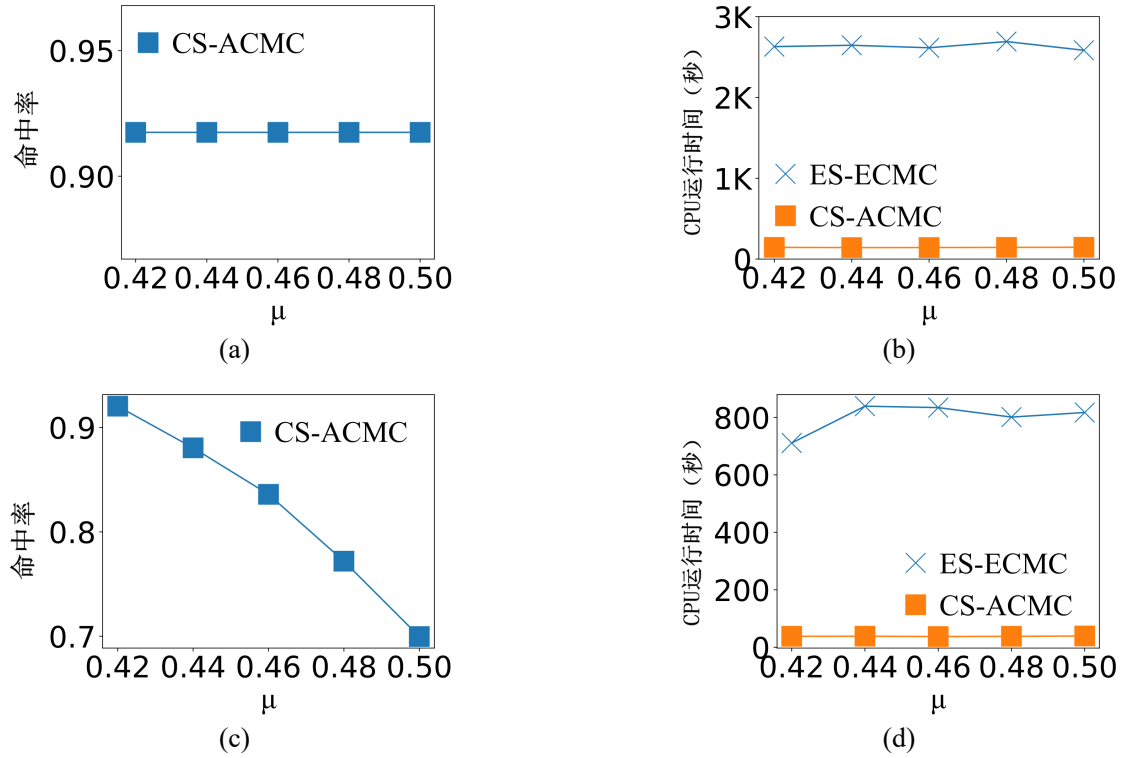


图 4-6 SCAN 聚类结构相似度参数的影响。(a) 北京数据集上的命中率 (b) 北京数据集上的运行时间 (c) 波尔图数据集上的命中率 (d) 波尔图数据集上的运行时间

4.6 本章小结

本研究针对传统联合移动行为模式定义过于严格、难以适应真实轨迹数据中噪声与不规则性的不足，提出了一种放宽的联合移动行为模式定义，在保证语义合理性的同时，提高了模型对实际数据的适应能力。

围绕上述问题，分别设计了一个精确搜索算法与一个社区感知的近似搜索算法。精确算法通过完整枚举与严格验证联合移动对，能够获得理论上的最优结果，但在大规模数据场景下面临较高的时间复杂度。为此，进一步提出了基于结构剪枝与候选压缩策略的近似算法，通过构建紧凑的联合移动行为图结构，有效减少冗余计算与无效候选，从而在保证较高命中率的前提下显著降低运行开销。两种算法在设计思路相互补充：前者保证结果精确性，后者强调规模可扩展性，共同构成了完整的解决方案体系。

在实验评估方面，基于北京与波尔图数据集两个真实世界轨迹数据集，从轨迹数目、联合移动时长阈值、伴随群体规模阈值等多个维度进行了系统分析。实验结果表明：(1) 在不同数据规模下，CS-ACMC 均表现出良好的可扩展性，其运行时间相比 ES-ECMC 至少降低一个数量级；(2) 在高密度数据集上，近似算法能

够保持稳定且较高的命中率，体现出较强的鲁棒性。

从应用角度来看，本章研究成果对于交通管理与城市计算具有重要意义。通过对潜在联合移动行为群体的提前识别，可以辅助分析车队行驶、异常聚集、隐性组织化出行等行为模式，为拥堵预测、路径规划优化以及公共安全监测提供数据支持。同时，所提出的近似建模与剪枝策略也可推广至其他大规模时空模式挖掘任务，具有较强的通用性。

第五章 面向大规模移动对象的暴露接触搜索

本章研究如何针对暴露接触搜索这一城市计算应用设计高性能搜索算法。传染性疾病长期以来一直是全球公共卫生领域面临的重要挑战。尤其在人员高度流动化和城市化背景下，个体之间频繁且复杂的时空交互显著加剧了疾病传播风险。暴露接触搜索作为疫情防控与风险追踪的关键技术手段，在及时识别潜在传播链条、阻断扩散路径方面发挥着不可替代的作用。在研究背景上，尽管已有方法在静态接触检测或离线轨迹分析方面取得了一定成果，但其大多假设数据是静态的且完整可得，缺乏对持续到达的轨迹位置流的支持，难以同时兼顾空间邻近约束与时间持续性约束，从而难以满足实时暴露接触监测的需求。通过分析现有工作的不足，本文引出并形式化定义了持续暴露接触搜索问题。在解决方案上，持续暴露接触搜索问题的精确求解需要在对象规模与时间维度双重增长的空间中进行大量距离计算与状态维护，其计算复杂度随查询对象数量和总体对象规模呈乘积增长，难以直接应用于实时系统。为此，本文提出了一种带有多种优化策略的精确分组处理算法，通过空间分组与组间剪枝机制显著减少对象级两两比较次数，并设计了基于距离上下界的预检查策略以避免不必要的精确验证。此外，为进一步满足高频数据流环境下的实时性需求，本文提出了一种近似分组处理算法，通过首尾优先检查与跳跃检测机制，在不产生漏检的前提下显著降低时间维度上的计算频率。最后，基于真实轨迹数据集开展了系统的实验评估。

5.1 引言

5.1.1 研究背景与意义

随着位置追踪设备以及基于位置服务的广泛应用，轨迹数据的规模与更新频率均呈现爆炸式增长。在多种轨迹查询任务中 [94, 98, 143, 144]，时空暴露接触搜索与追踪问题近年来受到了持续关注 [27, 118, 145]。一般而言，当两个移动对象在一段时间内保持较近的空间距离并产生潜在相互影响时，即可视为一次暴露接触。如何在大规模轨迹数据中高效且准确地识别此类暴露接触，对于诸多实际应用具有重要意义，包括疾病传播控制、公共卫生预警、信息扩散分析以及公共移动模式挖掘等。在突发公共卫生事件期间，及时搜索暴露接触对象不仅有助于限制人群传播范围，还能够为个体感染风险评估与资源调度提供数据支撑。因此，构建可扩展、低延迟且高精度的接触搜索技术，已成为时空数据挖掘领域和城市计算应用中的关键研究问题之一。

现有研究主要聚焦于面向单个查询对象的接触追踪任务，通常基于查询对象与数据对象之间的空间邻近关系进行建模和处理 [146, 147]，亦有部分工作通过计算轨迹片段之间的相似性来识别潜在接触关系 [148, 149]。然而，在疾病快速传播或大规模群体活动等场景下，感染个体数量可能在短时间内急剧增加，此时需要对海量移动对象执行连续且并行的接触搜索操作。相较于传统的单查询模式，一个有效的接触搜索框架不仅应支持大规模移动对象的高并发处理，还需在数据持续更新的环境下保证对每个对象的实时检测能力与结果一致性。

据所知，现有研究通常将该问题建模为独立查询处理问题、轨迹相似性搜索问题 [27, 120]，或多查询批处理问题 [119, 150]。这类方法要么仅适用于少量查询场景，要么假设输入数据为静态轨迹集合，难以适应持续更新的流式轨迹数据环境。因此，在面对大规模移动对象及大量并发查询时，这些方法在可扩展性与实时性方面均存在明显不足。此外，由于缺乏对全体对象状态的统一维护机制，现有方法也难以对每个移动对象持续提供准确且实时的接触检测结果，限制了其在实际动态场景中的应用价值。

5.1.2 研究内容与挑战

基于以上调研，一个有效的接触搜索框架应满足：在大规模、动态变化的时空环境中，将空间邻近关系与时间持续性约束进行联合建模，在保证检测准确性的同时，实现高效的实时处理。例如，在疫情暴发初期，大量个体的移动轨迹会在短时间内密集交汇，形成复杂的接触网络；又如，在大型公共活动或交通枢纽场景下，人群聚集与流动高度频繁，使得潜在暴露接触关系呈现持续且动态演化的特征。在此类场景中，若无法对轨迹位置流进行高效处理，将难以及时搜索关键暴露接触节点，从而影响风险控制决策。

在上述背景下，本文形式化定义并系统研究了持续暴露接触搜索问题。具体而言，给定一组移动对象，以及查询集合用于表示当前已确认的暴露接触案例，目标是在轨迹数据持续到达和更新的环境下，动态维护并实时扩展该查询集合，从而持续检测新的暴露接触案例。若某对象在空间距离阈值内，且在连续时间长度不少于给定阈值的时间区间内与查询集合中的任一对象共同移动，则该对象被判定为新的暴露接触案例，并被纳入查询集合中。该过程具有递归扩展特性，即新加入的对象将继续作为潜在传播源参与后续检测，从而形成动态传播链。图5-1给出了持续暴露接触搜索问题的一个示例。

示例 5.1（持续暴露接触搜索问题示例） 如图5-1，假设 o_1, o_2, o_3 为三个移动对象，初始查询集合为 $Q = \{o_1\}$ 。距离阈值 ϵ 设为 2 米，接触时长阈值 k 设为 3 分钟。图中使用红色、蓝色和紫色点分别表示已暴露接触对象（查询对象）的位

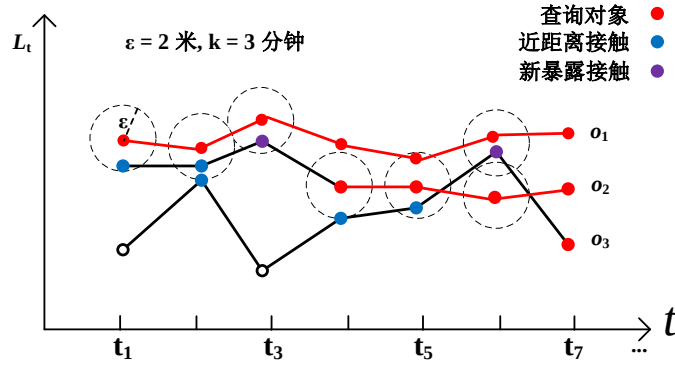


图 5-1 持续暴露接触搜索示例

置、与查询对象发生一次近距离接触的对象位置以及新确认的暴露接触对象位置。在连续时间区间 $[t_1, t_3]$ 内，对象 o_2 与查询对象 o_1 始终保持在 ϵ 距离范围内，且持续时长达到阈值 k ，因此在时刻 t_3 将 o_2 判定为新的暴露接触案例，并更新查询集合为 $Q = \{o_1, o_2\}$ ，同时输出当前实时结果。随后，在时间区间 $[t_4, t_6]$ 内，对象 o_3 与查询集合中的对象发生满足条件的暴露接触，因此在 t_6 时刻将其加入查询集合，查询更新为 $Q = \{o_1, o_2, o_3\}$ 。该示例说明，持续暴露接触搜索问题需要在数据动态演化的过程中持续维护传播集合，并对每一次集合扩展进行及时响应。

在问题描述上，给定一组移动对象及其随时间持续到达的位置流数据，同时给定一个动态更新的查询对象集合。本文所研究的持续暴露接触搜索问题旨在：在给定时间窗口内，持续搜索所有在空间上与至少一个查询对象保持近距离、且在时间上满足持续接触阈值约束的对象，并进一步识别其可能存在的直接或间接暴露接触关系。该问题同时涉及空间距离判定、时间连续性维护以及查询集合动态扩展等关键因素，具有数据规模大、更新频率高、实时性要求强等显著特点。具体而言，在连续到达的位置流中，以离散时间戳为基本处理单位，在每个时间快照上维护对象之间的空间邻近关系，并在滑动时间窗口内追踪接触持续状态。对于每一个非查询对象，需要判断其是否在连续时间段内满足距离阈值约束，并据此更新查询对象集合。

为持续暴露接触搜索问题设计一种兼具高效率与高精度的解决方案具有显著挑战性。一方面，移动对象规模可能达到百万甚至更高量级，直接基于两两对象比較的朴素方法在计算复杂度上难以扩展；另一方面，轨迹数据具有持续更新特性，算法不仅需要支持高吞吐量处理，还需保证低延迟响应，以满足实时监测需求。

5.1.3 研究成果与贡献

针对上述挑战，本文提出了一种精确的实时搜索算法，并结合多种优化策略以降低不必要的计算开销。此外，进一步设计了一种近似实时搜索算法，在保证

不产生漏检的前提下，通过引入可控的近似机制和分组处理策略进一步提升整体处理效率，从而在准确性与效率之间取得良好平衡。

本文的主要贡献总结如下：

- 形式化定义了持续暴露接触搜索问题，用于在大规模动态移动对象环境下持续维护最新暴露接触案例集合，并刻画了其递归扩展与实时更新特性。该问题为流行病传播监测与控制提供了统一的数据处理框架。
- 提出了一个精确分组处理算法和一个近似分组处理算法。同时，设计了分组过滤与预检查等优化策略，有效减少候选对象数量并降低计算开销。
- 进行了系统性的实验评估。在两个真实数据集上实验结果表明，所提出的方法在保证检测准确性的同时，能够在大规模移动对象场景下实现高效的实时暴露接触检测，展现出良好的可扩展性。

5.2 问题描述

以一组移动对象 $\mathcal{O} = \{o_1, \dots, o_n\}$ 以及查询对象集合 $\mathcal{Q}$ ($\mathcal{Q} \subseteq \mathcal{O}$) 作为输入。在实际应用场景中，移动对象可以表示行人或车辆，而查询集合中的对象可表示已被判定为高风险的个体。研究目标是在轨迹数据持续到达与更新的环境下，以实时方式检测暴露接触案例，并动态维护暴露接触集合。主要符号及其解释如表5-1所示。为形式化描述该问题，下面给出若干基本概念与定义。

表 5-1 主要符号解释

符号	说明
o	单个移动对象
$\mathcal{O}$	移动对象集合, $\mathcal{O} = \{o_1, \dots, o_n\}$
t	时间戳
l_t^o	对象 o 在时间 t 的位置, $l_t^o = ([x, y], t)$
$[x, y]$	对象在二维空间中的坐标
$L_t(\mathcal{O})$	时间 t 时所有移动对象的位置集合
ϵ	近距离接触空间距离阈值
k	暴露接触持续时长阈值
$\text{dist}(\cdot, \cdot)$	空间距离函数
$[t_s, t_e]$	连续时间区间, 其中 $t_e - t_s \geq k$ 则表示满足暴露接触时长

定义 5.1 (移动对象的位置流) 移动对象 o 在时间 t 的位置表示为 $l_t^o = ([x, y], t)$, 其中 $[x, y]$ 表示二维空间坐标, t 表示时间戳。由于对象持续移动, l_t^o 随时间动态更新, 形成位置流。使用 $L_t(\mathcal{O}) = \{l_t^o \mid o \in \mathcal{O}\}$ 表示时间 t 时刻所有移动对象的位置集合。该定义刻画了系统在任意时间点所接收和处理的瞬时空间状态。

定义 5.2 (近距离接触) 在给定距离阈值 ϵ 的条件下, 若对象 o_i 与对象 o_j 在时间 t 满足 $\text{dist}(l_t^{o_i}, l_t^{o_j}) \leq \epsilon$, 则称二者在时间 t 发生一次近距离接触。其中, $\text{dist}(\cdot, \cdot)$ 表示空间距离函数, 例如欧氏距离。该定义刻画了单个时间点上的瞬时接触关系。

定义 5.3 (暴露接触案例) 给定持续时长阈值 k , 若对象 o ($o \in \mathcal{O} \setminus \mathcal{Q}$) 与查询集合 $\mathcal{Q}$ 中任意对象在某一连续时间区间内保持近距离接触, 且该时间区间长度不少于 k , 则称对象 o 为一个暴露接触案例。该定义强调接触关系在时间维度上的连续性要求, 从而排除偶发性或瞬时性接触。

定义 5.4 (持续暴露接触搜索问题) 给定移动对象集合 $\mathcal{O}$ 、已暴露接触对象集合 $\mathcal{Q}$ 、接触距离阈值 ϵ 以及接触持续时长阈值 k , 持续暴露接触搜索问题旨在位置流持续更新的过程中, 实时检测满足暴露接触条件的对象, 并将其动态加入 $\mathcal{Q}$, 从而持续维护最新的暴露接触集合。该问题具有递归扩展特性, 即新识别的暴露接触对象将继续参与后续检测过程。

5.3 精确遍历算法

一种用于求解持续暴露接触搜索问题的精确遍历算法流程如下: 对于每一个时间戳 t , 系统枚举所有非查询对象 $o_i \in \mathcal{O} \setminus \mathcal{Q}$, 并与每个查询对象 $o_j \in \mathcal{Q}$ 进行距离计算。若满足 $\text{dist}(l_t^{o_i}, l_t^{o_j}) \leq \epsilon$, 则将对象 o_i 记录为在时间 t 与查询集合发生一次近距离接触。为持续监测接触状态, 在每个时间点维护一个哈希集合, 用于记录对象的当前连续接触时长或最近一次接触起始时间, 从而跟踪其在时间维度上的连续性。在检测过程中, 若某对象的近距离接触在尚未达到持续时长阈值 k 之前发生中断, 则重置其累计接触时长; 反之, 若对象在某一连续时间区间内持续满足近距离接触条件, 且持续时长不少于 k , 则将其判定为一个暴露接触案例。随后, 将新检测到的暴露接触对象加入查询集合 $\mathcal{Q}$, 并将更新后的 $\mathcal{Q}$ 作为当前时刻的实时结果输出。更新后的查询集合将在下一时间戳继续参与检测过程, 从而实现递归扩展。在每一个时间戳, ET 方法均需执行一次完整的两两比较操作, 其时间复杂度为 $O(|\mathcal{Q}| \times |\mathcal{O} \setminus \mathcal{Q}|)$ 。在最坏情况下, 当查询集合规模随时间不断增长时, 算法复杂度可接近 $O(|\mathcal{O}|^2)$ 。因此, 该方法在大规模移动对象环境下将产生显著的计算开销, 亟需更加高效的优化策略与索引机制。

5.4 精确分组处理算法

本小节提出了一种精确分组处理算法, 旨在高效回答持续暴露接触搜索问题的同时, 保证对每个移动对象进行精确且实时的检测。与直接两两比较的精确遍历方法不同, 精确分组处理算法不再显式计算每个非查询对象与每个查询对象之

间的距离,而是基于对象在当前时间戳的位置,对查询对象 Q 与非查询对象 $O \setminus Q$ 分别进行空间划分,从而构建查询组与非查询组。通过在组层面进行统一管理与计算,算法能够显著减少不必要的距离比较次数。

在整体流程上,精确分组处理算法采用两阶段处理框架以识别暴露接触案例。第一阶段为组级过滤阶段,算法首先计算查询组与非查询组之间的最小可能距离,并依据距离阈值判断哪些组对可能包含满足近距离接触条件的对象,从而生成潜在暴露接触候选集合。第二阶段为对象级精确验证阶段,针对第一阶段产生的候选对象,执行精确距离计算与连续时间判断,以确定其是否满足持续时长阈值的要求。通过先过滤后验证的分层策略,精确分组处理在保证结果精确性的同时显著降低了计算开销。此外,为进一步提升算法效率,设计了若干预检查策略,用于在组构建与候选生成之前提前排除明显不满足条件的对象或组。

5.4.1 组距离

在精确遍历算法中,为识别近距离接触关系,需要对每个非查询对象与每个查询对象执行显式的距离计算。当对象规模较大时,该两两比较过程将产生显著的时间开销。为避免在对象层面进行全面枚举,一种自然的思路是在组层面定义距离度量,以刻画两组位置之间的整体空间关系,从而实现更高层次的剪枝与过滤。现有研究中已提出多种用于衡量两组空间对象之间距离的度量方法,其中 Hausdorff 距离 [151] 是较具代表性的一种。Hausdorff 距离通过计算两组点集中任意一点到另一组点集的最大最小距离,刻画了两组对象之间的最坏情况偏差。然而,若两组对象分别包含 m 和 n 个位置点,则一次 Hausdorff 距离计算通常需要 $O(m \times n)$ 的时间复杂度。这意味着在大规模位置数据场景下,频繁计算此类组距离将带来较高的计算成本,难以满足实时处理需求。

因此,尽管组距离度量在理论上能够减少对象级别的比较次数,但若其自身计算复杂度过高,仍然无法从根本上解决持续暴露接触搜索问题在大规模环境下的效率瓶颈。为此,有必要设计一种计算代价更低、且适用于实时剪枝的组距离度量方法。

5.4.2 分组划分

为实现对近距离接触关系的快速搜索,本文基于对象在当前时间戳的位置,将所有移动对象划分为若干空间分组。其核心思想是利用空间局部性:空间上彼此接近的对象更有可能发生近距离接触,而距离较远的对象则可以在早期阶段直接排除。通过引入分组机制,算法能够在组层面完成初步剪枝,从而显著减少后续对象级别的精确计算开销。

具体而言，构建一个规则网格索引 $\mathcal{C}$ 对底层空间进行离散化划分。每个网格单元为边长为 $\frac{\sqrt{2}}{2}\epsilon$ 的正方形，其中 ϵ 表示近距离接触的距离阈值。该单元边长设计保证了任意两个位于同一单元内的对象之间的最大欧氏距离不超过 ϵ 。对于任意单元格 $c \in \mathcal{C}$ ，落入该单元中的查询对象与非查询对象分别构成查询组 $\mathcal{O}_c^q$ 与非查询组 $\mathcal{O}_c$ 。为便于高效访问与更新，查询组与非查询组分别进行存储与索引管理，从而支持后续分阶段处理流程。

5.4.3 预检查策略

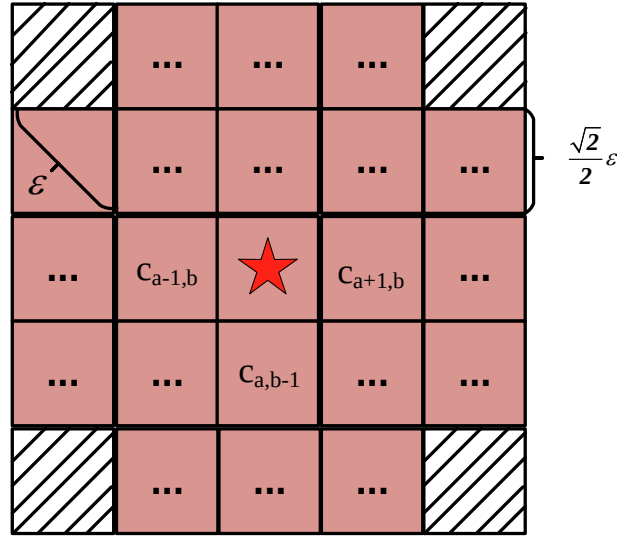


图 5-2 潜在的近距离接触候选集

图 5-2 展示了当查询组位于单元格 c_{ab} （以星号标记）时，如何识别可能包含暴露接触案例的候选分组。根据空间距离约束，仅当某单元格 c_{ij} 满足 $|i - a| \leq 2 \wedge |j - b| \leq 2 \wedge |i - a| + |j - b| < 4$ ，该单元格中的对象才有可能与 c_{ab} 中的查询对象在距离阈值 ϵ 内发生近距离接触。将所有满足上述条件的单元格集合记为 $SC = \{c_{ij} \in \mathcal{C} \mid |i - a| \leq 2 \wedge |j - b| \leq 2 \wedge |i - a| + |j - b| < 4\}$ 。如图中红色单元格所示，仅这些单元格内的对象组被视为潜在暴露接触候选，其余单元格（例如灰色单元格）由于与 c_{ab} 中对象的最小可能距离已超过 ϵ ，可在组级阶段直接剪枝。值得注意的是， $|SC|$ 为常数，其取值为 21，因此每个查询组在组级过滤阶段仅需检查有限数量的邻近单元，从而有效控制计算复杂度。

为识别非查询组 $\mathcal{O}_c$ 中所有可能与查询组 $\mathcal{O}_c^q$ 发生近距离接触的对象，若直接在对象层面进行两两距离计算，则时间复杂度为 $O(|\mathcal{O}_c^q| \times |\mathcal{O}_c|)$ 。当组规模较大时，该计算代价仍然较高。为进一步降低计算开销，本文基于空间分组结构设计了若干干预检查策略，在进入对象级精确验证之前完成有效剪枝。

具体而言，首先分别扫描 $\mathcal{O}_c^q$ 与 $\mathcal{O}_c$ 中的所有位置点，构建其最小外包矩形

(Minimum Bounding Rectangle, MBR), 记为 $MBR(\mathcal{O}_c^q)$ 与 $MBR(\mathcal{O}_{c'})$ 。在此基础上, 可通过计算两个 MBR 之间的最小可能距离与最大可能距离, 分别得到集合 $\{\text{dist}(l^{o_i}, l^{o_j}) \mid o_i \in \mathcal{O}_c^q, o_j \in \mathcal{O}_{c'}\}$ 的下界 $LB(\mathcal{O}_c^q, \mathcal{O}_{c'})$ 与上界 $UB(\mathcal{O}_c^q, \mathcal{O}_{c'})$, 其具体形式如公式 5-1 所示。与 Hausdorff 距离需要 $O(|\mathcal{O}_c^q| \times |\mathcal{O}_{c'}|)$ 的计算复杂度不同, 上述上下界计算仅依赖于 MBR 的边界信息, 因此整体时间复杂度为 $O(|\mathcal{O}_c^q| + |\mathcal{O}_{c'}|)$, 为线性时间。

基于该上下界信息, 可以进行如下剪枝判断: 当 $UB(\mathcal{O}_c^q, \mathcal{O}_{c'}) < \epsilon$ 时, 说明两组中任意对象对之间的最大可能距离仍小于阈值 ϵ , 因此可直接判定 $\mathcal{O}_{c'}$ 中所有对象在当前时刻均与查询组发生近距离接触, 无需进一步逐一验证; 相反, 当 $LB(\mathcal{O}_c^q, \mathcal{O}_{c'}) > \epsilon$ 时, 说明两组之间的最小可能距离已超过阈值, 可安全剪枝 $\mathcal{O}_{c'}$, 避免进入对象级计算。仅当 ϵ 落在 $[LB, UB]$ 区间内时, 才需要执行精确的对象级距离验证。通过该预检查策略, 算法能够在保证精确性的前提下显著减少不必要的距离计算次数。

$$\begin{aligned} UB(\mathcal{O}_c^q, \mathcal{O}_{c'}) &= \max(\text{dist}(MBR(\mathcal{O}_c^q), MBR(\mathcal{O}_{c'}))) \\ LB(\mathcal{O}_c^q, \mathcal{O}_{c'}) &= \min(\text{dist}(MBR(\mathcal{O}_c^q), MBR(\mathcal{O}_{c'}))) \end{aligned} \quad (5-1)$$

5.4.4 算法细节

算法 5-1 给出了精确分组处理算法的整体流程。算法输入包括: 移动对象集合 $\mathcal{O}$, 位置流 $L_t(\mathcal{O}) = \{l_t^o \mid o \in \mathcal{O}\}$ (表示在时间戳 t 时所有对象的位置), 查询对象集合 $\mathcal{Q}$, 接触距离阈值 ϵ , 以及接触持续时长阈值 k ; 输出为更新后的查询对象集合, 其中新增当前时刻新识别的暴露接触案例。

在初始化阶段, 对于所有非查询对象 $o \in \mathcal{O} \setminus \mathcal{Q}$, 将其接触时长 $cd(o)$ 置为 0。在搜索过程中, 算法使用集合 Π 记录至少包含一个查询对象的网格单元, 用于标识需要参与后续处理的查询群体; 同时使用集合 A 存储当前时间戳下检测到的近距离接触对象。初始时, Π 与 A 均被置为空集 (第 1 行)。

算法由两个主要阶段构成: 1) 群体划分阶段 (第 2–7 行); 2) 基于群体的近距离接触搜索阶段 (第 8–13 行)。在群体划分阶段, 对于每一个到达的位置 l_t^o , 首先确定其所属的网格单元 c 。若对象 o 属于查询集合 $\mathcal{Q}$, 则将其加入查询群体 $\mathcal{O}_c^q$, 并将单元格 c 加入集合 Π ; 否则, 将其加入对应的非查询群体 $\mathcal{O}_c$ 。经过该阶段处理后, 所有查询群体与非查询群体均构建完成, 为后续组级过滤提供基础。在近距离接触搜索阶段, 算法针对每一个查询群体 $\mathcal{O}_c^q$ 及其邻近、具有潜在暴露接触风险的非查询群体 $\mathcal{O}_{c'}$, 执行预检查策略 (第 10–13 行)。具体而言, 首先计算两群体之间距离的上下界 (参见公式 5-1)。若上界不大于阈值 ϵ , 则可直接将 $\mathcal{O}_{c'}$ 中所

算法 5-1: 精确分组处理算法

输入: 1) 移动对象集合 $\mathcal{O}$;
 2) 位置流 $L_t(\mathcal{O}), L_{t+1}(\mathcal{O}), \dots$;
 3) 查询对象集合 $\mathcal{Q} \subseteq \mathcal{O}$;
 4) 接触距离阈值 ϵ ;
 5) 接触时长阈值 k 。
输出: 更新后的查询对象集合 $\mathcal{Q}$ 。

```

1  初始化:  $\forall o \in (\mathcal{O} \setminus \mathcal{Q}): cd(o) \leftarrow 0; \Pi \leftarrow \emptyset; A \leftarrow \emptyset;$ 
2  for 位置点  $l_t^o \in L_t(\mathcal{O})$  of object  $o$  do /* 对象位置映射到网格 */
3      令  $c$  为覆盖位置  $l_t^o$  的网格单元;
4      if  $o \in \mathcal{Q}$  then
5           $\Pi \leftarrow \{\Pi, c\}; \mathcal{O}_c^q \leftarrow \{\mathcal{O}_c^q, o\};$ 
6      else
7           $\mathcal{O}_c \leftarrow \{\mathcal{O}_c, o\};$ 
8  for 查询单元  $c \in \Pi$  do /* 处理查询单元 */
9      for 非查询单元  $c' \in SC(c)$  do /* 检查相邻非查询单元 */
10         if  $UB(\mathcal{O}_c^q, \mathcal{O}_{c'}) \leq \epsilon$  then
11              $A \leftarrow \{A, \mathcal{O}_{c'}\};$ 
12         else if  $LB(\mathcal{O}_c^q, \mathcal{O}_{c'}) \leq \epsilon$  then
13              $A \leftarrow \{A, ET(\mathcal{O}_c^q, \mathcal{O}_{c'})\};$ 
14 更新接触时长  $cd(o)$ ;
15  $\mathcal{Q} \leftarrow \{\mathcal{Q} \cup o \mid cd(o) \geq k\};$ 
16 return 更新后的  $\mathcal{Q}$ ; /* 实时结果 */
    
```

有对象加入当前近距离接触集合 A ; 若下界大于 ϵ , 则安全剪枝该非查询群体, 因为其中任一对象均不可能与查询群体中的对象形成近距离接触。仅当 ϵ 位于上下界区间内时, 才需进一步在对象层面计算成对距离, 以完成精确验证。在完成群体级与对象级检测之后, 算法对所有对象的接触时长进行更新 (第 14 行)。对于不属于当前接触集合 A 的对象 $o \in \mathcal{O} \setminus A$, 其接触时长 $cd(o)$ 被重置为 0, 表示其连续接触在达到阈值 k 之前已发生中断; 对于属于集合 A 的对象, 则递增其接触时长。随后, 根据更新后的 $cd(o)$, 将所有满足 $cd(o) \geq k$ 的对象加入查询集合 $\mathcal{Q}$ (第 15 行), 从而识别新的暴露接触案例。最后, 输出更新后的 $\mathcal{Q}$ 作为当前时间戳的实时搜索结果 (第 16 行), 并进入下一时间戳的处理流程。

5.4.5 时间复杂度分析

在每个时间戳, 精确分组处理算法首先完成群体划分操作。由于每个对象仅需根据其当前位置确定所属网格单元并插入相应群体, 因此构建所有查询群体与

非查询群体的时间复杂度为 $O(|\mathcal{O}|)$ 。在近距离接触搜索阶段，算法仅针对包含查询对象的网格单元集合 Π 进行处理。设每个查询群体与非查询群体中平均包含的对象数量分别为 p 与 q ，则对于每一个查询群体，算法至多检查常数个相邻单元 SC （其中 $|SC|$ 为常数，参见群体划分过程）。在最坏情况下，对每一对候选群体需执行对象级精确验证，其时间开销为 $O(p \times q)$ 。因此，该阶段的总体时间复杂度可表示为 $O(|\Pi| \times |SC| \times p \times q) = O(|\Pi| \times p \times q)$ ，其中 $|SC|$ 为常数因子。在极端情况下，所有查询对象可能集中于同一网格单元，而所有非查询对象集中于某一相邻单元。例如，当 $|\Pi| = 1$ 、 $p = |\mathcal{Q}|$ 且 $q = |\mathcal{O} \setminus \mathcal{Q}|$ 时，由于 $|SC|$ 为常数，每一轮 EGP 的时间复杂度退化为 $O(|\mathcal{Q}| \times |\mathcal{O} \setminus \mathcal{Q}|)$ ，与 ET 方法在数量级上相同。然而，在大多数实际应用场景中，移动对象在空间上呈现分散分布特性，使得单个网格单元内的对象数量远小于全局规模，即通常有 $p \ll |\mathcal{Q}|$ 且 $q \ll |\mathcal{O}|$ 。因此，EGP 能够显著降低对象级比较次数，从而在整体上优于直接两两比较方法，并在保证精确性的前提下实现更好的可扩展性。

5.5 近似分组处理算法

为进一步提升暴露接触案例搜索的效率，并在保证结果可控误差范围内显著降低计算开销，本文提出了一种近似分组处理算法。该算法在精确分组处理框架的基础上，引入时间维度上的剪枝策略与跳跃式计算机制，从而在大规模时空数据环境下实现更优的计算性能。本节将详细介绍所采用的近似策略及其设计思想。

5.5.1 首尾优先检查策略

在连续时间戳之间，往往存在大量对象在空间上彼此接近地移动。然而，从暴露接触案例判定的角度来看，仅在单个时间戳上发生的短暂接触通常不具备实际意义。若某一非查询对象在其接触持续时长尚未达到阈值 k 之前便中断与查询对象的近距离接触，则该对象不可能成为最终的暴露接触案例。因此，在连续时间窗口内识别具有持续接触潜力的对象，是提高检测效率的关键。

基于上述观察，本文提出首尾优先检查策略。与按照时间顺序逐一计算每个时间戳上的近距离接触集合不同，该策略首先关注时间窗口两端的接触情况。具体而言，在当前时刻 t ，分别计算近距离接触集合 A_t 与 A_{t-k+1} ，其中 A_t 表示时刻 t 的近距离接触对象集合。通过求取两者的交集 $A_t \cap A_{t-k+1}$ ，可以获得在时间窗口 $[t-k+1, t]$ 内仍具有持续接触可能性的候选对象集合。只有属于该交集的对象，才可能在整个时间窗口内保持连续接触关系。在此基础上，对于区间 $[t-k+1, t]$ 内的中间时间戳，仅需针对上述候选集合进一步验证其接触情况，而无需对全部

对象进行扫描。该策略在时间维度上实现了有效剪枝，显著减少了不必要的距离计算与状态维护开销，从而提升了整体计算效率。

5.5.2 跳跃检查

在实际应用中，接触持续时长阈值 k 通常远大于单个时间间隔。若对象 o 在连续 k 个时间段 $[t_i, t_{i+k-1}]$ 内被视为潜在暴露接触对象，并且在时间戳 t_i 与 t_{i+k-1} 均与至少一个查询对象发生近距离接触，则可以将对象 o 认定为一个近似暴露接触案例。该判定方式弱化了对中间时间点逐一验证的依赖，为进一步降低计算成本提供了可能。

基于上述近似判定原则，本文提出跳跃检查策略。具体而言，不再对所有时间戳上的位置数据进行处理，而仅在部分离散时间戳集合 $T = \{0, m, 2m, \dots\}$ 上执行分组与接触检测操作，其中 m 表示跳跃长度。当 $m = 1$ 时，算法退化为逐时间戳处理的形式；当 $m > 1$ 时，则通过降低处理频率来减少总体计算次数。对于不属于集合 T 的时间戳 $t \notin T$ ，算法采用保守假设，即暂时将所有对象视为近距离接触对象，以避免潜在暴露接触对象被遗漏。在随后的跳跃时间点上，再结合首尾优先检查策略进行统一验证。通过这种方式，算法在控制漏检风险的前提下有效压缩了时间维度上的计算规模，从而在大规模连续时空数据环境下获得更高的运行效率与更好的可扩展性。

5.5.3 算法细节

算法 5-2 给出了近似分组处理算法的伪代码。与精确分组处理方法相比，近似分组处理算法额外引入跳跃长度 m 作为输入参数，用以控制时间维度上的检测频率。算法的输出为更新后的查询对象集合 $\mathcal{Q}$ ，其中既包含原有查询对象，也包含最新识别的近似暴露接触案例。为刻画对象作为潜在暴露接触案例的持续状态，定义 $cd'(o)$ 表示对象 o 的近似接触持续时长，并在算法开始时对所有非查询对象进行初始化（第 1 行）。

在执行过程中，仅当当前时间戳满足 $t \bmod m = 0$ 时，算法才在该时间点执行潜在暴露接触案例检测（第 2 行），从而降低整体计算频率。与精确分组处理相同，算法使用 Π 表示至少包含一个查询对象 $o \in \mathcal{Q}$ 的网格单元集合，并分别以 A 与 A' 表示近距离接触集合与潜在暴露接触案例集合。对于对象 o 在时刻 t 的位置 $l_t^o \in L_t(\mathcal{O})$ ，首先执行与精确分组处理相同的网格划分操作，构建查询分组 $\mathcal{O}_c^q$ 与非查询分组 $\mathcal{O}_c$ （第 4–10 行），从而在空间维度上完成对象组织。随后，对于每个包含查询对象的单元 $c \in \Pi$ ，扫描其相邻单元集合 $SC(c)$ ，并将其中的非查询对象视为潜在暴露接触候选对象，加入集合 A' ，同时增加其近似接触持续时长 $cd'(o)$ （第

算法 5-2: 近似分组处理算法

输入: 1) 移动对象集合 $\mathcal{O}$;
 2) 位置流 $L_t(\mathcal{O}), L_{t+1}(\mathcal{O}), \dots$;
 3) 查询对象集合 $\mathcal{Q} \subseteq \mathcal{O}$;
 4) 接触距离阈值 ϵ ;
 5) 接触持续时长阈值 k ;
 6) 跳跃步长 m 。

输出: 实时近似暴露接触结果 (更新后的查询集合 $\mathcal{Q}$)。

```

1  初始化: 对所有  $o \in (\mathcal{O} \setminus \mathcal{Q})$ , 令  $cd'(o) \leftarrow 0$ ;
2  if  $t \bmod m = 0$  then                                     /* 仅在跳跃时间点进行处理 */
3       $\Pi \leftarrow \emptyset$ ;  $A' \leftarrow \emptyset$ ;
4      foreach 对象  $o$  在时刻  $t$  的位置  $l_t^o \in L_t(\mathcal{O})$  do      /* 分组阶段 */
5          令  $c$  为覆盖  $l_t^o$  的网格单元;
6          if  $o \in \mathcal{Q}$  then                                       /* 查询对象 */
7               $\Pi \leftarrow \Pi \cup \{c\}$ ;
8               $\mathcal{O}_c^q \leftarrow \mathcal{O}_c^q \cup \{o\}$ ;
9          else                                                     /* 非查询对象 */
10              $\mathcal{O}_c \leftarrow \mathcal{O}_c \cup \{o\}$ ;
11      foreach 查询单元  $c \in \Pi$  do                               /* 近邻单元扫描 */
12          foreach 非查询单元  $c' \in SC(c)$  do
13               $A' \leftarrow A' \cup \mathcal{O}_{c'}$ ;
14      更新所有对象的  $cd'(o)$ ;                                     /* 更新接触持续时长 */
15       $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{o \mid o \in A_t \cap A_{t-k+1} \wedge cd'(o) \geq k\}$ ; /* 头尾优先检查并更新查询集合 */
16 return 更新后的  $\mathcal{Q}$  作为实时近似结果;
  
```

11–13 行)。为保证持续时长的正确性, 对于未出现在 A' 中的对象 o , 重置其 $cd'(o)$ 为零 (第 14 行), 以反映接触关系的中断; 对于出现在 A' 中的对象 o , 使其持续时长加 1。对于满足 $cd'(o) \geq k$ 的对象, 算法进一步检查其在时间戳 t 与 $t - k + 1$ 是否均属于近距离接触集合 A 。若上述条件成立, 则将对象 o 判定为近似暴露接触案例, 并将其加入查询集合 $\mathcal{Q}$ (第 15 行)。最后, 返回更新后的 $\mathcal{Q}$ 作为实时近似检测结果 (第 16 行)。

5.5.4 时间复杂度分析

网格分组阶段需要对每个对象执行一次单元映射与插入操作, 因此其时间复杂度为 $O(|\mathcal{O}|)$ 。由于不再执行对象级的精确两两距离计算, 而仅在网格层面扫描包含查询对象的单元及其相邻单元, 因此潜在暴露接触案例的搜索开销为 $O(|\Pi| \times |SC|)$ 。考虑到 $|SC|$ 为常数, 该部分复杂度可简化为 $O(|\Pi|)$ 。在最坏情况下, 若任意两个查询对象位于不同的网格单元中, 则有 $|\Pi| = |\mathcal{Q}|$ 。因此, 在单轮跳跃检测

中的整体时间复杂度为 $O(|\mathcal{O}| + |\mathcal{Q}|)$ 。同时需要指出, $|\Pi|$ 的上界为 $|\mathcal{Q}|$, 因而该复杂度界是紧致的。结合跳跃长度 m 的设置, 实际平均计算开销还可进一步降低, 从而在大规模时空数据环境下表现出良好的可扩展性。

5.6 实验研究

在本节中, 开展了广泛的实验对本章算法性能进行评估。首先介绍实验设定, 包括实验所使用的数据集, 运行环境等, 然后给出实验结果与分析。实验结果主要关注所提出的算法的运行时间以及搜索到的暴露接触案例数量。实验的目的在于, 通过设置不同的参数 (查询对象数目, 距离阈值, 接触时长阈值等), 对比不同的算法的运行时间和暴露接触案例数量并进行分析。

5.6.1 实验设置

5.6.1.1 数据准备

表 5-2 评估参数设定

	北京 (Beijing)	波尔图 (Porto)
经度	[116.250, 116.550]	[-8.735, -8.156]
纬度	[39.830, 40.030]	[40.953, 41.307]
距离阈值 ϵ	{2, 4, 6, 8, 10 米}; 默认 2 米	{2, 4, 6, 8, 10 米}; 默认 2 米
接触时长阈值 k	{5, 10, 15, 20, 25}; 默认 15	{5, 7, 9, 11}; 默认 15
查询数量 $ \mathcal{Q} $	{2, 4, 6, 8, 10} $\times 10^2$; 默认 600	{2, 4, 6, 8, 10} $\times 10^4$; 默认 100,000

实验基于两个真实世界的轨迹数据集开展。第一个数据集来源于北京出租车数据集, 包含在北京一周内跟踪到的 10,357 辆出租车的轨迹, 总数据点数量约为 1,500 万。第二个数据集采集自葡萄牙波尔图市, 时间跨度为 19 个月, 包含超过 170 万条轨迹。在北京数据集中, 平均采样时间间隔为 177 秒, 对应的平均移动距离约为 623 米; 而在波尔图数据集中, 每辆出租车以 15 秒的间隔上报其位置信息。时间戳的取值范围被限制在 24 小时之内。在数据预处理中, 限制经纬度范围如表 5-2 所示, 以保证采样位置具有足够高的空间密度。此外, 过滤掉采样点数量较少的轨迹, 具体而言, 仅保留采样点数量不少于 10 的轨迹用于实验。为了能够在任意时间点计算对象之间的距离, 通过线性插值对所有轨迹进行时间对齐, 使得每个对象以固定频率返回其实时位置, 其中北京和波尔图数据集的频率分别为 10 秒和 5 秒。最终, 北京和波尔图数据集中分别包含 296,364,033 和 23,767,470 个有效位置点。在不失一般性的前提下, 从 $\mathcal{O}$ 中随机选取部分对象 $\mathcal{Q}$ 作为初始查询对象。默认评估参数设置如表 5-2 所示, 该设置基于两个数据集的具体数据分布。

5.6.1.2 比较算法

表 5-3 比较算法

算法标识	算法描述
ET	精确遍历算法
EGP	精确分组处理算法
AGP	近似分组处理算法

表5-3列举了实验研究中的比较算法,包括精确遍历算法(Exact Traversal, ET)、精确分组处理算法(Exact Group Processing, EGP)以及近似分组处理算法(Approximate Group Processing, AGP)。对于 AGP,默认的跳跃长度设置为 2。效率评估和效果评估所采用的性能指标分别为 CPU 时间以及搜索的暴露接触案例总数。CPU 时间定义为每一轮算法运行的平均执行时间。此外,还评估了 AGP 的准确性,其计算方式为真实暴露接触案例数量与近似暴露接触案例数量之比。

5.6.1.3 实验环境

所有算法均采用 Java 实现,并在配备 AMD Ryzen 5 处理器(3.6GHz)和 32GB 内存的 Windows 10 平台上进行评测。除非另有说明,实验结果均为在不同查询对象输入条件下进行 20 次独立实验后的平均值。

5.6.2 实验结果与分析

5.6.2.1 查询对象数量的影响

首先,在默认参数设置下,研究了查询对象数量 $|Q|$ 对算法性能的影响。从图 5-3(c) 和图 5-3(a) 可以观察到,随着 $|Q|$ 的增加,三种算法所检测到的暴露接触案例数量整体呈上升趋势,但不同数据集上的增长模式存在差异。在波尔图数据集上,暴露接触案例数量随 $|Q|$ 的增加呈现出近似线性增长,这主要是由于波尔图数据集中对象分布较为密集且轨迹重叠程度较高,当查询对象数量增加时,与其发生持续接触的对象数量显著增多。相比之下,在北京数据集上,暴露接触案例数量的增长趋势相对平缓,说明该数据集中的空间分布更加分散,新增查询对象所带来的额外接触关系较为有限。可以进一步看到,ET 与 EGP 在两个数据集上得到的暴露接触案例数一致,这是因为二者均采用精确的距离计算与判定机制,保证了检测结果的一致性与正确性。AGP 所得到的结果与精确算法高度接近,仅在个别情况下存在轻微差异,说明所提出的近似策略在有效降低计算开销的同时,能够较好地保持结果质量,不会对整体统计特性产生显著影响。

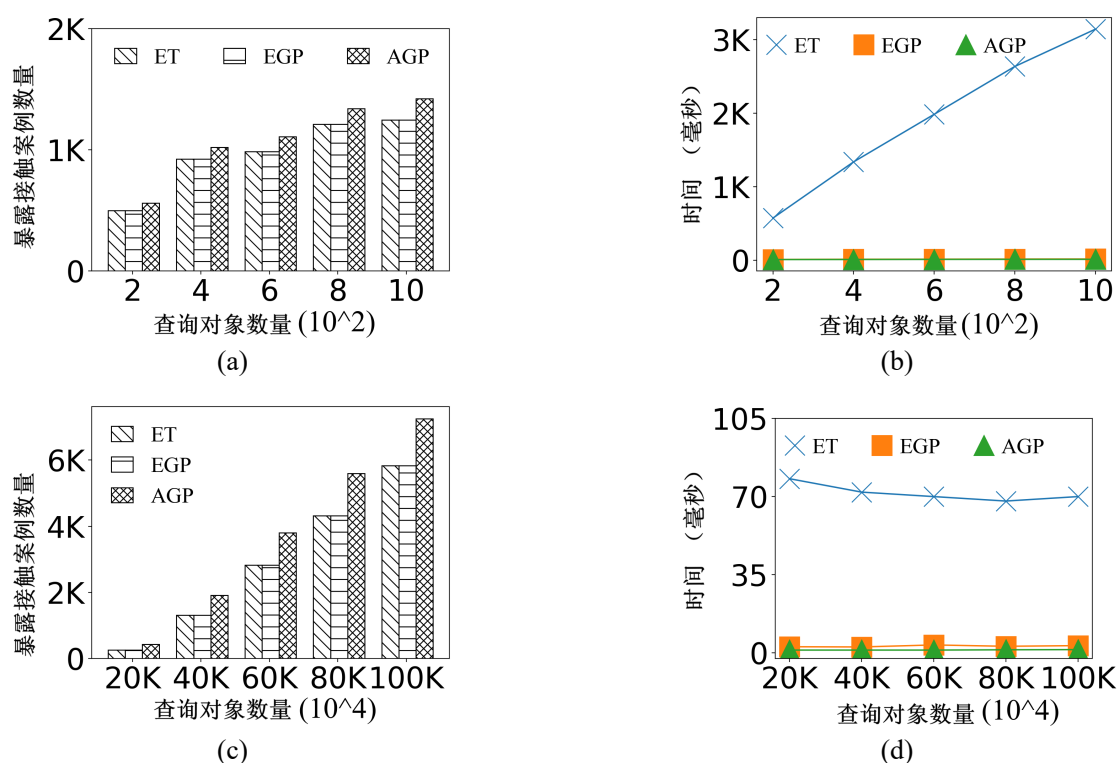


图 5-3 查询对象数量的影响。(a) 北京数据集上暴露接触案例数量；(b) 北京数据集上算法运行时间；(c) 波尔图数据集上暴露接触案例数量；(d) 波尔图数据集上算法运行时间

在运行时间方面，如图 5-3(b) 和图 5-3(d) 所示，随着 $|Q|$ 的增加，ET 的 CPU 时间显著上升，且增长趋势近似线性。在北京数据集上，当 $|Q|$ 取较大值时，ET 的运行时间超过 1,000 ms，最高达到数千毫秒，表明其难以满足实时暴露接触检测的需求。相比之下，EGP 与 AGP 的运行时间始终保持在较低水平，且随 $|Q|$ 的变化波动较小。这是因为两种组处理方法均避免了全局两两距离计算，其计算开销主要与包含查询对象的网格单元数量相关，而该数量远小于全体对象规模。此外，AGP 在 CPU 时间上进一步优于 EGP，这得益于跳跃检查与首尾优先策略对时间维度的有效剪枝，使得算法无需在每一个时间戳上执行完整检测。总体而言，与 ET 相比，EGP 和 AGP 至少将 CPU 时间降低了一个数量级，在保持检测精度基本不变的前提下显著提升了处理效率，验证了所提出方法在大规模时空数据环境下的实用性与可扩展性。

5.6.2.2 距离阈值的影响

直观而言，较大的距离阈值 ϵ 会放宽近距离接触的判定条件，使得对象更容易被识别为与查询对象发生接触关系。图 5-4 展示了当 ϵ 从 2 米逐步增加至 10 米时，

各算法在北京和波尔图数据集上的性能变化情况。可以观察到，随着 ϵ 的增大，三种算法检测到的暴露接触案例数量均呈现单调增长趋势。这是因为更大的距离阈值扩大了空间匹配范围，从而显著提高了对象之间发生接触的概率，并进一步增加了满足持续时长阈值的对象数量。

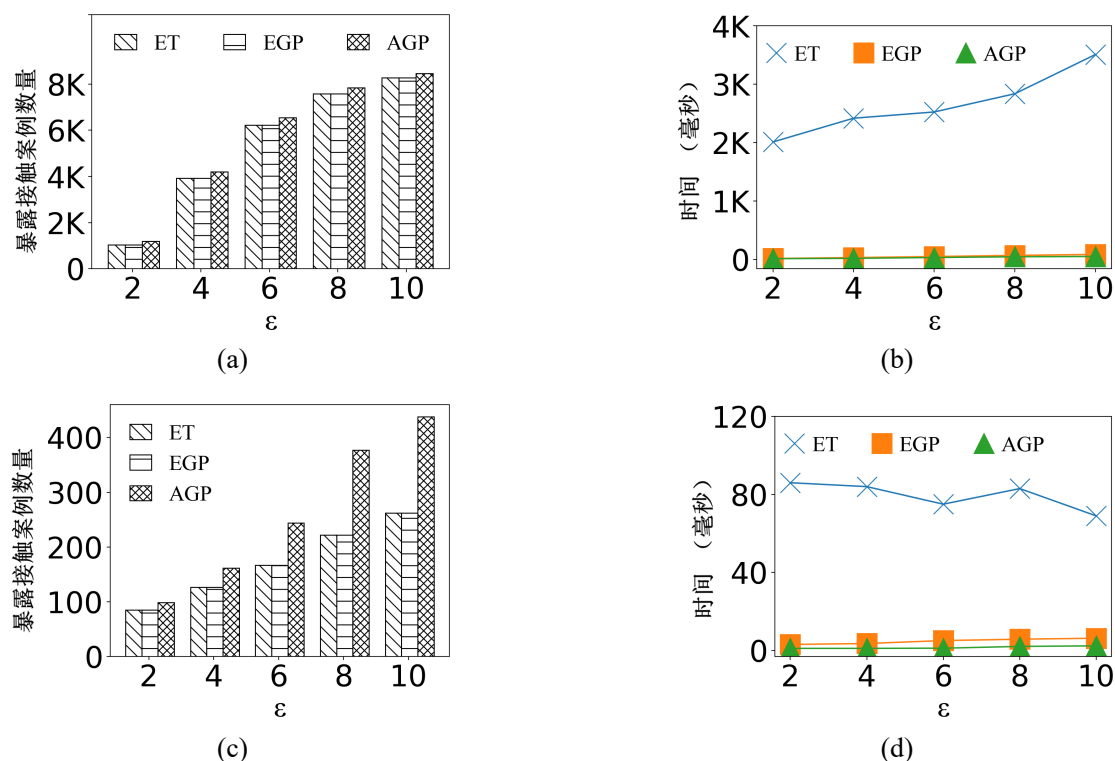


图 5-4 距离阈值的影响。(a) 北京数据集上暴露接触案例数量；(b) 北京数据集上算法运行时间；(c) 波尔图数据集上暴露接触案例数量；(d) 波尔图数据集上算法运行时间

从图 5-4(a) 可以看出，在北京数据集上，随着 ϵ 的增大，AGP 与精确算法 ET 之间的结果差异逐渐缩小。这表明在空间分布较为分散或轨迹变化相对平稳的场景下，距离阈值的放宽会降低近似策略带来的边界误差，从而使近似结果更加接近精确结果。相比之下，在波尔图数据集上，AGP 与 ET 之间的差异随着 ϵ 的增大而更加明显。这可能是由于波尔图数据集中对象分布更加密集、轨迹重叠程度更高，当 ϵ 增大时，潜在接触关系呈现更强的聚集效应，从而放大了近似策略在边界判定上的累计误差。在运行时间方面，如图 5-4(b) 和图 5-4(d) 所示，ET 的运行时间并未随着 ϵ 的变化呈现显著波动趋势。这是因为 ET 主要执行对象之间的两两距离计算，其计算复杂度主要取决于查询对象数量及总体对象规模，而与距离阈值本身关系较弱。相比之下，EGP 与 AGP 的运行时间始终维持在较低水平，且随 ϵ 的变化仅产生轻微波动。

5.6.2.3 接触持续时长阈值的影响

进一步研究了接触持续时长阈值 k 对算法性能的影响。参数 k 用于控制对象之间接触关系在时间维度上的持续性要求。较小的 k 意味着对象只需在较短时间内保持近距离接触即可被判定为暴露接触案例，从而显著增加暴露接触案例的数量。同时，这也会导致在每个时间快照上需要维护和更新更多候选对象的状态，进而增加计算负担。如图 5-5(a) 和图 5-5(c) 所示，随着 k 的增大，三种算法在北京和波尔图数据集上的暴露接触案例数量均呈现明显下降趋势。这一现象符合预期，因为更大的持续时长阈值提高了暴露接触判定的时间约束条件，使得只有在较长时间内持续接触的对象才会被识别为暴露接触案例，从而有效过滤了大量短时接触关系。此外，可以观察到 ET 与 EGP 在各个 k 取值下的检测结果一致，再次验证了两种精确算法在结果正确性上的一致性；AGP 的结果整体接近精确算法，且在 k 较大时差异进一步减小，说明时间约束的增强在一定程度上缓解了近似策略带来的误差累积。

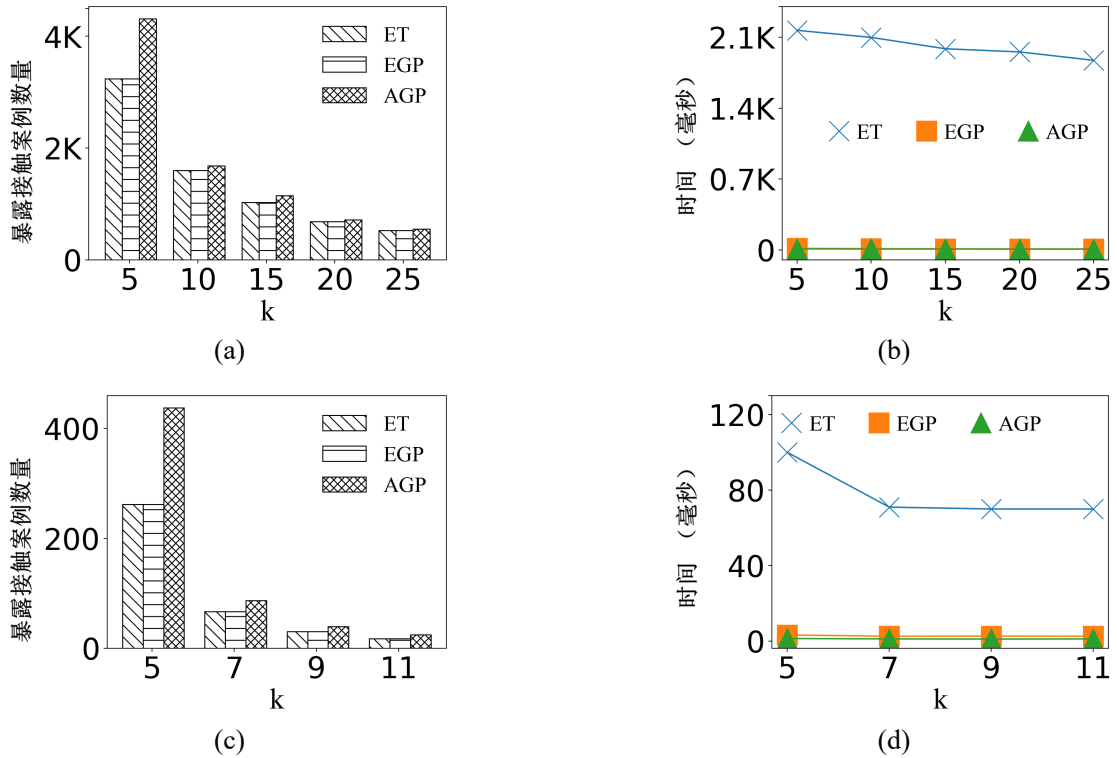


图 5-5 接触持续时长阈值的影响。(a) 北京数据集上暴露接触案例数量；
(b) 北京数据集上算法运行时间；(c) 波尔图数据集上暴露接触案例数量；
(d) 波尔图数据集上算法运行时间

在运行时间方面，如图 5-5(b) 和图 5-5(d) 所示，随着 k 的增大，三种算法的 CPU 时间整体呈现轻微下降趋势。其主要原因在于，当 k 较大时，能够满足持续时

长条件的候选对象数量显著减少，从而降低了需要维护的接触状态数量以及后续验证操作的规模。尽管 ET 的计算开销主要来源于对象间的距离计算，但由于有效暴露接触候选数量的减少，其总体运行时间仍略有下降。相比之下，EGP 与 AGP 在整个参数区间内始终保持较低且稳定的运行时间，进一步表明基于分组的处理机制在不同持续时长阈值设置下具有良好的适应性。

5.6.2.4 近似算法的评估

图 5-6 展示了在改变查询对象数量 $|Q|$ 以及接触距离阈值 ϵ 时，AGP 算法在两个数据集上的准确率表现。总体来看，AGP 在不同参数设置下均能够保持较高的检测精度，表明所提出的近似策略在降低计算开销的同时，并未显著影响结果质量。在北京数据集上，AGP 的准确率始终维持在较高水平，并且随着 $|Q|$ 或 ϵ 的增加未出现明显的性能退化。当查询对象数量增加时，尽管潜在接触关系数量上升，但由于对象空间分布相对分散，近似分组策略带来的误差并未被显著放大；当 ϵ 增大时，更多接触关系被纳入检测范围，但同时首尾优先与跳跃检查策略仍能够有效筛选持续接触对象，因此整体准确率保持稳定。

相比之下，在波尔图数据集上，随着 $|Q|$ 或 ϵ 的增加，AGP 的准确率呈现一定程度的下降趋势。这一现象可能源于波尔图数据集中对象分布更为密集、轨迹重叠程度更高，当查询对象数量或距离阈值增大时，空间邻近关系呈现更强的聚集效应，从而放大了近似判定在边界情况下的误差积累。尽管如此，AGP 在所有参数设置下仍保持较为理想的准确率水平。

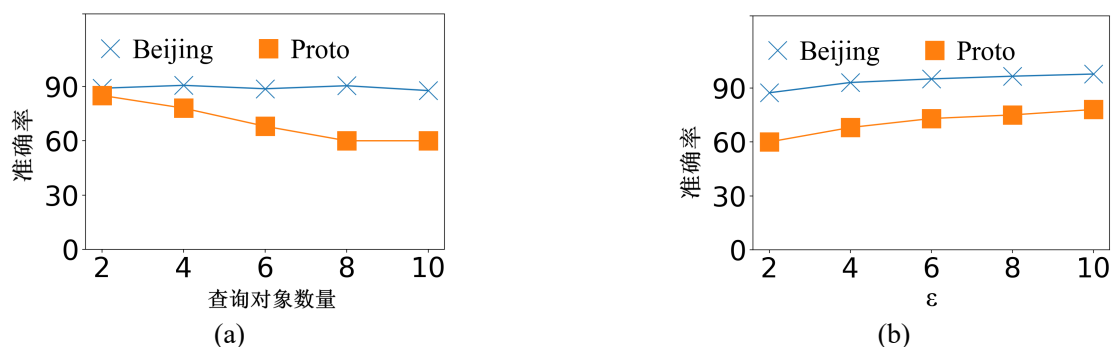


图 5-6 近似算法的评估。(a) 变化查询对象数目；(b) 变化接触阈值

5.6.2.5 预检查策略的评估

为评估预检查策略对算法性能的影响，将未采用预检查机制的精确分组处理方法记为 EGP*PC。图 5-7 展示了在改变查询对象数量 $|Q|$ 的情况下，EGP 与 EGP*PC 在北京和波尔图数据集上的 CPU 时间对比结果。可以观察到，在两个数据集上，

EGP 的运行时间始终低于 EGP*PC，且随着 $|Q|$ 的增加，两者之间的性能差距逐渐扩大。这表明，当查询对象数量增多时，候选网格单元及潜在接触对象数量随之增加，若缺乏有效的预筛选机制，将导致大量不必要的对象级比较操作，从而显著增加计算开销。预检查策略通过在组层面计算最小外包矩形之间的距离上下界，能够在早期阶段剪枝明显不可能发生接触的对象组，避免进入后续精确验证阶段，因此有效减少了距离计算次数。

此外，在北京数据集上，由于对象规模较大且空间分布较为复杂，预检查策略所带来的性能提升更加明显；在波尔图数据集上，尽管整体运行时间较低，但 EGP 仍持续优于 EGP*PC，说明该策略在不同数据规模与分布特性下均具有稳定的优化效果。综上所述，实验结果充分验证了预检查机制在降低计算复杂度和提升执行效率方面的有效性。

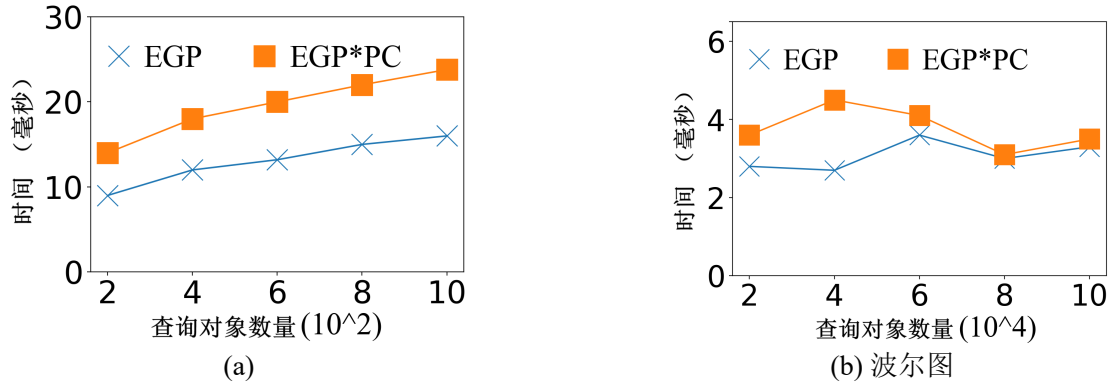


图 5-7 预检查策略的评估。(a) 北京数据集；(b) 波尔图数据集

5.7 本章小结

本章提出并系统研究了持续暴露接触搜索问题。该问题旨在从连续到达的空位置数据中，实时识别与查询对象在空间上保持近距离且在时间上满足持续性约束的潜在暴露接触对象。与传统静态或离线接触查询不同，该问题同时涉及大规模对象之间的空间邻近判定与时间连续性验证，具有数据规模大、计算频率高以及实时性要求高等特点。

现有方法均难以直接用于解决本文提出的问题。一方面，部分方法假设输入为静态轨迹集合，缺乏对持续更新数据流的支持；另一方面，不同工作对密切接触事件或感染判定规则的定义存在差异，难以满足本文所定义的连续接触搜索需求。因此，有必要设计一种面向动态环境、支持大规模对象并行处理且符合本文密切接触定义的新型解决方案。针对该问题，首先调研并分析了现有基于两两距离计算的精确检测方法，指出其在大规模场景下存在计算开销高、扩展性不足等局限

性。随后，系统分析了持续暴露接触搜索问题面临的核心挑战，包括：如何在保证检测精确性的前提下降低对象级距离计算次数；如何在时间维度上有效维护持续接触状态；以及如何在流式环境中实现高效、可扩展的实时处理机制。

为应对上述挑战，本文提出了一种精确分组处理算法，通过基于网格的空间划分机制，将对象组织为查询组与非查询组，并在组层面进行候选筛选，在对象层面执行精确验证，从而显著减少不必要的两两距离计算。同时，设计了基于最小外包矩形的预检查策略，通过计算组间距离上下界实现有效剪枝，进一步降低计算开销。在此基础上，为满足更高实时性需求，本文进一步提出了近似分组处理算法，引入首尾优先检查与跳跃检查策略，在保证较高准确率的同时显著降低时间维度上的处理频率，实现效率与精度之间的有效平衡。大量实验结果表明，与传统精确方法相比，所提出的分组处理框架在不同数据集和参数设置下均能够显著降低 CPU 时间开销，并保持与精确算法高度一致的检测结果。尤其是在大规模数据场景下，EGP 与 AGP 均展现出良好的可扩展性与稳定性。此外，预检查策略能够有效避免大量无效的组间比较操作，是提升整体执行效率的重要因素。综上所述，本章提出的方法在效率与有效性方面均表现优异，为大规模轨迹数据中的实时暴露接触事件检测提供了一种高效可行的解决方案。

第六章 运动范围感知的轨迹交集相似度连接

本章研究如何针对特定场景下的轨迹相似度连接这一城市计算应用设计高性能算法。如何在存在位置不确定性的轨迹数据上，设计一种移动范围感知的相似连接方法，以更加准确地刻画移动对象之间的潜在时空交互关系。在真实应用场景中，由于采样频率不一致、定位误差不可避免、以及相邻采样点之间运动过程不可观测等因素，轨迹数据往往具有不确定性与稀疏性特征。现有多数相似度连接方法主要基于离散采样点进行距离计算，难以有效反映对象在采样间隔内的连续运动状态，从而可能遗漏在现实中具有重要意义的潜在交互行为。

为克服上述局限，本章提出了一种相似连接模型——交集相似度连接。不同于传统基于点邻近关系的建模方式，以对象在给定时间段内的潜在运动范围为基本分析单元。首先将运动范围定义为对象在某一时间段内可能到达的空间区域，用以刻画其位置不确定性；在此基础上，引入交集相似度度量，用于量化两个对象运动范围之间的重叠程度。通过该建模方式，能够从区域特征交集的角度识别对象对，从而更真实地反映移动对象之间的潜在接触或协同行为。

在解决方案上，由于需要在大规模对象集合上进行区域重叠判定，其计算复杂度在最坏情况下接近二次方级别，难以直接应用于实时或准实时系统。为此，本章设计了一种带有重划分策略的混合球树索引结构，通过层次化组织运动范围并动态调整节点划分边界，实现高效的候选对象过滤。同时，引入预检查与多层剪枝机制，在索引遍历阶段提前排除不可能满足交集条件的对象对，从而显著降低精确计算阶段的开销。最后，基于两个真实轨迹数据集开展了系统实验评估。实验结果表明，与基线方法相比，提出的方法在保证结果准确性的同时，在运行效率上最高可实现约 3 倍的加速。上述结果验证了所提模型与索引结构在准确性、效率与可扩展性方面的优势，也为城市出行分析、交通监测、野生动物追踪以及公共卫生场景中的接触分析等应用提供了新的技术支撑。

6.1 引言

6.1.1 研究背景与意义

随着智能手机、车载导航系统以及可穿戴设备等 GPS 终端的普及，海量轨迹数据被持续采集与存储，围绕移动对象的连接、范围查询、 k NN 查询以及相似度查询等问题已成为时空数据管理领域的研究热点。相似移动对象的连接是时空数据管理领域中的一项基础性操作，在轨迹分析、群体行为发现、异常检测以及公

共安全等应用中具有重要意义。现有研究工作 [94, 102, 118, 144] 通常将移动对象建模为一系列带时间戳的离散位置采样点，并基于采样点之间的空间邻近关系来度量对象间的相似度。

尽管上述基于点的表示方式具有建模简洁、计算高效等优点，但其隐含前提是对对象轨迹具有足够高的采样频率。然而，在真实应用场景中，由于隐私保护要求、商业数据限制、通信带宽约束、设备能耗限制以及数据丢失等因素 [152]，移动对象往往只能以较低频率被观测。此时，相邻观测点之间的实际运动路径不可直接获取，导致对象在两个连续时间戳之间存在显著的空间不确定性。在稀疏采样条件下，传统基于采样点邻近性的相似度度量方法可能严重低估对象之间的真实交互程度，从而错误地遗漏本应满足相似度条件的对象对。

为解决上述问题，已有研究提出了多种增强表示模型。一类方法利用最小外接矩形结合速度约束构建基于运动模型的表示方式，用以近似刻画对象在观测间隔内的可能位置区域 [12–14, 153, 154]。另一类方法 [16] 采用分段聚合近似 [17] 技术，将轨迹划分为若干片段，并将每个片段映射为来自预定义字母表的符号，从而将轨迹转换为符号序列以支持高效匹配。尽管上述方法在一定程度上提升了轨迹建模的灵活性，并增强了相似度计算的可扩展性，但它们仍难以精确刻画采样点之间的连续运动过程。在稀疏采样轨迹场景下，这些方法通常采用区域近似或符号抽象，缺乏对对象潜在运动轨迹之间“运动交集”关系的显式建模能力。因此，当两个对象在未观测时间段内存在真实空间交叠或运动重叠时，现有方法可能由于建模能力不足而将其错误剪枝。由此可见，如何在存在观测不确定性的条件下有效刻画移动对象之间的潜在运动交集关系，并在海量移动对象数据环境下高效地发现所有满足交集条件的对象对，仍然是一个亟待解决的开放性问题。

为解决上述挑战，本文提出了面向移动对象的交集相似度连接问题。通过引入“运动范围”的概念，对对象在相邻观测样本之间可能访问的空间区域进行建模。基于此，两个对象之间的交集相似度不再依赖于离散采样点之间的距离，而是由其运动范围内特征区域的重叠程度来刻画。形式上，给定关系表 `MovingObject` 中的一组移动对象以及相似度阈值 θ ，定义交集相似度连接操作如下所示。每个对象 o 由一个 id 属性、两个位置坐标属性以及两个时间戳属性来表征，其中时间戳表示对象访问对应位置的时间。条件 $o.id < o'.id$ 用于避免重复比较。函数 $IS(o, o', I)$ 用于度量对象 o 与 o' 在时间段 I 内基于各自运动范围中特征重叠程度的交集相似度。参数 θ 为预先设定的相似度阈值。

```
SELECT o.id, o'.id
FROM MovingObject AS o, MovingObject AS o'
WHERE o.id < o'.id AND IS(o, o', I) ≥ θ
```

交集相似度连接在多个实际应用场景中具有重要价值，例如疾病防控、轨迹数据清洗、拼车推荐以及交通拥堵预测等 [2, 155]。这些场景的共同特点在于：对象之间的潜在交互往往发生在未被精确观测的时间段内，因此仅依赖离散采样点难以准确刻画真实关系。通过引入运动范围并基于区域交集进行相似度度量，能够更稳健地建模对象之间的潜在交互行为。下面讨论若干代表性应用场景。

6.1.1.0.1 场景一：城市交通监测与拥堵分析 在城市交通管理系统中，移动对象通常对应于车辆，其运动轨迹反映了实时交通状态。特征集合可以包括道路路段、交叉路口以及交通信号灯等交通基础设施要素。通过计算车辆之间的交集相似度，可以刻画其在一定时间段内经历相似交通环境或共享道路资源的程度，从而辅助拥堵预测、异常路段识别以及交通调度优化。对于兴趣点和道路交叉口等点状特征，可为每一个特征分配唯一标识符。对于线状道路网络，为保证建模粒度的一致性，可将道路划分为等长度子路段，并为每个子路段分配唯一标识符。基于此表示方式，车辆在某时间段内的运动范围可映射为一组可能访问的路段或节点集合。若两辆车辆的运动范围在特征集合层面存在显著交集，则说明它们可能受到相似交通流影响。相比传统基于同步采样点距离的相似度计算方法，能够在稀疏采样条件下更准确地反映潜在交通关联关系。

6.1.1.0.2 场景二：野生动物迁徙分析 在生态监测与保护领域，移动对象可对应于佩戴定位设备的野生动物。由于设备能耗限制与通信条件约束，动物轨迹通常具有较低采样频率。在此背景下，基于采样点的直接距离度量难以识别共享迁徙路径或潜在接触行为。通过引入运动范围，可以综合考虑栖息地、水源分布以及已知迁徙通道等环境特征，构建动物在观测间隔内的潜在活动区域。交集相似度用于衡量不同个体之间在生态特征空间中的重叠程度，从而识别共享迁徙走廊或高频活动区域。此外，该建模方式在节能数据采集与准确交互检测之间提供了一种有效折中方案 [156]：即在降低采样频率的同时，仍能通过区域建模保持对关键交互事件的识别能力。

6.1.1.0.3 场景三：接触追踪与疾病传播控制 在公共卫生领域，移动对象可表示为个体用户，其潜在接触关系是疾病传播建模的关键因素。在诸如新冠疫情等大规模传染病暴发期间，精准识别高风险接触事件对于抑制传播链条具有重要意义 [136]。由于实际系统中定位数据通常存在时间间隔和空间误差，仅依据同步采样点判断是否发生接触容易产生误判。通过运动范围建模，可以估计个体在给定时间段内的潜在活动区域，从而更合理地推断可能的接触事件。交集相似度度量反映了两

个个体在潜在活动区域层面的重叠程度，可用于评估感染风险等级并支持后续防控决策。

综上所述，不同应用场景虽然关注对象类型与特征集合各不相同，但其核心需求均可归结为：在存在观测不确定性的条件下，识别对象之间的潜在运动交集关系。交集相似度连接通过统一的运动范围建模与区域交集度量框架，为上述问题提供了通用且可扩展的解决方案。

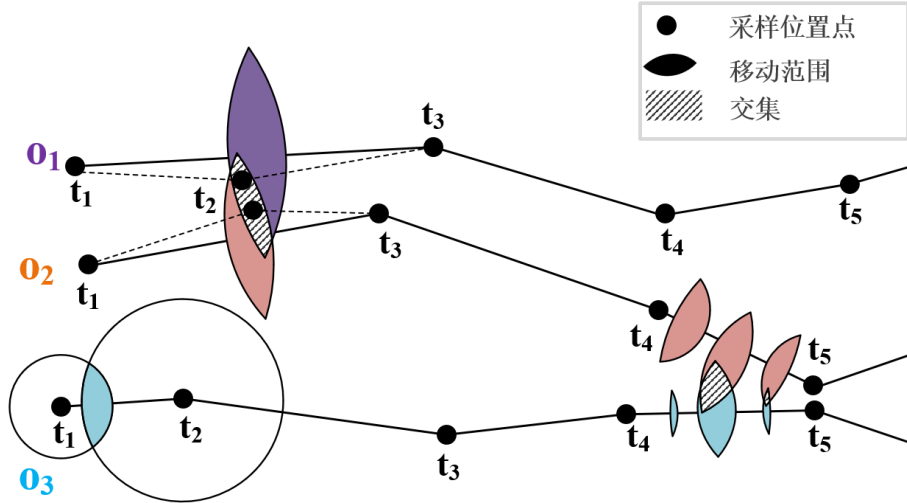


图 6-1 交集相似度连接示例

示例 6.1 如图 6-1 所示，考虑三个移动对象 o_1, o_2 和 o_3 ，它们在时间戳 t_1 至 t_5 之间均具有位置采样点。为便于说明，假设这些采样点在时间维度上是对齐的。在时间段 $[t_1, t_3]$ 内，对象 o_1 与 o_2 在 t_2 时刻的位置采样缺失或由于噪声过大而不可用。若采用传统基于采样点距离阈值的相似度连接方法，仅判断同步时间点上两对象之间的欧氏距离是否小于给定阈值，则可能得到 (o_1, o_2) 在 $[t_1, t_3]$ 内不满足相似度条件的结论。然而，在实际运动过程中，两对象在 t_2 时刻的真实距离可能低于该阈值，从而存在潜在的空间交集。由此可见，单纯依赖离散采样点进行判断，容易在稀疏采样或存在数据噪声的情况下产生误判。

6.1.2 研究内容与挑战

为克服上述局限性，本文引入“运动范围”以刻画对象在观测间隔内的潜在可达区域。以对象 o_3 在 t_1 和 t_2 时刻的位置为例，其在时间段 $[t_1, t_2]$ 内任意时间点 t 的运动范围可表示为由速度约束确定的透镜形区域（其严格定义将在第 6.2 节给出）。类似地，对象 o_1 和 o_2 在时间段 (t_1, t_3) 内的运动范围分别对应图中的紫色与棕色透镜区域。可以观察到，两者的运动范围在某些时间点存在重叠区域（图中阴影部分），这表明两对象在未观测时间段内可能存在空间交集。进一步地，考虑

时间段 $[t_4, t_5]$ 。图中展示了对象 o_2 和 o_3 在该区间若干离散时间点上的运动范围示例。为在理论上处理连续时间段，本文将区间 $[t_4, t_5]$ 内的交集相似度定义为该区间内采样时间点交集相似度的平均值。

给定时间段 $[T_s, T_e]$ 以及相似度阈值 θ ，交集相似度连接问题旨在识别并返回所有在该时间段内平均交集相似度不低于 θ 的对象对。与现有移动对象相似度连接研究主要基于离散采样点或区域近似不同，本文在连接语义层面显式引入“运动范围”概念，并将相似度度量从点级距离扩展至连续时间下的区域交集度量。这一转变显著提升了在稀疏采样条件下识别潜在交互关系的能力。然而，使交集相似度连接在实际系统中具备可行性面临两项关键挑战：

- (1) **连续时间计算挑战**。交集相似度在连续时间段上定义，理论上涉及无限多个时间点，直接计算精确结果在实际中不可行。因此，需要构建可计算的等价离散化模型或可证明误差界的近似计算方法。
- (2) **高效连接挑战**。现有相似度连接算法主要针对基于点距离或静态区域重叠的查询而设计，无法直接支持连续时间下的运动范围交集计算。若直接改造现有方法，往往导致候选对数量急剧膨胀，从而产生显著的计算开销。

6.1.3 研究成果与贡献

为应对上述挑战，本文提出了一种由时间段运动范围引导的高效求解框架。具体而言，设计了一种混合球树的层次化索引结构，用于组织对象在时间段内的运动范围，并结合重划分策略与多级剪枝技术，有效减少候选对象对数量。本文主要贡献总结如下：

- 提出了交集相似度连接。与传统基于离散位置采样点距离或静态区域重叠的相似度连接不同，交集相似度连接在语义层面显式引入“运动范围”概念，将相似度度量从点级邻近关系扩展至连续时间下的区域交集关系。该建模方式能够更准确地刻画稀疏采样轨迹中潜在的运动交互行为，从而弥补现有方法在未观测时间段内建模能力不足的缺陷。
- 设计了一种由时间段运动范围引导的高效求解框架。具体而言，设计了混合球树索引结构，用于层次化组织对象在时间段内的运动范围表示，并结合重划分策略与多级剪枝技术，实现对候选对象对的有效过滤。
- 开展了系统性的实验评估，从运行时间、可扩展性以及参数敏感性等多个维度对所提出方法进行了深入分析。实验结果表明，相较于精心设计的基线方法，本文方法在保持查询结果准确性的同时，所提出算法的平均运行时间可降低约 3 倍。

6.2 问题描述

本节首先给出移动对象运动范围的形式化定义，并在此基础上定义移动对象之间的交集相似度。随后，引入放宽交集相似度概念，并对交集相似度连接问题进行严格的形式化描述。本文中使用的符号汇总于表 6-1。

表 6-1 主要符号解释

符号	含义
o	一个移动对象
s	一个带时间戳的位置采样点
$[x, y], t$	坐标 / 时间戳
I	一个时间段
$MR(\cdot)$	对象的运动范围
$dist(\cdot)$	距离度量函数
$\mathcal{F}(\cdot)$	运动范围的特征集合
$IS(\cdot)$	交集相似度度量
θ	交集相似度阈值
ϵ	放宽划分参数
T^ϵ	采样时间戳集合
$\mathcal{A}, \mathcal{B}$	两个移动对象集合
$\mathcal{R}$	交集相似度连接的结果集合

定义 6.1（带时间戳的位置采样） 移动对象的一个带时间戳的位置采样表示为 $s = ([x, y], t)$ ，其中 $[x, y] \in \mathbb{R}^2$ 表示对象在二维空间中的坐标位置， $t \in \mathbb{R}$ 表示对应的时间戳。对于移动对象 o ，其轨迹可表示为按时间升序排列的采样序列 $\{s_1, s_2, \dots, s_n\}$ ，且满足 $t_1 < t_2 < \dots < t_n$ 。

定义 6.2（时间点运动范围） 给定移动对象 o 的两个连续位置采样点 $s_i = ([x_i, y_i], t_i)$ 和 $s_j = ([x_j, y_j], t_j)$ ，其中 $t_i < t_j$ ，以及对象的最大速度 $v_{\max}$ ，则对于任意时间点 $t \in [t_i, t_j]$ ，对象 o 在时间 t 的运动范围（motion range）记为 $MR(o, t)$ ，其定义如式6-1。其中 $dist(\cdot, \cdot)$ 表示欧氏距离度量。几何上， $MR(o, t)$ 为两个圆形区域的交集，呈透镜形状，如图 6-2 所示，其两个半径分别为 $r_i = v_{\max}(t - t_i)$ 和 $r_j = v_{\max}(t_j - t)$ 。

$$MR(o, t) = \{(x, y) | dist((x, y), (x_i, y_i)) \leq v_{\max} \cdot (t - t_i) \wedge dist((x, y), (x_j, y_j)) \leq v_{\max} \cdot (t_j - t)\} \quad (6-1)$$

上述时间点运动范围的建模方式与不确定移动对象研究中的经典模型保持一致 [28, 157, 158]。通过引入最大速度约束，所构建的运动范围在理论上能够覆盖

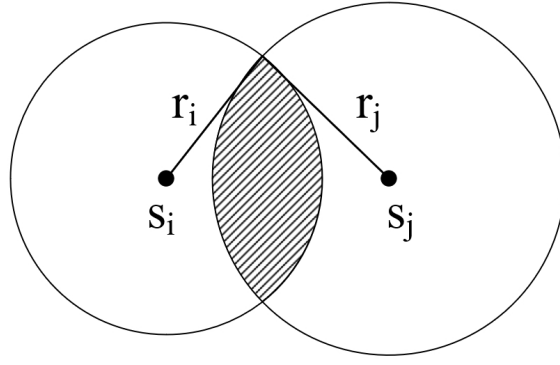


图 6-2 时间点运动范围示例

对象在采样间隔内的所有可能位置，从而保证结果的完备性。特别地，当 $t = t_i$ 或 $t = t_j$ 时，有 $MR(o, t_i) = \{(x_i, y_i)\}$ ， $MR(o, t_j) = \{(x_j, y_j)\}$ ，即运动范围退化为对应时间戳处的采样点。需要指出的是，若采样时间间隔异常长，则基于最大速度构建的运动范围可能过于宽松，从而导致候选区域显著扩大。本文假设采样间隔处于合理范围内，异常长间隔的处理不在本文讨论范围之内。

定义 6.3（运动范围特征） 设 $\mathcal{F}(o, t)$ 表示位于运动范围 $MR(o, t)$ 内的一组特定特征，即 $\mathcal{F}(o, t) = \{f \mid f \text{ 位于 } MR(o, t)\}$ 。该特征集合可根据具体应用进行定义，例如运动范围的面积度量、范围内的兴趣点集合、经过的道路路段集合，或与地理空间区域及环境因素相关的特征等。

定义 6.4（时间点交集相似度） 对象 o 与 o' 在时间 t 处的时间点交集相似度记为 $IS(o, o', t)$ ，用于度量其特征集合的重叠程度，定义如下式6-2。

$$IS(o, o', t) = \frac{|\mathcal{F}(o, t) \cap \mathcal{F}(o', t)|}{|\mathcal{F}(o, t)|} \cdot \frac{|\mathcal{F}(o, t) \cap \mathcal{F}(o', t)|}{|\mathcal{F}(o', t)|} \quad (6-2)$$

其中， $|\mathcal{F}(\cdot)|$ 表示特征集合的数目， $\cap$ 表示集合交运算。相似度取值范围为 $[0, 1]$ ：当两个特征集合完全重叠时取值为 1，当不存在任何重叠时取值为 0。时间点交集相似度的定义基于如下考虑。 $MR(o, t)$ 描述对象 o 在时间 t 可能占据的位置范围，而 $\mathcal{F}(o, t)$ 表示该范围内的相关特征集合。采用乘积形式意味着两个比例因子必须同时较大，相似度才会取得较高数值，从而保证仅当运动范围重叠程度高且特征相似度强这两个条件同时满足时，相似度得分才显著提升。相比 Jaccard 指数、算术平均或最小值策略，该定义能够更有效地平衡覆盖率与一致性。

定义 6.5（时间段交集相似度） 给定时间段 $I = [t_s, t_e]$ ，对象 o 与 o' 在区间 I 上的时间段交集相似度定义为时间点交集相似度的时间平均值，由式6-3计算。

$$IS(o, o', I) = \frac{\int_{t_s}^{t_e} IS(o, o', t) dt}{t_e - t_s} \quad (6-3)$$

计算时间段交集相似度并非易事。由于时间段的连续性，理论上需要在无限

多个时间点上评估 $IS(o, o', t)$ 。为在大规模移动对象数据场景下实现可扩展计算，并在不显著损失一般性的前提下获得高效近似结果，本文进一步提出了一种放宽的交集相似度定义。

定义 6.6（放宽的时间段交集相似度） 给定一个较小的放宽划分常数 ϵ ，对时间段 $I = [t_s, t_e]$ 进行 ϵ -采样，通过均匀采样得到一组时间点 $T^\epsilon = \{t_1^\epsilon, \dots, t_m^\epsilon\}$ ，其中 $t_1^\epsilon = t_s$ ， $t_m^\epsilon = t_e$ ，且满足 $t_{i+1}^\epsilon - t_i^\epsilon = \epsilon$ 。随后，将 ϵ -放宽的时间段交集相似度定义为对象 o 与 o' 在所有采样时间点 $t^\epsilon \in T^\epsilon$ 上的时间点交集相似度的平均值，记为 $IS(o, o', I, \epsilon)$ ，其计算方式如公式 6-4 所示。

$$IS(o, o', I, \epsilon) = \frac{\sum_{i=1}^m IS(o, o', t_i^\epsilon)}{m} \quad (6-4)$$

从本质上看，时间段相似度的计算需要对多个时间点上的时间点交集相似度进行评估。在下文中，如无特殊说明，为表述简洁起见，使用“交集相似度”来指代 ϵ -放宽的交集相似度。当 ϵ 趋近于 0 时， $IS(o, o', I, \epsilon)$ 将收敛到 $IS(o, o', I)$ 。

示例 6.2 图 6-3 给出了一个时间段交集相似度计算的示例。考虑时间段 $I = (t_4, t_5)$ ，令 $\epsilon = |I|/4$ ，则采样得到的时间集合为 $T^\epsilon = t_4, t_4 + \epsilon, t_4 + 2\epsilon, t_4 + 3\epsilon, t_5$ 。两个移动对象 o_2 和 o_3 在 T^ϵ 中各采样时间点处均具有对应的时间点运动范围。在时间点 t_4 和 $t_4 + \epsilon$ 处，两者的运动范围不存在交集，因此 $IS(o_2, o_3, t_4) = 0$ 且 $IS(o_2, o_3, t_4 + \epsilon) = 0$ 。在 t_5 时刻， o_2 与 o_3 的位置采样点相同，从而得到 $IS(o_2, o_3, t_5) = 1$ 。因此，区间 I 上的 ϵ -放宽时间段交集相似度可计算为 $(0 + 0 + IS(o_2, o_3, t_4 + 2\epsilon) + IS(o_2, o_3, t_4 + 3\epsilon) + 1)/5$ 。其中， $IS(o_2, o_3, t_4 + 2\epsilon)$ 和 $IS(o_2, o_3, t_4 + 3\epsilon)$ 的取值依据公式 6-2 计算，该公式能够适应不同应用场景下的具体需求。

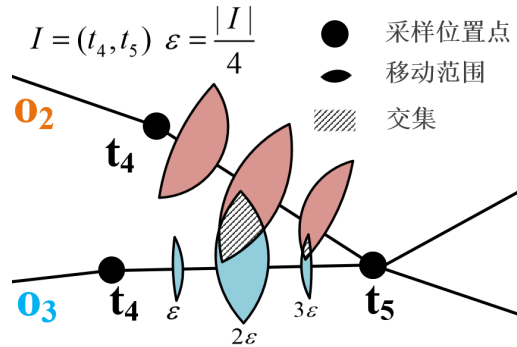


图 6-3 时间段交集相似度示例

定义 6.7（交集相似度连接问题） 给定对象集合 $\mathcal{A}$ 和 $\mathcal{B}$ 、带有放宽划分参数 ϵ 的查询时间段 I ，以及相似度阈值 θ ，交集相似度连接旨在从这两个集合中找出所有交集相似度不小于 θ 的对象对所构成的集合 $\mathcal{P}$ ，即对于任意 $(o_i, o_j) \in (\mathcal{A} \times \mathcal{B}) \setminus \mathcal{P}$ ，均有 $IS(o_i, o_j, I, \epsilon) < \theta$ 。

6.3 时间段预检查

为高效地回答交集相似度连接查询，提出了一种由时间段运动范围引导的框架。在时间段阶段，首先进行预检查，以判断两个对象的时间点运动范围之间是否存在交集。具体而言，定义时间段运动范围 $MR(o, I)$ ，用于刻画对象 o 在区间 I 内可能到达的所有位置。如果 $MR(o, I) \cap MR(o', I) = \emptyset$ ，则可推出 $IS(o, o', I, \epsilon) = 0$ ，从而对对象对 (o, o') 进行剪枝。本节介绍时间段运动范围的概念，并给出具有理论保证的预检查策略。

定义 6.8 (时间段运动范围) 给定移动对象 o 的两个带时间戳的位置样本 $s_i = ([x_i, y_i], t_i)$ 和 $s_j = ([x_j, y_j], t_j)$ ，以及在 t_i 与 t_j 之间的最大速度 v_{max} ，对象 o 在时间段 $I = [t_i, t_j]$ 内的运动范围，记为 $MR(o, I)$ ，被定义为对象 o 在区间 I 内可能到达的空间区域。

定理 6.1 给出了计算 $MR(o, I)$ 的方法。

定理 6.1 对象 o 的运动范围 $MR(o, I)$ 是由公式 6-5 表示的一个椭圆区域。椭圆的中心点记为 (x_c, y_c) ，由公式 6-6 计算得到；椭圆的旋转角度记为 α ，由公式 6-7 计算，其中 a 和 b 分别表示半长轴和半短轴。

$$\left[\frac{(x - x_c) \cos \alpha + (y - y_c) \sin \alpha}{a} \right]^2 + \left[\frac{(x_c - x) \sin \alpha + (y - y_c) \cos \alpha}{b} \right]^2 = 1 \quad (6-5)$$

$$\begin{aligned} x_c &= \frac{x_i + x_j}{2}, \quad y_c = \frac{y_i + y_j}{2}, \\ a &= \frac{v_{max} \cdot (t_j - t_i)}{2}, \\ b &= \frac{\sqrt{v_{max}^2 \cdot (t_j - t_i)^2 - (x_j - x_i)^2 - (y_j - y_i)^2}}{2} \end{aligned} \quad (6-6)$$

$$\alpha = \arctan\left(\frac{y_j - y_i}{x_j - x_i}\right) \quad (6-7)$$

证明：对象 o 在时间段 I 内能够行进的最大距离为 $(t_j - t_i) \cdot v_{max}$ ，该结论同时适用于欧氏空间和路网空间。给定两个位置样本 s_i 和 s_j ，对象 o 在某一具体时间点 t 的运动范围可由公式 6-1 定义。需要注意的是，椭圆上任意一点到其两个焦点的距离之和为常数 $2a$ 。当 $v_{max} \cdot (t_j - t_i) = 2a$ 时，从 t_i 到 t_j 的所有可能到达位置构成一个以 (x_i, y_i) 和 (x_j, y_j) 为焦点的椭圆区域，如图 6-4 所示。对时间段运动范围的建模方式与已有研究保持一致 [28, 158]。 ■

基于定理 6.2，提出了一种预检查策略：若 $MR(o, I) \cap MR(o', I) = \emptyset$ ，则可以在早期阶段安全地剪枝对象对 (o, o') ，从而避免对特征集合（例如 POI、运动范围

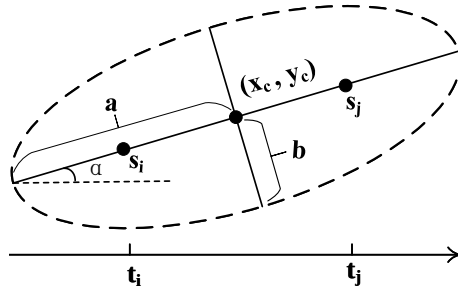


图 6-4 时间段运动范围示例

面积等) 进行不必要的交集计算。

定理 6.2 给定两个对象 o 和 o' , 若 $MR(o, I) \cap MR(o', I) = \emptyset$, 则 $IS(o, o', I, \epsilon) = 0$ 。

证明: 对于对象 o 和 o' , 如果它们在时间段 I 内的时间段运动范围不相交, 则在 I 内任意时间点上, o 与 o' 的时间点运动范围都不可能发生重叠。由于与各自时间点运动范围相关的特征集合 $P(o, t)$ 和 $P(o', t)$ 在整个区间 I 内均不存在交集, 因此在区间 I 上的交集相似度为 0。 ■

为了判断两个时间段运动范围是否相交, 一种直接的方法是计算它们的两两距离。具体而言, 若距离 $dist(MR(o, I), MR(o', I))$ 大于 $MR(o, I).a + MR(o', I).a$, 则两个运动范围不相交。其中, $dist(MR(o, I), MR(o', I))$ 表示 $MR(o, I)$ 与 $MR(o', I)$ 中心点之间的距离, a 表示运动范围的半长轴。这种直接方法在进行一对一比较时需要 $O(\mathcal{A} \times \mathcal{B})$ 的时间复杂度, 计算代价较高。

6.4 基于 M^* 树的连接算法

首先, 介绍一种基于 M^* 树的连接算法作为本文的基线方法。 M^* 树是经典 M 树的扩展 [22, 23]。 M 树是一种面向一般度量空间的动态平衡索引结构, 其设计建立在距离函数满足三角不等式的基础之上, 因此能够在不依赖具体空间坐标结构的情况下进行有效剪枝。 M^* 树继承了这一性质, 并针对时间段运动范围的近似表示进行了结构扩展。

基于 M^* 树的连接算法采用两阶段处理框架, 并结合预检查策略以增强候选对象的过滤能力。该框架遵循“先粗后精”的思想: 首先利用度量空间索引结构进行快速筛选, 随后对候选对象进行精确验证, 从而在保证正确性的前提下提升整体效率。在阶段 1 中, M^* 树使用近似圆对时间段 I 内的运动范围 $MR(o, I)$ 进行紧凑表示, 并基于圆心与覆盖半径构建层次化索引结构。具体而言, 每个时间段运动范围被映射为一个能够完全覆盖该范围的最小近似圆。由于圆之间的相交关系可通过中心距离与半径之和进行判定, 因此可以在树遍历过程中基于三角不等

式进行安全剪枝。该阶段的目标是快速识别所有“可能相交”的候选对象对，而不进行昂贵的相似度计算。在阶段 2 中，对阶段 1 产生的候选对象对执行精炼操作，即基于时间点运动范围计算放宽的时间段交集相似度 $IS(o, o', I, \epsilon)$ ，并根据阈值 θ 进行最终判定。由于精炼阶段仅作用于候选集合，因此可以显著降低总体计算成本。下面将详细介绍 M^* 树的结构设计及其在交集相似度连接中的具体应用。

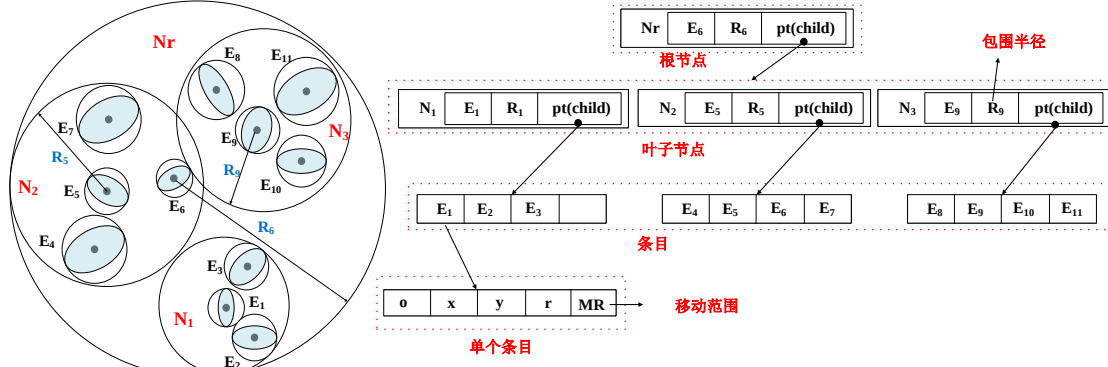


图 6-5 M^* 树的结构示例

6.4.1 M^* 树结构

图 6-5 展示了 M^* 树的结构，其指定容量为 $M = 4$ ，表示每个叶节点的最大条目数。近似圆被视为单独的条目，并独立存储在叶节点中。使用近似圆表示时间段的运动范围，并在 M^* 树中对其进行索引。每个圆存储为条目 $E = \{o, x, y, r, MR(o, I)\}$ ，其中 o 为对象标识， (x, y) 表示圆心， r 为半径， $MR(o, I)$ 包含时间段的运动范围。 M^* 树中的每个节点 N 选择叶节点中的一个条目 E 作为其路由对象，格式为 $N = \{E, R, pt(child)\}$ 。这里， E 是路由对象， R 是能够覆盖 N 中所有条目的半径， $pt(child)$ 是指向子节点的指针。需要注意的是， M^* 树在条目插入策略、节点分裂策略和路由对象选择策略上遵循了已有研究 [23, 28] 的做法，并将这些策略适配到 M^* 树结构中。值得注意的是，同一 M^* 树内的时间段运动范围在时间上是对齐的。为特定时间段索引所有运动范围，当时间段改变时，需要重建 M^* 树索引。

6.4.2 交集检查

从查询条目 $E_q = (q, x_q, y_q, r_q, MR(q, I))$ 和 M^* 树的根节点开始。目标是检索所有可能与 E_q 相交的条目 $E = \{o, x, y, r, MR(o, I)\}$ 。为了确定潜在交集，计算查询对象 q 与每个条目 E 之间的距离 $d(q, E) = \sqrt{(x_q - x)^2 + (y_q - y)^2}$ 。如果 $d(q, E) \leq r + r_q$ ，则时间段运动范围 $MR(q, I)$ 和 $MR(o, I)$ 可能相交。其中， r 和 r_q 分别表示条目 E 和 E_q 的半径。对于每个查询对象 q ，索引所有满足 $d(q, E) \leq r + r_q$ 的条目，并将对应的 (q, o) 加入候选集合，其中 o 是与条目 E 关联的对象标识。

6.4.3 算法细节

算法 6-1 给出了基于 M* 树的两阶段剪枝匹配算法的伪代码。算法的输入包括两个移动对象集合、M* 树的根节点、带放宽划分参数 ϵ 的时间段，以及相似度阈值 θ 。使用列表 $\mathcal{R}$ 存储所有满足条件的对象对，初始时 $\mathcal{R}$ 为空（第 1 行）。

算法 6-1: $MBJ(\mathcal{A}, \mathcal{B}, N_r, I, \epsilon, \theta)$

```

输入: 1) 两组移动对象,  $\mathcal{A}$  和  $\mathcal{B}$ ;
        2) M* 树的根节点  $N_r$ ;
        3) 时间间隔  $I$ ;
        4) 放宽划分参数  $\epsilon$ ;
        5) 交集相似度阈值  $\theta$ 。

输出:  $\mathcal{R} = \{(o, o') \mid IS(o, o', I, \epsilon) \geq \theta, (o, o') \in \mathcal{A} \times \mathcal{B}\}$ 。

1  初始化:  $\mathcal{R} \leftarrow \emptyset$ ;

2  for  $o \in \mathcal{A}$  do                                     /* 遍历集合  $\mathcal{A}$  */
3       $\mathcal{L} \leftarrow \{N_r\}$ ;
4      while  $\mathcal{L} \neq \emptyset$  do                           /* M* 树遍历 */
5           $N \leftarrow \mathcal{L}.pop()$ ;
6          if  $N$  是非叶子节点 then                         /* 非叶子节点 */
7              for  $N_c \in N.pt(child)$  do                 /* 遍历子节点 */
8                  if  $d(o, N_c) \leq N_c.R + o.r$  then
9                       $\mathcal{L}.push(N_c)$ ;
10             else                                       /* 叶子节点处理 */
11                 if  $d(o, N) > N.R + o.r$  then
12                      $prune(o, N.o)$ ;                 /* 预检查策略 */
13                 if  $IS(o, N.o, I, \epsilon) \geq \theta$  then
14                      $\mathcal{R} \leftarrow \{\mathcal{R}, (o, N.o)\}$ ; /* 精炼步骤 */

15 return  $\mathcal{R}$ ;

```

该算法由两个阶段组成: (1) 基于 M* 树的预检查 (第 4–12 行), 以及 (2) 精炼过程 (第 13–14 行)。在预检查阶段, 根据 M* 树获取候选对象集合。使用列表 $\mathcal{L}$ 存储可能与移动对象相交的节点, 初始时 $\mathcal{L}$ 仅包含根节点 (第 3 行)。当 $\mathcal{L}$ 非空时, 从中取出与对象 o 的距离 $d(o, N)$ 最小的 M* 树节点 N (第 5 行)。如果 N 不是叶节点, 则考虑其子节点。对于 N 的每个子节点 N_c , 检查 o 与 N_c 之间的距离。如果该距离不大于 o 与 N_c 半径之和, 则将 N_c 压入优先队列 (第 9 行)。如果 N 是叶节点, 且 $d(o, N) > N.R + o.r$, 则可以安全地剪枝 $(o, N.o)$ (第 11–12 行)。当交集相似度不小于阈值 θ 时, 通过将 $(o, N.o)$ 添加到结果集来更新 $\mathcal{R}$ (第 14 行)。最后, 返回 $\mathcal{R}$ 作为得到的交集对象对 (第 15 行)。

6.4.3.1 时间复杂度分析

一般来说, 构建 M^* 树需要 $O(|\mathcal{B}| \times n)$ 的时间, 其中 $\mathcal{B}$ 是被 M^* 树索引的对象集合, n 是节点分裂操作的次数。具体来说, 当由于对象插入导致节点已满时, 需要将其分裂为多个节点, 这一操作的时间复杂度为 $O(|\mathcal{B}|)$ 。值得注意的是, 实际构建时间可能会因数据分布等因素而有所不同。设 $\mathcal{A}$ 为查询对象集合。为了找到所有候选对象, 过滤过程的时间复杂度为 $O(|\mathcal{A}| \times \log |\mathcal{B}|)$ 。若时间点运动范围内的 POI 平均数量为 p , 则每次计算 $IS(o, o', I, \epsilon)$ 的时间复杂度为 $O(|I|/\epsilon \times p)$ 。设 $\mathcal{C}$ 为候选对象集合。对候选对象进行精炼的时间复杂度为 $O(|\mathcal{C}| \times |I|/\epsilon \times p)$ 。因此, 每轮执行的总时间复杂度为 $O(|\mathcal{B}| \times n + |\mathcal{A}| \times \log |\mathcal{B}| + |\mathcal{C}| \times |I|/\epsilon \times p)$ 。

6.5 基于混合球树的连接方法

为高效判定 $MR(o, I) \cap MR(o', I) = \emptyset$, 提出了一种混合球树结构。同时, 引入了一种基于交集相似度上界的剪枝增强方法, 以进一步剔除不满足条件的对象对, 从而细化整体处理流程。在精化阶段, 计算运动范围内特征的交集以度量交集相似度。首先介绍基于时间段运动范围的预检查策略, 随后给出基于混合球树的连接算法。

6.5.1 基于混合球树的交集预检查

6.5.1.1 混合球树

选择球树 [24, 25, 159] 作为基础索引结构, 这是因为相较于 R 树 [18] 和四叉树 [160] 等索引, 球树在处理类圆对象时表现更优 [22, 23, 28]。然而, 当将球树应用于交集相似度连接查询时, 仍然存在两个关键局限性。首先, 球树主要面向点或圆级别的最近邻搜索设计, 无法直接支持时间段运动范围的索引。其次, 其划分过程未考虑数据点的分布情况, 可能导致划分不均衡, 从而形成高度不平衡的树结构 [26]。

为了解决第一个问题, 将每个时间段运动范围近似为一个圆 [28]。具体而言, 该圆以时间段运动范围的中心为圆心, 其半径等于半长轴长度。该近似方式简化了时间段运动范围的表示, 使能够直接利用现有针对圆形对象设计的空间索引结构。其关键推论在于: 如果两个近似圆不相交, 则其对应的时间段运动范围也必然不相交。该推论构成了过滤策略的基础, 从而能够高效识别不可能发生交集的对象对。为了解决第二个问题, 提出了混合球树, 这是一种能够考虑数据分布特性的全新索引结构。

图 6-6 给出了混合球树的示例图。每个时间段运动范围被表示为 $E =$

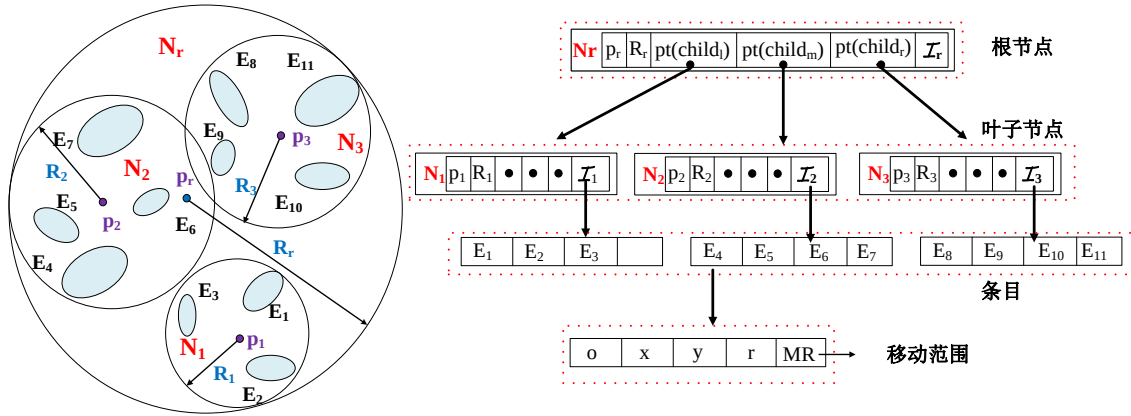


图 6-6 混合球树结构示例

$\{o, x, y, r, MR(o, I)\}$ ，其中 (x, y) 和 r 分别表示近似圆的中心和半径， o 表示对象标识符。与原始球树每个非叶节点仅包含两个子节点不同，混合球树通过一种重新划分技术，使得每个父节点最多可以拥有三个子节点。每个节点的结构表示为 $N = \{p, R, pt(child_l), pt(child_m), pt(child_r), \mathcal{I}\}$ ，其中 p 表示节点 N 所覆盖的时间段运动范围的中心点， R 表示能够包围节点 N 中所有条目（即圆）的半径， $pt(\cdot)$ 为指向子节点的指针， $\mathcal{I}$ 表示该节点中包含的时间段运动范围集合。混合球树中的插入策略、分裂策略以及枢轴点选择均遵循已有研究中的成熟方法 [159]。当某一子划分中的对象数量低于预定义阈值（即生成新子节点所需的最小对象数）时，子划分过程终止。

6.5.1.2 混合球树的重新划分策略

直观上，更加平衡的树结构通常能够在搜索过程中减少节点访问次数。重新划分方法的整体思想如下：对于每一个初始划分，评估其两个子划分中的点数差异是否超过重新划分阈值 β ($\beta \in (0, 1)$)。如果差异超过该阈值，则保留点数较少的子划分，并将点数较多的子划分进一步划分为两个新的子划分。考虑一个节点 N ，初始时 $N.child_m = \emptyset$ 。当较大的子划分规模至少以比例 β 超过较小子划分时，即触发重新划分。例如，当 $|N.child_l.\mathcal{I}| > |N.child_r.\mathcal{I}|$ ，且 $\frac{|N.child_l.\mathcal{I}| - |N.child_r.\mathcal{I}|}{|N.child_r.\mathcal{I}|} \geq \beta$ ，则触发重新划分。从理论上讲，较小的重新划分阈值会使初始子树更容易被重新划分，从而增加三叉节点的数量，使树结构更加平衡。然而，三叉节点数量的增加也可能导致节点访问次数上升，因此需要在树高与三叉节点数量之间进行权衡。

对于非叶节点，初始划分基于 $N.\mathcal{I}$ 中选取的两个最远点进行。具体而言，树构建过程中两个最远点的选取步骤如下：(i) 随机种子点：从 $N.\mathcal{I}$ 中随机选择一个点作为初始种子；(ii) 第一个最远点：在数据集中找到距离该种子点最远的点；(iii) 第二个最远点：在数据集中找到距离第一个最远点最远的点。设这两个最远点分

别为 $N.child_l.p$ 和 $N.child_r.p$ ，计算二者的中点，并在 $N.I$ 中选取距离该中点最近的中心点，记为 p_m 。点数较少的子划分被视为稳定划分，而对点数较多的子划分进行重新划分。具体地，若 $|N.child_l.I| > |N.child_r.I|$ ，则使用 p_m 和 $N.child_l.p$ 将 $N.child_l.I$ 中的运动范围重新划分为两个新的子划分，其中一个保留在 $N.child_l.I$ ，另一个分配给 $N.child_m.I$ ；反之，若 $|N.child_l.I| < |N.child_r.I|$ ，则使用 p_m 和 $N.child_r.p$ 对 $N.child_r.I$ 进行重新划分，一个子划分保留在 $N.child_r.I$ ，另一个分配给 $N.child_m.I$ 。最终，重新划分将原有的两个子划分扩展为三个子划分，新生成的子划分由 $N.pt(child_m)$ 索引。该重新划分技术带来了两方面优势：（1）降低树高与划分数目：通过提升平衡性，减少树的高度并最小化划分数目；（2）提升搜索效率：更加平衡的树结构能够减少树节点访问次数，从而提高搜索效率。

6.5.2 基于混合球树的连接

提出了一种基于混合球树的连接算法。为了进一步提升效率，引入了一种剪枝增强方法，利用交集相似度的上界来进一步剔除不满足条件的对象对，从而优化整体处理流程。

6.5.2.1 剪枝增强策略

由于运动范围交集形状的不规则性，很难直接计算两个特征集合 $\mathcal{F}(o, t)$ 与 $\mathcal{F}(o', t)$ 的交集。给定时间点 t ，若 $MR(o, t) \cap MR(o', t) \neq \emptyset$ ，当其中一个对象的运动范围完全被另一个对象覆盖时， $IS(o, o', t)$ 取得其最大值。基于此，公式 6-8 给出了 $IS(o, o', t)$ 的一个上界。将公式 6-8 代入公式 6-4，可得到公式 6-9，用于估计 $IS(o, o', I, \epsilon)$ 的上界。若 $IS(o, o', I, \epsilon)_{ub} < \theta$ ，则可以安全地剪枝对象对 (o, o') 。

$$IS(o, o', t)_{ub} = \frac{\min(|\mathcal{F}(o, t)|, |\mathcal{F}(o', t)|)}{\max(|\mathcal{F}(o, t)|, |\mathcal{F}(o', t)|)} \geq IS(o, o', t) \quad (6-8)$$

$$IS(o, o', I, \epsilon)_{ub} = \frac{\sum_{i=1}^m IS(o, o', t_i^{\epsilon})_{ub}}{m} \quad (6-9)$$

6.5.2.2 算法细节

算法 6-2 给出了基于混合球树的连接算法的伪代码。该算法的输入包括两个移动对象集合、混合球树的根节点、给定的时间间隔、放宽划分参数、重分区阈值以及交集相似度阈值；输出结果为交集对象对集合 $\mathcal{R}$ 。初始时， $\mathcal{R}$ 为空（第 1 行）。对于每一个 $o \in \mathcal{A}$ ，使用列表 $\mathcal{L}$ 存储可能与 $MR(o, I)$ 发生交集的候选节点，初始时 $\mathcal{L}$ 仅包含根节点 N_r （第 3 行）。混合球树通过插入对象集合 $\mathcal{B}$ 的时间段运动范

算法 6-2: 基于混合球树的连接

输入: 1) 两组移动对象, $\mathcal{A}$ 和 $\mathcal{B}$;
 2) 混合球树的根节点 N_r ;
 3) 时间间隔 I ;
 4) 放宽划分参数 ϵ ;
 5) 重分区阈值 β ;
 6) 交集相似度阈值 θ 。

输出: 交集相似度连接结果, $\mathcal{R}$ 。

```

1 初始化:  $\mathcal{R} \leftarrow \emptyset$ ;
2 for  $o \in \mathcal{A}$  do                                     /* 遍历所有移动对象 */
3    $\mathcal{L} \leftarrow \{N_r\}$ ;
4   while  $\mathcal{L} \neq \emptyset$  do                         /* BFS 遍历混合球树 */
5      $N \leftarrow \mathcal{L}.pop()$ ;
6     if  $N$  是非叶子节点 then                          /* 非叶子节点分支 */
7       for  $N_c \in N.pt(child)$  do                  /* 遍历子节点 */
8         if  $dist(o, N_c) \leq N_c.R + o.r$  then
9            $\mathcal{L}.push(N_c)$ ;
10    else                                              /* 叶子节点处理 */
11      for  $E \in N.\mathcal{I}$  do                             /* 遍历叶子节点对象 */
12        if  $dist(o, E) > E.r + o.r$  then
13           $prune(o, E.o)$ ;                          // 预检查策略
14        if  $IS(o, E.o, I, \epsilon)_{ub} < \theta$  then
15           $prune(o, E.o)$ ;                          // 剪枝增强策略
16        if  $IS(o, E.o, I, \epsilon) \geq \theta$  then
17           $\mathcal{R} \leftarrow \{\mathcal{R}, (o, E.o)\}$ ;      // 精炼步骤
18 return  $\mathcal{R}$ ;
```

围构建完成。当 $\mathcal{L}$ 非空时, 算法调用 $\mathcal{L}.pop()$ 取出一个节点 N (第 5 行)。随后, 在“圆级别”执行基于混合球树的过滤, 以识别潜在的交集候选 (第 6–9 行)。具体而言, 检索混合球树中满足 $dist(o, N_c) \leq R + r_q$ 的节点, 从而判断 $MR(o, I)$ 与 N_c 之间是否可能存在交集。若 N 为非叶节点, 则检查其子节点。对于每个子节点 N_c , 若对象 o 与 N_c 的距离不大于二者半径之和, 则将 N_c 压入优先队列 (第 8–9 行)。若 N 为叶节点, 则遍历 $N.\mathcal{I}$ 中的所有条目。对于每个条目 E , 计算 $(E.x, E.y)$ 与 $(MR(o, I).x_c, MR(o, I).y_c)$ 之间的欧氏距离 $dist(o, E)$ 。若 $dist(o, E) > E.r + o.r$, 则可安全剪枝 $(o, E.o)$ (第 12–13 行)。否则, 继续执行进一步的剪枝操作, 即剪枝增强策略 (第 14–15 行)。仅对保留下来的对象对, 才进行精确的交集相似度计算, 即利用公式 6-2 计算多个时间点的交集相似度 (第 16–17 行)。

6.5.2.3 时间复杂度

混合球树的构建时间复杂度为 $O(|\mathcal{B}| \times \log |\mathcal{B}|)$ 。预检查阶段的时间复杂度为 $O(|\mathcal{A}| \times \log |\mathcal{B}|)$ 。设 $\mathcal{C}'$ 为候选集合, 则候选精化阶段的时间复杂度为 $O(|\mathcal{C}'| \times |I|/\epsilon \times p)$ 。因此, 基于混合球树的连接算法的总体时间复杂度为 $O((|\mathcal{A}| + |\mathcal{B}|) \times \log |\mathcal{B}| + |\mathcal{C}'| \times |I|/\epsilon \times p)$ 。基于混合球树的连接算法得益于增强的剪枝策略, 显著减少了需要进行精化计算的候选对象对数量。

6.6 实验研究

在本节中, 开展了广泛的实验对本章算法性能进行评估。首先介绍实验设定, 包括实验所使用的数据集, 运行环境等, 然后给出实验结果与分析。实验结果主要关注所提出的搜索算法的运行时间, 即高效性方面的表现。实验的目的在于, 通过设置不同的参数 (移动对象数目、特征数量以及相似度阈值等), 对比不同的算法的运行效率并进行分析。

6.6.1 实验设置

6.6.1.1 数据准备

实验研究在两个真实世界的轨迹数据集上进行。第一个数据集 Geolife [161, 162] 包含来自 182 名用户、历时四年的 17,621 条轨迹, 涵盖了多种户外移动行为, 包括日常通勤以及购物、骑行等休闲活动。第二个数据集波尔图 [163] 包含在葡萄牙波尔图市 19 个月内记录的超过 170 万条出租车轨迹。波尔图数据集的轨迹以 15 秒为采样间隔记录, 而 Geolife 的采样频率则不固定。时间戳被限定在 24 小时范围内, 而不考虑具体日期, 因为大多数城市交通出行行为具有日常重复性。将对象的最大速度 v_{max} 定义为其在给定时间段内观测到的最高速度。确定 v_{max} 的真实取值或最优设置超出了本文的研究范围。为保证实际应用相关性, 剔除了采样间隔超过一小时的极端轨迹, 因为这类轨迹在现实应用中实用性有限, 因此不纳入本研究。注意到, 在隧道等环境中 GPS 信号可能较弱甚至完全丢失, 从而导致异常位置或较低的采样率。尽管对这类异常值进行显式建模超出了本文的研究范围, 数据预处理流程仍通过基于经度、纬度、轨迹长度和速度等约束过滤异常轨迹, 从而在一定程度上缓解了这些影响。预处理后, Geolife 数据集中保留了 18,670 条轨迹中的 12,015 条有效轨迹, 波尔图数据集中保留了 1,710,670 条轨迹中的 1,392,385 条。随后, 从数据集中随机选取两个包含相同数量移动对象的对象组。

6.6.1.2 对比算法

表6-2列举了实验研究中的比较算法。将所提出的方法与一种设计良好的基于

表 6-2 比较算法

算法标识	算法描述
MBJ-Alg	基于 M* 树的连接算法
HBJ-Alg	基于混合球树的连接算法
HBJ-Alg#	不包含剪枝增强策略的基于混合球树的连接算法
HBJ-Alg*	不采用重新划分策略的基于混合球树的连接算法

M 树的连接算法进行对比，记为 MBJ-Alg，作为基线方法。为评估剪枝增强策略的影响，还测试了不包含剪枝增强策略的 HBJ-Alg，记为 HBJ-Alg#。在效率评估方面，同时衡量剪枝率和运行时间。剪枝率定义为 $1 - |C|/|O|^2$ ，其中 C 表示候选集。运行时间表示算法每一轮的平均运行时间。此外，还评估了不采用重新划分策略的 HBJ-Alg，记为 HBJ-Alg*。

6.6.1.3 运行环境

参数设置如表 6-3 所示。M* 树和混合球树的叶节点容量均设置为 10。默认情况下，重新划分阈值 α 设为 0.5。在实验中，特征集合 $\mathcal{F}$ 由随机生成的兴趣点构成。具体而言，首先确定所有轨迹的最小和最大经度与纬度值，在该空间范围内使用均匀分布随机生成固定数量的兴趣点。默认情况下，在波尔图数据集的坐标范围内生成 100,000 个兴趣点，在 Geolife 数据集中生成 600,000 个兴趣点。默认距离度量采用欧氏距离。所有方法均使用 Java 实现，并在配备 AMD Ryzen 5 CPU (3.6GHz) 和 32GB 内存的 Ubuntu 系统上进行评测，采用单线程执行。除非另有说明，实验结果均为 20 次独立实验的平均值，每次实验使用不同的轨迹数据和特征集。

表 6-3 评估参数设置

	Geolife 数据集	波尔图数据集
$ O $	$\{4,6,8,10,12\} \times 10^3$ ；默认 8K	$\{1,2,3,4,5\} \times 10^4$ ；默认 10K
$ I $	$\{10s, 20s, 30s, 40s, 50s\}$ ；默认 30s	$\{15s, 30s, 45s, 60s, 75s\}$ ；默认 45s
$ \mathcal{F} $	$\{2, 4, 6, 8, 10\} \times 10^5$ ；默认 600K	$\{1, 2, 3, 4, 5\} \times 10^5$ ；默认 100K
ϵ	$\{\frac{ I }{2}, \frac{ I }{4}, \frac{ I }{6}, \frac{ I }{8}, \frac{ I }{10}\}$ ；默认 $\frac{ I }{6}$	$\{\frac{ I }{2}, \frac{ I }{4}, \frac{ I }{6}, \frac{ I }{8}, \frac{ I }{10}\}$ ；默认 $\frac{ I }{6}$

6.6.2 实验结果与分析

6.6.2.1 剪枝有效性评估

首先，在默认参数设置下考察各算法的剪枝有效性。实验结果如表 6-4 所示。通过比较 MBJ-Alg 与 HBJ-Alg 的候选集规模和剪枝率可以发现，在 Geolife 和波尔图数据集上，HBJ-Alg 的候选集规模分别仅为 MBJ-Alg 的 0.5% 和 2.3%。这些结果表明，所提出的方法能够显著压缩搜索空间。

表 6-4 剪枝性能

指标/数据集	方法	
	MBJ-Alg	HBJ-Alg
候选集规模 (Geolife)	6540764	28862
剪枝率 (Geolife)	89.7800%	99.9549%
候选集规模 (波尔图)	2142988	50072
剪枝率 (波尔图)	97.8570%	99.9499%

接下来，在不同参数设置下进一步评估所提出算法的性能。需要说明的是，在改变某一个参数时，其余参数均保持为默认值。

6.6.2.2 移动对象数目的影响

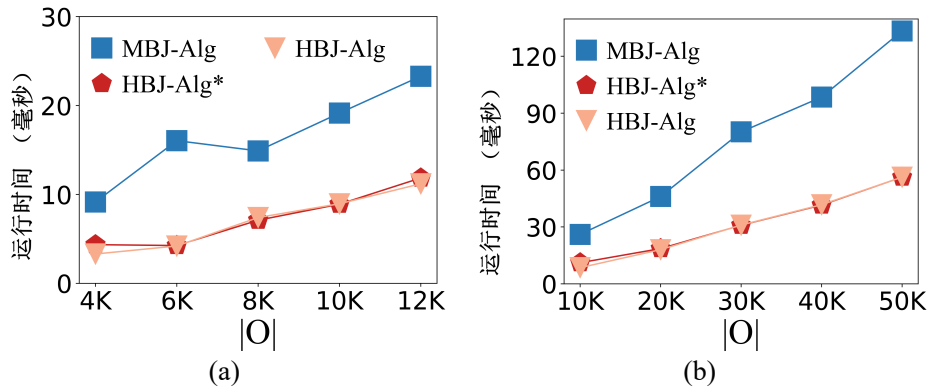


图 6-7 不同移动对象数目下的构建时间。(a): Geolife; (b): 波尔图

直观上，较大的移动对象数目 $|O|$ 会导致需要处理更多的时间段运动范围，因此所有算法的运行时间都将随之增加。首先，评估了 MBJ-Alg、HBJ-Alg 以及 HBJ-Alg* 的索引构建时间，不同移动对象数目 $|O|$ 对算法性能的影响如图 6-7 所示。可以清楚地看到，MBJ-Alg 的构建开销明显高于 HBJ-Alg。与 HBJ-Alg* 相比，HBJ-Alg 表现出相近且略有提升的性能。这一改进主要得益于 HBJ-Alg 中的再划分技术：尽管在触发再划分时会引入额外计算开销，但该技术能够减少总体分区数量，

从而在整体上有助于降低构建时间。

随着移动对象数量的增加，由于交集候选对数量增多，计算负载显著上升。从图 6-8 可以观察到，随着数据规模的扩大，MBJ-Alg 的运行时间显著增加。例如，在 Geolife 数据集包含 12,000 个移动对象、波尔图数据集包含 50,000 个移动对象时，MBJ-Alg 的查询时间分别超过 3 秒和 30 秒，计算开销较大。值得注意的是，HBJ-Alg 的运行时间几乎没有明显退化，充分体现了所提出方法的良好可扩展性。此外，与 HBJ-Alg# 相比，HBJ-Alg 的运行时间降低了约 30%，突出了剪枝增强策略所带来的性能提升。具体而言，在 Geolife 数据集包含 12,000 个移动对象、波尔图数据集包含 50,000 个移动对象时，HBJ-Alg 相比 MBJ-Alg 在 Geolife 上实现了 $3.1x$ 的加速，在波尔图上实现了 $3.3x$ 的加速。

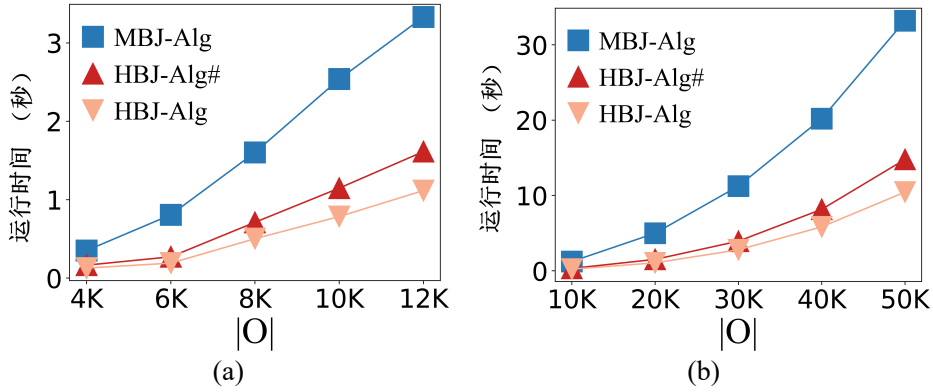


图 6-8 不同移动对象数目下的查询时间。(a):Geolife; (b): 波尔图

6.6.2.3 特征数量的影响

在实验中，将运动范围的特征集合设定为兴趣点 (POI)。直观而言，在相同的时间段运动范围内，较大的 $|F|$ 意味着两个对象之间存在更多潜在的交集位置，这可能由于需要进行额外的位置检查而导致运行时间增加。如图 6-9 所示，随着特征数量 $|F|$ (即兴趣点数目) 的增大，所有算法的运行时间均呈现出轻微的上升趋势。然而，所提出的预检查策略和剪枝增强方法能够在早期阶段有效地剔除大量不可能满足条件的对象对，从而显著减少需要进行 POI 交集计算的候选数量。因此，POI 集合规模对整体运行时间的影响较小。

6.6.2.4 时间段的影响

使用 $|I|$ 表示给定时间段的持续时长。具体而言，在 Geolife 数据集中， $|I|$ 从 10 秒变化到 50 秒；在波尔图数据集中， $|I|$ 从 15 秒变化到 75 秒。较大的 $|I|$ 会扩大潜在的时间段运动范围，从而导致具有交集的对象对数量增加以及候选集规模

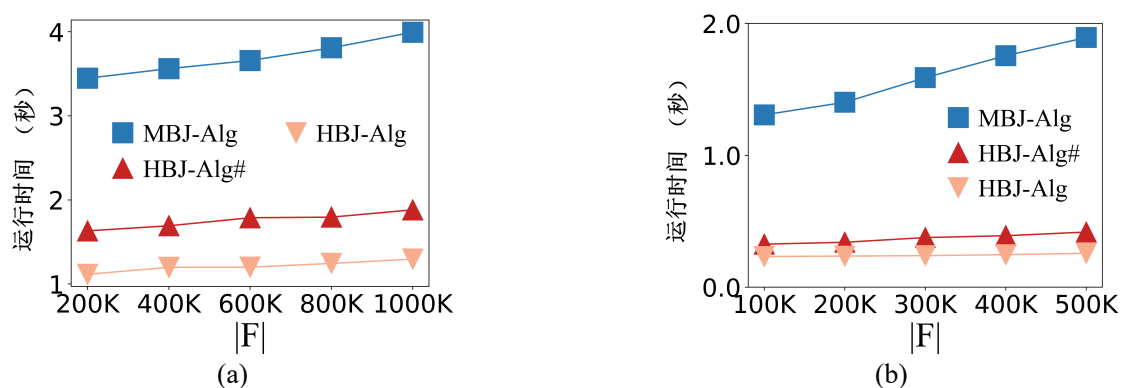


图 6-9 兴趣点数量对运行时间的影响。(a):Geolife; (b): 波尔图

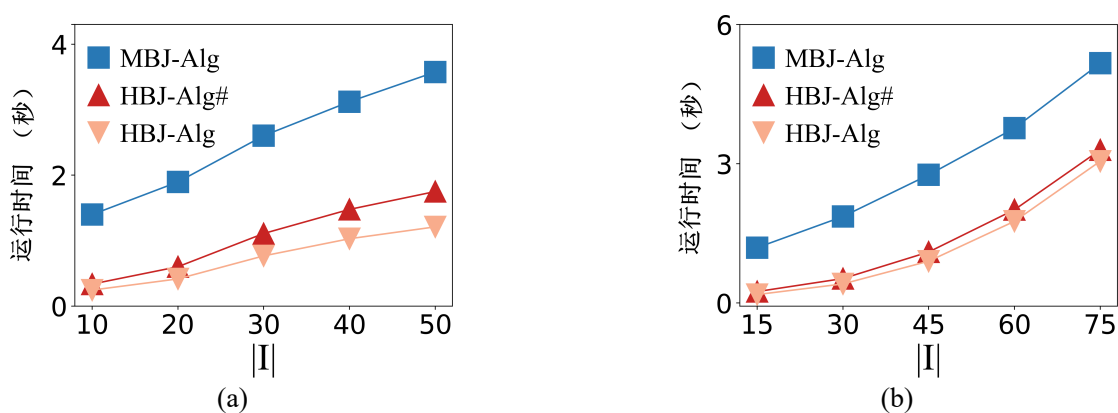


图 6-10 时间段对运行时间的影响。(a):Geolife; (b): 波尔图

增大。因此，所有算法在识别满足条件的对象对时都需要更多的计算开销。图 6-10 展示了随着 $|I|$ 增大时的效率表现。正如预期的那样，随着 $|I|$ 的增加，所有算法的运行时间均呈上升趋势，其中 HBJ-Alg 在所有参数设置下始终优于 MBJ-Alg 和 HBJ-Alg#。通过利用 M* 树，在一定程度上提升了过滤性能，并减少了后续需要进一步处理的候选对象数量。然而，由于 M 树的原始设计在构建阶段需要进行耗时的节点分裂操作，其并不适合直接用于交集相似度连接查询。此外，在大规模移动对象场景下，对仅覆盖单个时间段运动范围的圆进行索引，可能会导致搜索空间膨胀，从而对查询性能产生不利影响。

6.6.2.5 放宽参数的影响

较小的放宽参数 ϵ 值意味着需要探测更多的时间点运动范围，使得放松后的相似度更加接近真实的交集相似度，从而带来更高的计算开销，并导致运行时间的增加。图 6-11 展示了在变化放松参数 ϵ 时的效率表现。在实验中，将时间点运动范围的数量从 2 变化到 10。随着 ϵ 的变化，可以观察到运行时间呈现出轻微的上升趋势。同时也可以看到，在所有参数设置下，HBJ-Alg 始终表现出最优的性能。

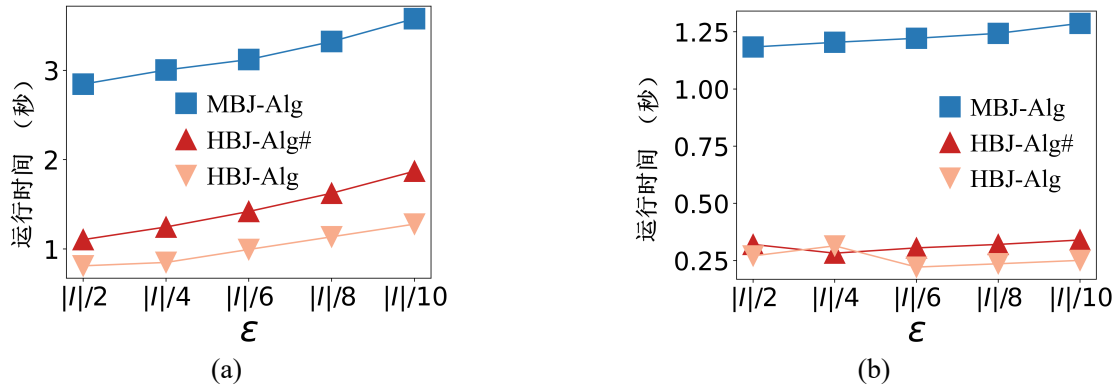


图 6-11 放宽参数对运行时间的影响。(a):Geolife; (b): 波尔图

6.7 本章小结

本文提出了一种面向移动对象的运动范围感知的交集相似度连接。该框架通过引入对象在时间段内的潜在运动范围，而非仅依赖离散的轨迹点，有效刻画了对象之间连续且不确定的时空交互关系，从而弥补了传统基于点连接方法在表达能力上的不足。不同于现有轨迹相似连接方法侧重于点序列之间的距离度量，交集相似度连接以运动范围之间的交集相似度为核心建模对象，使查询语义更加直观且具有更强的鲁棒性，能够自然地支持多种时空分析需求，适用于交通监控、野生动物追踪、公共安全分析以及接触追踪等典型应用场景。

交集相似度连接问题的核心在于：在存在位置不确定性的时空环境中，以区域重叠替代点距离作为相似度判定标准，在保证查询语义合理性的同时，实现高效的大规模对象对筛选。例如，在城市交通监测场景中，车辆在采样间隔内可能存在路径偏移，仅依赖离散采样点可能无法捕捉真实的近距离接触；在野生动物追踪或接触追踪场景中，对象之间的潜在活动区域重叠往往比瞬时位置更具分析价值。因此，从运动范围交集的角度进行相似连接具有重要的理论意义与应用价值。围绕交集相似度连接查询的高效处理问题，本文系统分析了其计算挑战，包括候选对象规模庞大、时间段对齐要求严格以及交集判定代价较高等难点。为此，设计了一种融合重划分策略的混合球树索引结构，通过对运动范围进行层次化组织与动态重划分，有效缓解数据分布不均带来的性能瓶颈。同时，提出了预检查与多层剪枝机制，在索引遍历阶段提前过滤大量不可能相交的对象对，从而显著降低精确计算阶段的开销，提高整体查询效率与可扩展性。

最后，基于两个真实世界数据集开展了系统而全面的实验评估。实验结果表明，在不同数据规模与参数设置下，所提出的方法均表现出稳定且优越的性能表现；与精心设计的基线方法相比，在典型场景下最高可获得约 3 倍的加速效果，验证了所提框架与索引结构在提升算法性能上的优越性。

第七章 总结与展望

本章对全文进行总结，并分别列出每个研究内容的贡献创新，并讨论了未来可行的研究工作展望。

7.1 全文总结

随着 GPS 设备与智能移动终端的广泛普及，带有地理位置信息的时空数据呈现出爆炸式增长，这为城市计算与时空数据挖掘提供了丰富的数据基础，也带来了新的研究挑战。特别是在路径规划、接触搜索、联合移动模式挖掘以及轨迹相似性分析等任务中，如何在大规模、动态且存在观测不确定性的环境下进行高效计算，成为学术界和工业界亟需解决的问题。传统方法往往依赖离散轨迹点或静态路径集合，难以满足实时性、可扩展性及准确性要求，尤其在连续到达的行程请求流或大规模移动对象场景下，其性能和适用性存在明显局限。为应对上述挑战，本文系统研究了面向城市计算的高效算法设计，提出了一系列针对不同场景与任务的算法框架。具体而言，本文涵盖四类核心研究工作：(1) 持续最优路径组合规划：针对动态路网中持续到达的行程请求流，提出了全局协同优化的路径规划方法，通过将时间窗口内的请求作为整体进行联合优化，实现整体通行效率的提升，有效缓解交通拥堵。(2) 联合移动模式挖掘：面向大规模移动对象轨迹数据，设计了高效的群体模式检测算法，用于高效发现一种宽松的联合移动模式，支持交通管理、拼车推荐及异常行为识别等应用。(3) 接触搜索与疾病传播分析：针对大规模连续轨迹数据，提出可扩展且低延迟的接触搜索方法，通过运动范围建模有效识别潜在接触事件，为公共卫生监测、疫情防控及风险评估提供数据支撑。(4) 轨迹相似连接：提出基于运动范围的交集相似性连接框架，突破传统离散点方法的局限，在稀疏采样或不确定条件下仍能稳健刻画对象间潜在交互关系，适用于交通监控、野生动物迁徙分析及接触追踪等场景。

7.1.1 面向大规模行程请求流的持续最优路线组合规划

本文在第三章研究了城市计算应用中面向大规模行程请求流的全局协同路径规划问题。该问题的复杂性在于：行程请求以数据流形式持续到达，早期规划结果会影响未来交通状态，不同行程路径通过共享路段形成复杂耦合关系，从而导致组合搜索空间呈指数级增长。传统单次最优路径规划方法无法应对这种持续性、动态性和全局耦合性的挑战，容易在局部最优决策叠加下形成新的拥堵热点。

为此，设计了一种高效的自感知批处理解决方案，用于持续优化行程请求流中的路线组合。解决方案包含两个核心步骤：初始路径搜索与批量优化处理。初始路径搜索步骤为每批新到达的行程请求生成高质量的初始路径组合，以避免潜在交通拥堵；批量优化处理步骤则在初始组合基础上进一步优化所有路径，以最小化总通行时间并提升整体路网效率。研究成果和贡献总结如下：

- 提出了一种新颖的持续最优路线组合问题模型，适用于城市交通流管理、实时路径规划及拥堵预防等多种应用场景。
- 设计了高效的自感知批处理算法及剪枝机制，实现对大规模行程请求流的实时优化，兼顾准确性与计算效率。
- 验证了所提方法在真实路网数据上的表现，相较于基线方法展现出更高的性能和可扩展性，在总通行时间优化和实时处理能力上具有显著优势。

7.1.2 基于轨迹数据的群体联合移动模式挖掘

本文在第四章研究了城市计算应用中面向大规模移动对象轨迹的联合移动模式挖掘问题。该问题的复杂性在于：轨迹数据规模庞大，移动对象随时间持续变化，联合移动模式需要在空间邻近性、时间持续性及群体规模约束下进行精确判定；同时，传统模式定义在时间上过于严格，导致候选生成与验证成本极高，尤其在实时或近实时应用场景中难以满足计算效率要求。此外，群体成员可能暂时离开群体并重新加入，历史数据与新到达轨迹间存在强动态依赖，使得模式挖掘具有高度动态性与组合耦合性。

为此，设计了一种针对伴随群体的高效挖掘框架，包含两类核心算法：精确搜索算法（ES-ECMC）和社区感知的近似搜索算法（CS-ACMC）。ES-ECMC 算法基于严格的时空约束判定机制，对候选对象对及群体进行精确验证，保证结果完备性与准确性，适用于离线或中小规模数据场景；CS-ACMC 算法通过近似计算与结构化聚类方法压缩搜索空间，实现高效增量挖掘，满足实时或近实时应用需求。整体解决方案的步骤为：首先在近似联合移动对搜索阶段，通过网格划分将轨迹离散化，并快速估计对象间联合移动程度生成候选对；随后在社区感知群体发现阶段，将候选对构建为图结构并利用社区检测算法进行聚类，从而高效识别伴随群体。研究成果和贡献总结如下：

- 提出了伴随群体模式定义，兼顾空间邻近性、时间持续性与群体规模约束，并适应动态轨迹数据环境，具备广泛应用价值，包括交通监测、疫情防控及公共安全预警等场景。
- 设计了精确与近似两类算法（ES-ECMC 与 CS-ACMC），有效解决了准确性与效率之间的权衡问题。

- 验证了所提方法在用户可交互时间内高效发现伴随群体的能力，同时展示了在大规模数据环境下的可扩展性与实用性；

总体而言，本章在理论与实践层面系统研究了伴随群体发现问题，提出了高效、可扩展且具有应用价值的解决方案，为大规模动态轨迹数据环境下的群体行为挖掘提供了新的研究方法与技术工具。

7.1.3 大规模移动对象上的持续密切接触发现

本文在第五章研究了城市计算应用中面向大规模动态移动对象的实时密切接触检测问题。该问题的复杂性在于：轨迹数据以持续流的形式到达，移动对象数量巨大且不断变化，同时需要在空间邻近性与时间持续性约束下实时识别潜在密切接触对象。此外，密切接触具有递归扩展特性，即新发现的接触对象将继续参与后续检测，导致传播链持续增长，增加了算法的计算复杂度与状态维护难度。在突发公共卫生事件或大型人群聚集场景下，若无法高效处理轨迹流，将难以及时识别关键接触节点，从而影响风险控制决策。

为此，设计了一种针对持续密切接触搜索问题的高效解决方案。解决方案包含两类核心算法：精确实时搜索算法和近似实时搜索算法。精确实时搜索算法通过严格的空间与时间约束判定，对候选对象进行精确验证，保证检测结果的完备性与准确性；近似实时搜索算法在保证不漏检的前提下，通过首尾优先检查、跳跃检测及近似机制显著降低计算量，实现高效实时处理。整体解决方案的步骤为：首先对轨迹对象进行分组过滤，减少候选对象数量；随后基于距离上下界的预检查策略避免不必要的精确距离计算；最后在滑动时间窗口内维护对象间接触状态，实现持续更新与递归扩展。研究成果和贡献总结如下：

- 定义了持续密切接触搜索问题，为流行病传播监测、公共卫生预警及风险追踪提供统一的数据处理框架，明确了递归扩展与实时更新特性。
- 设计了精确与近似两类实时处理算法，结合分组过滤、预检查及跳跃检测等优化策略，有效解决了准确性与效率之间的权衡难题。
- 验证了所提出方法具有良好的可扩展性与稳定性：在保证检测精度的同时，实现了大规模动态场景下的高效实时密切接触检测。

总体而言，本章系统研究了持续密切接触搜索问题，提出了高效、可扩展且具有应用价值的解决方案，为大规模流式轨迹数据环境下的实时疫情监控与风险管理提供了新的技术方法与实践参考。

7.1.4 面向时空运动范围的轨迹相似度连接

本文在第六章揭示了城市计算应用中移动对象相似性连接问题中的一个关键挑战——在稀疏采样条件下如何准确刻画对象间潜在交互关系。传统基于离散采样点的相似性度量方法在对象轨迹采样频率不足或存在观测缺失时容易低估真实相似性，导致潜在对象对被误剪枝。为解决这一问题，提出了交集相似性连接问题，通过引入“运动范围”概念，将对象在相邻观测样本之间可能到达的空间区域建模为运动范围，并基于这些区域的交集程度进行相似性度量，从而在稀疏采样或数据噪声条件下更稳健地刻画潜在交互。

交集相似性连接问题在多个实际场景中具有重要应用价值。例如，在城市交通监测中，车辆轨迹的运动范围可映射为道路子路段或交叉口集合，通过交集相似性识别潜在交通关联，有助于拥堵预测与异常路段识别；在野生动物迁徙分析中，运动范围结合栖息地与迁徙通道信息，可识别共享走廊或高频活动区域，实现低采样频率下的关键交互检测；在公共卫生领域，个体潜在接触事件可通过运动范围交集推断，从而支持疾病传播建模与风险评估。不同应用虽对象类型与特征不同，但核心需求均为在观测不确定条件下识别潜在运动交集关系，该研究提供了统一且可扩展的解决框架。

在建模与计算上具有以下关键创新：首先，明确将相似性度量从离散点级距离扩展到连续时间下的区域交集，允许在理论上处理未观测时间段的潜在交互；其次，将连续时间区间内的交集相似性定义为时间平均函数，既保留连续性信息，又便于离散化近似计算。该建模显著提升了稀疏采样条件下潜在对象对识别能力，同时引入连续时间处理与大规模候选对计算的两大挑战：一是连续时间计算的复杂性，需设计可计算的离散化或可控近似方法；二是高效连接的挑战，现有点或静态区域相似性算法无法直接支持连续时间运动范围的交集计算，否则候选对象对数量会急剧膨胀。

为应对上述挑战，提出了基于时间区间运动范围的高效求解框架。核心思想是通过混合球树索引对对象运动范围进行层次化组织，并结合重划分策略与多级剪枝技术，有效过滤候选对象对，从而在保证正确性与完整性的前提下显著降低计算开销。该框架适用于稀疏采样轨迹数据，能够高效识别满足相似性阈值的对象对，实现连续时间区间下的交集相似性计算。研究成果和贡献总结如下：

- 提出了交集相似性连接问题，将相似性度量从离散点扩展到连续时间下的运动范围交集，显著增强稀疏采样轨迹潜在交互建模能力。
- 构建了混合球树层次索引结构，并结合重划分与多级剪枝策略，实现大规模连续时间区间下的高效连接计算。

- 验证了所提出方法的可扩展性与实用价值：结果表明方法在保证准确性的前提下，相较于现有基线方法平均运行时间降低约 3 倍。

总之，本文针对稀疏采样轨迹数据中潜在运动交集关系的识别提出了统一建模与高效计算框架，为交通分析、生态监测及公共卫生等多种实际场景提供了可行且高效的技术方案。

7.2 展望

本文针对城市计算应用中的 4 个需求，即面向大规模行程请求流的持续最优路线组合规划需求，基于轨迹数据的群体联合移动模式挖掘需求，面向大规模移动对象的持续密切接触查询需求，以及面向时空运动范围的轨迹相似度连接需求展开了深入研究，发表了一系列研究成果，但仍存在一些值得进一步研究和探索的空间。随着移动对象数据规模和实时更新频率的持续增长，单机处理在计算能力与内存资源上面临显著瓶颈。未来可能的进一步研究方向可以向分布式环境、深度学习优化、时空大模型拓展，潜在研究方向包括：

7.2.0.0.1 分布式查询 针对大规模移动对象或位置标注数据，当数据量超出单机处理能力时，可在分布式框架中研究并行化的索引机制和查询算法。研究重点可包括：面向海量时空-文本数据的并行查询架构，优化任务调度与负载均衡；以及分布式数据迁移与副本管理策略，以保证连续查询过程中的数据一致性与高可用性。在分布式环境下，将交集相似性连接或连续密切接触搜索等计算任务进行分片处理，同时保持跨节点候选对象的完整性与准确性，是提高系统可扩展性与实时性的重要方向。相关研究可包括分布式滑动窗口维护、跨节点运动范围交集计算以及并行剪枝策略设计。分布式处理环境中，节点故障、网络延迟及数据倾斜可能导致查询性能下降。未来可探索自适应任务调度、容错恢复机制以及数据划分优化方法，以保障大规模连续时空查询的稳定性与效率。

7.2.0.0.2 基于深度学习的方法优化 随着移动对象数据的持续增长与复杂化，传统基于规则或索引的算法在处理大规模、动态、多源数据时面临性能和可扩展性瓶颈。深度学习技术因其强大的表示学习和模式捕捉能力，为未来时空数据分析与连续查询提供了新的研究契机。具体而言，潜在的研究方向可包括以下几个方面：（1）在大规模行程请求流场景下，如何为海量移动对象实时生成最优路线组合是关键问题。深度强化学习可用于建模连续决策过程，将路线选择、时间窗口约束和交通动态因素作为状态与奖励函数，实现高效、可扩展的路径优化。通过结合历史轨迹数据与实时交通信息，模型可学习群体行程模式，从而在动态环境

中提供持续优化的路线方案。(2) 群体行为分析和联合移动模式识别在城市规划、公共交通调度及疫情传播监控中具有重要意义。深度学习方法,如图神经网络或时空卷积网络,可以将移动对象及其轨迹视为时空图结构或序列数据,从而自动捕捉潜在的群体协同行为、同步移动模式及关键影响节点,实现对复杂群体行为的高效挖掘。(3) 持续密切接触检测涉及大规模对象在动态环境下的空间邻近与时间连续性判定。深度学习模型可通过学习对象轨迹的潜在运动分布和邻近概率,实现对未观测时间段潜在接触的预测,从而在海量数据流环境下支持实时、鲁棒的密切接触检测。结合序列建模与空间嵌入方法,可以有效提升接触预测的精度与响应速度。(4) 在稀疏采样轨迹或不完整数据条件下,基于传统规则的运动范围交集计算容易受限。深度表示学习可通过将轨迹映射到低维时空潜在空间,捕捉连续时间区间内的潜在重叠关系,从而高效完成交集相似性连接。此外,生成式模型可以辅助补全未观测轨迹,进一步增强相似性连接的准确性与鲁棒性。

7.2.0.0.3 时空大模型设计 未来可以构建一个统一的时空大模型框架,将轨迹数据分析、群体联合移动模式挖掘、持续密切接触查询以及轨迹相似度连接等任务整合在同一平台中,并提出接口用于用户交互。该框架可通过深度学习方法实现统一时空表示,将多源轨迹、行为和环境信息映射到低维潜在空间,从而支持个体轨迹预测、群体模式识别、实时接触检测和交集相似性计算。针对大规模和高频数据,可在分布式计算环境下进行流式处理与增量模型更新,实现高效并行计算与实时响应。同时,框架可提供统一接口与查询语义,便于不同应用场景快速接入,包括智能交通、公共卫生监控和生态保护等。结合可解释性和隐私保护技术,系统既能输出可理解的决策依据,又保障个体数据安全。整体来看,该统一时空大模型有望通过深度学习和大数据优化,实现跨任务、跨数据源的高效、实时和可扩展时空智能分析。

致 谢

参考文献

- [1] Chen L, Shang S, Yang C, et al. Spatial keyword search: a survey[J]. *GeoInformatica*, 2020, 24(1): 85–106.
- [2] Li K, Chen L, Shang S. Towards alleviating traffic congestion: Optimal route planning for massive-scale trips[C]. *IJCAI*, ijcai.org, 2020: 3400–3406.
- [3] Wilkie D, Baykal C, Lin M C. Participatory route planning[C]. *SIGSPATIAL/GIS*, ACM, 2014: 213–222.
- [4] Ding B, Yu J X, Qin L. Finding time-dependent shortest paths over large graphs[C]. *EDBT*, ACM, vol. 261 of ACM International Conference Proceeding Series, 2008: 205–216.
- [5] Kanoulas E, Du Y, Xia T, et al. Finding fastest paths on A road network with speed patterns[C]. *ICDE*, IEEE Computer Society, 2006: 10.
- [6] Wang Y, Li G, Tang N. Querying shortest paths on time dependent road networks[J]. *Proc. VLDB Endow.*, 2019, 12(11): 1249–1261.
- [7] Xu J, Guo L, Ding Z, et al. Traffic aware route planning in dynamic road networks[C]. *DASFAA* (1), Springer, vol. 7238 of Lecture Notes in Computer Science, 2012: 576–591.
- [8] Shang S, Liu J, Zheng K, et al. Planning unobstructed paths in traffic-aware spatial networks[J]. *GeoInformatica*, 2015, 19(4): 723–746.
- [9] Shang S, Guo D, Liu J, et al. Prediction-based unobstructed route planning[J]. *Neurocomputing*, 2016, 213: 147–154.
- [10] Wilkie D, van den Berg J P, Lin M C, et al. Self-aware traffic route planning[C]. *Proceedings of the Twenty-Fifth AAAI Conference on Artificial Intelligence*, AAAI 2011, San Francisco, California, USA, August 7-11, 2011, AAAI Press.
- [11] Dijkstra E W. A note on two problems in connexion with graphs[J]. *Numerische Mathematik*, 1959, 1: 269–271.
- [12] Sistla A P, Wolfson O, Chamberlain S, et al. Modeling and querying moving objects[C]. *ICDE*, 1997: 422–432.
- [13] Patel J M, Chen Y, Chakka V P. STRIPES: an efficient index for predicted trajectories[C]. *SIGMOD*, 2004: 637–646.
- [14] Saltenis S, Jensen C S, Leutenegger S T, et al. Indexing the positions of continuously moving objects[C]. *SIGMOD*, 2000: 331–342.
- [15] Ding Y, Zhou X, Wu G, et al. Mining spatio-temporal reachable regions with multiple sources over massive trajectory data[J]. *IEEE Trans. Knowl. Data Eng.*, 2021, 33(7): 2930–2942.

- [16] Bakalov P, Hadjieleftheriou M, Keogh E J, et al. Efficient trajectory joins using symbolic representations[C]. Chrysanthos P K, Samaras G, (Eds.) MDM, ACM, 2005: 86–93.
- [17] Yi B, Faloutsos C. Fast time sequence indexing for arbitrary lp norms[C]. VLDB, Morgan Kaufmann, 2000: 385–394.
- [18] Guttman A. R-trees: A dynamic index structure for spatial searching[C]. SIGMOD, 1984: 47–57.
- [19] Brinkhoff T, Kriegel H, Seeger B. Efficient processing of spatial joins using r-trees[C]. SIGMOD, 1993: 237–246.
- [20] Bentley J L. Multidimensional binary search trees used for associative searching[J]. Commun. ACM, 1975, (9): 509–517.
- [21] Zhang M, Chen S, Jensen C S, et al. Effectively indexing uncertain moving objects for predictive queries[J]. Proc. VLDB Endow., 2009, 2(1): 1198–1209.
- [22] Ciaccia P, Patella M, Rabitti F, et al. Indexing metric spaces with m-tree[C]. SEBD, 1997: 67–86.
- [23] Ciaccia P, Patella M, Zezula P. M-tree: An efficient access method for similarity search in metric spaces[C]. VLDB, 1997: 426–435.
- [24] Liu T, Moore A W, Gray A G. New algorithms for efficient high-dimensional nonparametric classification[J]. J. Mach. Learn. Res., 2006, 7: 1135–1158.
- [25] Omohundro S M. Five balltree construction algorithms[M]. International Computer Science Institute Berkeley, 1989.
- [26] Dolatshah M, Hadian A, Minaei-Bidgoli B. Ball*-tree: Efficient spatial indexing for constrained nearest-neighbor search in metric spaces[J]. CoRR, 2015, abs/1511.00628. 1511.00628.
- [27] Chao P, He D, Li L, et al. Efficient trajectory contact query processing[C]. DASFAA, 2021: 658–666.
- [28] Zhang X, Ray S, Shoeleh F, et al. Efficient contact similarity query over uncertain trajectories[C]. EDBT, 2021: 403–408.
- [29] Likhachev M, Ferguson D I, Gordon G J, et al. Anytime dynamic a*: An anytime, replanning algorithm[C]. ICAPS, AAAI, 2005: 262–271.
- [30] Cai X, Kloks T, Wong C K. Time-varying shortest path problems with constraints[J]. Networks, 1997, 29(3): 141–150.
- [31] Narváez P, Siu K, Tzeng H. New dynamic SPT algorithm based on a ball-and-string model[J]. IEEE/ACM Trans. Netw., 2001, 9(6): 706–718.
- [32] Lim S, Balakrishnan H, Gifford D, et al. Stochastic motion planning and applications to traffic[J]. Int. J. Robotics Res., 2011, 30(6): 699–712.

-
- [33] Yang B, Guo C, Jensen C S, et al. Stochastic skyline route planning under time-varying uncertainty[C]. ICDE, IEEE Computer Society, 2014: 136–147.
- [34] Pedersen S A, Yang B, Jensen C S. Anytime stochastic routing with hybrid learning[J]. Proc. VLDB Endow., 2020, 13(9): 1555–1567.
- [35] Hua M, Pei J. Probabilistic path queries in road networks: traffic uncertainty aware path selection[C]. EDBT, ACM, vol. 426 of ACM International Conference Proceeding Series, 2010: 347–358.
- [36] Malviya N, Madden S, Bhattacharya A. A continuous query system for dynamic route planning[C]. ICDE, IEEE Computer Society, 2011: 792–803.
- [37] Yang B, Guo C, Ma Y, et al. Toward personalized, context-aware routing[J]. VLDB J., 2015, 24(2): 297–318.
- [38] Guo C, Yang B, Hu J, et al. Context-aware, preference-based vehicle routing[J]. VLDB J., 2020, 29(5): 1149–1170.
- [39] Letchner J, Krumm J, Horvitz E. Trip router with individualized preferences (TRIP): incorporating personalization into route planning[C]. AAAI, AAAI Press, 2006: 1795–1800.
- [40] Balteanu A, Jossé G, Schubert M. Mining driving preferences in multi-cost networks[C]. SSTO, Springer, vol. 8098 of Lecture Notes in Computer Science, 2013: 74–91.
- [41] Ge Y, Xiong H, Tuzhilin A, et al. An energy-efficient mobile recommender system[C]. Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, July 25–28, 2010, ACM, 2010: 899–908.
- [42] Huang J, Huangfu X, Sun H, et al. Backward path growth for efficient mobile sequential recommendation[J]. TKDE, 2015, 27(1): 46–60.
- [43] Ye Z, Xiao K, Deng Y. A unified theory of the mobile sequential recommendation problem[C]. IEEE International Conference on Data Mining, ICDM 2018, Singapore, November 17–20, 2018, IEEE Computer Society, 2018: 1380–1385.
- [44] Ye Z, Xiao K, Ge Y, et al. Applying simulated annealing and parallel computing to the mobile sequential recommendation[J]. IEEE Trans. Knowl. Data Eng., 2019, 31(2): 243–256.
- [45] Ye Z, Zhang L, Xiao K, et al. Multi-user mobile sequential recommendation: An efficient parallel computing paradigm[C]. Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2018, London, UK, August 19–23, 2018, ACM, 2018: 2624–2633.
- [46] Chen Z, Shen H T, Zhou X, et al. Searching trajectories by locations: an efficiency study[C]. SIGMOD Conference, ACM, 2010: 255–266.

- [47] Zheng K, Shang S, Yuan N J, et al. Towards efficient search for activity trajectories[C]. Jensen C S, Jermaine C M, Zhou X, (Eds.) 29th IEEE International Conference on Data Engineering, ICDE 2013, Brisbane, Australia, April 8-12, 2013, IEEE Computer Society, 2013: 230–241.
- [48] Shang S, Chen L, Wei Z, et al. Collective travel planning in spatial networks[C]. ICDE, IEEE Computer Society, 2017: 59–60.
- [49] Li R, Qin L, Yu J X, et al. Optimal multi-meeting-point route search[J]. IEEE Trans. Knowl. Data Eng., 2016, 28(3): 770–784.
- [50] Reza R M, Ali M E, Cheema M A. The optimal route and stops for a group of users in a road network[C]. Proceedings of the 25th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems, GIS 2017, Redondo Beach, CA, USA, November 7-10, 2017, ACM, 2017: 4:1–4:10.
- [51] Hashem T, Hashem T, Ali M E, et al. Group trip planning queries in spatial databases[C]. Advances in Spatial and Temporal Databases - 13th International Symposium, SSTD 2013, Munich, Germany, August 21-23, 2013. Proceedings, Springer, vol. 8098 of Lecture Notes in Computer Science, 2013: 259–276.
- [52] Hashem T, Barua S, Ali M E, et al. Efficient computation of trips with friends and families[C]. Proceedings of the 24th ACM International Conference on Information and Knowledge Management, CIKM 2015, Melbourne, VIC, Australia, October 19 - 23, 2015, ACM, 2015: 931–940.
- [53] Lim S, Rus D. Stochastic distributed multi-agent planning and applications to traffic[C]. ICRA, IEEE, 2012: 2873–2879.
- [54] Zeng Y, Tong Y, Song Y, et al. The simpler the better: An indexing approach for shared-route planning queries[J]. Proc. VLDB Endow., 2020, 13(13): 3517–3530.
- [55] Zheng L, Chen L, Ye J. Order dispatch in price-aware ridesharing[J]. Proc. VLDB Endow., 2018, 11(8): 853–865.
- [56] Li F, Cheng D, Hadjieleftheriou M, et al. On trip planning queries in spatial databases[C]. SSTD, Springer, vol. 3633 of Lecture Notes in Computer Science, 2005: 273–290.
- [57] Sharifzadeh M, Kolahdouzan M R, Shahabi C. The optimal sequenced route query[J]. VLDB J., 2008, 17(4): 765–787.
- [58] Cao X, Chen L, Cong G, et al. Keyword-aware optimal route search[J]. Proc. VLDB Endow., 2012, 5(11): 1136–1147.
- [59] Yao B, Tang M, Li F. Multi-approximate-keyword routing in GIS data[C]. GIS, ACM, 2011: 201–210.

-
- [60] Zheng B, Su H, Hua W, et al. Efficient clue-based route search on road networks[J]. IEEE Trans. Knowl. Data Eng., 2017, 29(9): 1846–1859.
- [61] Wen Y T, Yeo J, Peng W, et al. Efficient keyword-aware representative travel route recommendation[J]. IEEE Trans. Knowl. Data Eng., 2017, 29(8): 1639–1652.
- [62] Li W, Cao J, Guan J, et al. Efficient retrieval of bounded-cost informative routes[C]. ICDE, IEEE Computer Society, 2018: 1811–1812.
- [63] Jossé G, Schmid K A, Schubert M. Probabilistic resource route queries with reappearance[C]. Proceedings of the 18th International Conference on Extending Database Technology, EDBT 2015, Brussels, Belgium, March 23-27, 2015, OpenProceedings.org, 2015: 445–456.
- [64] Zhang C, Liang H, Wang K, et al. Personalized trip recommendation with POI availability and uncertain traveling time[C]. Proceedings of the 24th ACM International Conference on Information and Knowledge Management, CIKM 2015, Melbourne, VIC, Australia, October 19 - 23, 2015, ACM, 2015: 911–920.
- [65] Yawalkar P, Ranu S. Route recommendations on road networks for arbitrary user preference functions[C]. ICDE, IEEE, 2019: 602–613.
- [66] Chen H, Ku W, Sun M, et al. The multi-rule partial sequenced route query[C]. GIS, ACM, 2008: 10.
- [67] Chen H, Ku W, Sun M, et al. The partial sequenced route query with traveling rules in road networks[J]. GeoInformatica, 2011, 15(3): 541–569.
- [68] Li J, Yang Y D, Mamoulis N. Optimal route queries with arbitrary order constraints[J]. IEEE Trans. Knowl. Data Eng., 2013, 25(5): 1097–1110.
- [69] Sharifzadeh M, Shahabi C. Processing optimal sequenced route queries using voronoi diagrams[J]. GeoInformatica, 2008, 12(4): 411–433.
- [70] Zeng Y, Chen X, Cao X, et al. Optimal route search with the coverage of users' preferences[C]. IJCAI, AAAI Press, 2015: 2118–2124.
- [71] Li Y, Yang W, Dan W, et al. Keyword-aware dominant route search for various user preferences[C]. Database Systems for Advanced Applications - 20th International Conference, DAS-FAA 2015, Hanoi, Vietnam, April 20-23, 2015, Proceedings, Part II, Springer, vol. 9050 of Lecture Notes in Computer Science, 2015: 207–222.
- [72] Kanza Y, Safra E, Sagiv Y, et al. Heuristic algorithms for route-search queries over geographical data[C]. GIS, ACM, 2008: 11.
- [73] Lu E H, Lin C, Tseng V S. Trip-mine: An efficient trip planning approach with travel time constraints[C]. Mobile Data Management (1), IEEE Computer Society, 2011: 152–161.

- [74] Kanza Y, Levin R, Safra E, et al. An interactive approach to route search[C]. GIS, ACM, 2009: 408–411.
- [75] Levin R, Kanza Y, Safra E, et al. Interactive route search in the presence of order constraints[J]. Proc. VLDB Endow., 2010, 3(1): 117–128.
- [76] Roy S B, Das G, Amer-Yahia S, et al. Interactive itinerary planning[C]. ICDE, IEEE Computer Society, 2011: 15–26.
- [77] Fan L, Bonomi L, Shahabi C, et al. Multi-user itinerary planning for optimal group preference[C]. Advances in Spatial and Temporal Databases - 15th International Symposium, SSTD 2017, Arlington, VA, USA, August 21-23, 2017, Proceedings, Springer, vol. 10411 of Lecture Notes in Computer Science, 2017: 3–23.
- [78] Fan L, Bonomi L, Shahabi C, et al. Optimal group route query: Finding itinerary for group of users in spatial databases[J]. GeoInformatica, 2018, 22(4): 845–867.
- [79] Levin R, Kanza Y. TARS: traffic-aware route search[J]. GeoInformatica, 2014, 18(3): 461–500.
- [80] Salgado C, Cheema M A, Taniar D. An efficient approximation algorithm for multi-criteria indoor route planning queries[C]. SIGSPATIAL/GIS, ACM, 2018: 448–451.
- [81] Feng Z, Liu T, Li H, et al. Indoor top-k keyword-aware routing query[C]. ICDE, IEEE, 2020: 1213–1224.
- [82] Agrawal R, Faloutsos C, Swami A N. Efficient similarity search in sequence databases[C]. FODO, Springer, vol. 730 of Lecture Notes in Computer Science, 1993: 69–84.
- [83] Yi B, Jagadish H V, Faloutsos C. Efficient retrieval of similar time sequences under time warping[C]. Proceedings of the Fourteenth International Conference on Data Engineering, Orlando, Florida, USA, February 23-27, 1998, IEEE Computer Society, 1998: 201–208.
- [84] Vlachos M, Gunopulos D, Kollios G. Discovering similar multidimensional trajectories[C]. Proceedings of the 18th International Conference on Data Engineering, San Jose, CA, USA, February 26 - March 1, 2002, IEEE Computer Society, 2002: 673–684.
- [85] Chen L, Ng R T. On the marriage of lp-norms and edit distance[C]. (e)Proceedings of the Thirtieth International Conference on Very Large Data Bases, VLDB 2004, Toronto, Canada, August 31 - September 3 2004, Morgan Kaufmann, 2004: 792–803.
- [86] Chen L, Özsu M T, Oria V. Robust and fast similarity search for moving object trajectories[C]. Proceedings of the ACM SIGMOD International Conference on Management of Data, Baltimore, Maryland, USA, June 14-16, 2005, ACM, 2005: 491–502.
- [87] Ranu S, P D, Telang A D, et al. Indexing and matching trajectories under inconsistent sampling rates[C]. ICDE, IEEE Computer Society, 2015: 999–1010.

-
- [88] Su H, Zheng K, Wang H, et al. Calibrating trajectory data for similarity-based analysis[C]. SIGMOD, ACM, 2013: 833–844.
- [89] Perng C, Wang H, Zhang S R, et al. Landmarks: a new model for similarity-based pattern querying in time series databases[C]. Proceedings of the 16th International Conference on Data Engineering, San Diego, California, USA, February 28 - March 3, 2000, IEEE Computer Society, 2000: 33–42.
- [90] Chen Y, Nascimento M A, Ooi B C, et al. Spade: On shape-based pattern detection in streaming time series[C]. Chirkova R, Dogac A, Özsu M T, et al., (Eds.) Proceedings of the 23rd International Conference on Data Engineering, ICDE 2007, The Marmara Hotel, Istanbul, Turkey, April 15-20, 2007, IEEE Computer Society, 2007: 786–795.
- [91] Li X, Zhao K, Cong G, et al. Deep representation learning for trajectory similarity computation[C]. ICDE, IEEE Computer Society, 2018: 617–628.
- [92] Yao D, Cong G, Zhang C, et al. Computing trajectory similarity in linear time: A generic seed-guided neural metric learning approach[C]. ICDE, IEEE, 2019: 1358–1369.
- [93] Zhou S, Shang S, Chen L, et al. RED: effective trajectory representation learning with comprehensive information[J]. Proc. VLDB Endow., 2024, 18(2): 80–92.
- [94] Shang S, Chen L, Wei Z, et al. Parallel trajectory similarity joins in spatial networks[J]. VLDB J., 2018, 27(3): 395–420.
- [95] Chen Y, Patel J M. Design and evaluation of trajectory join algorithms[C]. 17th ACM SIGSPATIAL International Symposium on Advances in Geographic Information Systems, Washington, USA: ACM, 2009: 266–275.
- [96] Tampakis P, Doulkeridis C, Pelekis N, et al. Distributed subtrajectory join on massive datasets[J]. ACM Trans. Spatial Algorithms Syst., 2020, 6(2): 8:1–8:29.
- [97] Yuan H, Li G. Distributed in-memory trajectory similarity search and join on road network[C]. ICDE, IEEE, 2019: 1262–1273.
- [98] Chen L, Shang S, Jensen C S, et al. Parallel semantic trajectory similarity join[C]. ICDE, IEEE, 2020: 997–1008.
- [99] Pan Z, Chao P, Fang J, et al. Trasp: A general framework for online trajectory similarity processing[C]. WISE, Springer, 2020: 384–397.
- [100] Atev S, Miller G, Papanikolopoulos N. Clustering of vehicle trajectories[J]. IEEE Transactions on Intelligent Transportation Systems, 2010, 11: 647–657.
- [101] Fang Z, Gong S, Chen L, et al. Ghost: A general framework for high-performance online similarity queries over distributed trajectory streams[C]. Proceedings of the ACM on Management of Data, vol. 1, 2023: 1 - 25.

- [102] Ding Z, Li K, Chen L, et al. Parallel online similarity join over trajectory streams[C]. Proceedings of the ACM on Web Conference 2025, 2025: 3426–3437.
- [103] Jeung H, Yiu M L, Zhou X, et al. Discovery of convoys in trajectory databases[J]. Proc. VLDB Endow., 2008, 1(1): 1068–1080.
- [104] Li Z, Ding B, Han J, et al. Swarm: Mining relaxed temporal moving object clusters[J]. Proc. VLDB Endow., 2010, 3(1): 723–734.
- [105] Tang L, Zheng Y, Yuan J, et al. On discovery of traveling companions from streaming trajectories[C]. IEEE 28th International Conference on Data Engineering (ICDE 2012), IEEE Computer Society, 2012: 186–197.
- [106] Zheng K, Zheng Y, Yuan N J, et al. On discovery of gathering patterns from trajectories[C]. ICDE, IEEE Computer Society, 2013: 242–253.
- [107] Vieira M R, Bakalov P, Tsotras V J. On-line discovery of flock patterns in spatio-temporal data[C]. GIS, ACM, 2009: 286–295.
- [108] Tanaka P S, Vieira M R, Kaster D S. An improved base algorithm for online discovery of flock patterns in trajectories[J]. J. Inf. Data Manag., 2016, 7(1): 52–67.
- [109] Naserian E, Wang X, Xu X, et al. Discovery of loose travelling companion patterns from human trajectories[C]. HPCC, IEEE Computer Society, 2016: 1238–1245.
- [110] Shein T T, Puntheeranurak S, Imamura M. Discovery of loose group companion from trajectory data streams[J]. IEEE Access, 2020, 8: 85856–85868.
- [111] Wang Y, Lim E, Hwang S. Efficient mining of group patterns from user movement data[J]. Data Knowl. Eng., 2006, 57(3): 240–282.
- [112] Li Y, Bailey J, Kulik L. Efficient mining of platoon patterns in trajectory databases[J]. Data Knowl. Eng., 2015, 100: 167–187.
- [113] Fang Z, Gao Y, Pan L, et al. Coming: A real-time co-movement mining system for streaming trajectories[C]. Maier D, Pottinger R, Doan A, et al., (Eds.) SIGMOD, ACM, 2020: 2777–2780.
- [114] Chen L, Gao Y, Fang Z, et al. Real-time distributed co-movement pattern detection on streaming trajectories[J]. Proc. VLDB Endow., 2019, 12(10): 1208–1220.
- [115] Liu Y, Zhao K, Cong G, et al. Online anomalous trajectory detection with deep generative sequence modeling[C]. ICDE, IEEE, 2020: 949–960.
- [116] Zhao Y, Chen Q, Cao W, et al. Deep learning for risk detection and trajectory tracking at construction sites[J]. IEEE Access, 2019, 7: 30905–30912.
- [117] Eusuf S S, Islam K A, Ali M E, et al. A web-based system for efficient contact tracing query in a large spatio-temporal database[C]. SIGSPATIAL '20: 28th International Conference on Advances in Geographic Information Systems, ACM, 2020: 473–476.

-
- [118] Xu J, Lu H, Bao Z. IMO: A toolbox for simulating and querying "infected" moving objects[J]. *Proc. VLDB Endow.*, 2020, 13(12): 2825–2828.
- [119] He H, Li R, Wang R, et al. Efficient suspected infected crowds detection based on spatio-temporal trajectories[J]. *CoRR*, 2020, abs/2004.06653. 2004.06653.
- [120] Alarabi L, Basalamah S M, Hendawi A M, et al. Traceall: A real-time processing for contact tracing using indoor trajectories[J]. *Inf.*, 2021, 12(5): 202.
- [121] Zhang Y, Liu A, Liu G, et al. Deep representation learning of activity trajectory similarity computation[C]. *ICWS, IEEE*, 2019: 312–319.
- [122] Yao D, Zhang C, Zhu Z, et al. Trajectory clustering via deep representation learning[C]. *IJCNN, IEEE*, 2017: 3880–3887.
- [123] Boyle J, Nawaz T, Ferryman J M. Deep trajectory representation-based clustering for motion pattern extraction in videos[C]. *AVSS, IEEE Computer Society*, 2017: 1–6.
- [124] Fang Z, Du Y, Chen L, et al. E²dtc: An end to end deep trajectory clustering framework via self-training[C]. *ICDE, IEEE*, 2021: 696–707.
- [125] Xu Y, Chen L, Yao B, et al. Location-based top-k term querying over sliding window[C]. *WISE (1), Springer*, vol. 10569 of *Lecture Notes in Computer Science*, 2017: 299–314.
- [126] Shang S, Chen L, Wei Z, et al. Collective travel planning in spatial networks[J]. *TKDE*, 2016, 28(5): 1132–1146.
- [127] Guo C, Yang B, Hu J, et al. Learning to route with sparse trajectory sets[C]. *34th IEEE International Conference on Data Engineering, ICDE 2018, Paris, France, April 16-19, 2018, IEEE Computer Society*, 2018: 1073–1084.
- [128] Chen L, Shang S, Jensen C S, et al. Effective and efficient reuse of past travel behavior for route recommendation[C]. *KDD, ACM*, 2019: 488–498.
- [129] Chen L, Shang S, Guo T. Real-time route search by locations[C]. *AAAI, AAAI Press*, 2020: 574–581.
- [130] Chen L, Shang S, Yao B, et al. Pay your trip for traffic congestion: Dynamic pricing in traffic-aware road networks[C]. *AAAI, AAAI Press*, 2020: 582–589.
- [131] Shang S, Ding R, Zheng K, et al. Personalized trajectory matching in spatial networks[J]. *VLDB J.*, 2014, 23(3): 449–468.
- [132] Dafermos, C S. The traffic assignment problem for multiclass-user transportation networks[J]. *Transportation Science*, 1972, 6(1): 73–87.
- [133] Javani B, Babazadeh A, Ceder A. Path-based capacity-restrained dynamic traffic assignment algorithm[J]. *Transportmetrica B: Transport Dynamics*, 2019, 7(1): 741–764.

- [134] Zheng K, Zheng Y, Yuan N J, et al. Online discovery of gathering patterns over trajectories[J]. TKDE, 2014, 26(8): 1974–1988.
- [135] Xu X, Yuruk N, Feng Z, et al. SCAN: a structural clustering algorithm for networks[C]. SIGKDD, ACM, 2007: 824–833.
- [136] Li K, Chen L, Shang S, et al. Towards controlling the transmission of diseases: Continuous exposure discovery over massive-scale moving objects[C]. IJCAI, 2022: 3891–3897.
- [137] Pelekis N, Kopanakis I, Kotsifakos E E, et al. Clustering trajectories of moving objects in an uncertain world[C]. ICDM, IEEE Computer Society, 2009: 417–427.
- [138] Lee J, Han J, Whang K. Trajectory clustering: a partition-and-group framework[C]. SIGKDD, ACM, 2007: 593–604.
- [139] Li Z, Lee J, Li X, et al. Incremental clustering for trajectories[C]. DASFAA, Springer, vol. 5982 of Lecture Notes in Computer Science, 2010: 32–46.
- [140] Li H, Liu J, Wu K, et al. Spatio-temporal vessel trajectory clustering based on data mapping and density[J]. IEEE Access, 2018, 6: 58939–58954.
- [141] Yuan J, Zheng Y, Xie X, et al. Driving with knowledge from the physical world[C]. Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, 2011: 316–324.
- [142] Yuan J, Zheng Y, Zhang C, et al. T-drive: driving directions based on taxi trajectories[C]. ACM-GIS, ACM, 2010: 99–108.
- [143] Chen L, Shang S, Feng S, et al. Parallel subtrajectory alignment over massive-scale trajectory data[C]. Zhou Z, (Ed.) IJCAI, ijcai.org, 2021: 3613–3619.
- [144] Shang S, Chen L, Zheng K, et al. Parallel trajectory-to-location join[J]. TKDE, 2019, 31(6): 1194–1207.
- [145] Xiong L, Shahabi C, Da Y, et al. REACT: real-time contact tracing and risk monitoring using privacy-enhanced mobile tracking[J]. ACM SIGSPATIAL Special, 2020, 12(2): 3–14.
- [146] Faloutsos C, Ranganathan M, Manolopoulos Y. Fast subsequence matching in time-series databases[C]. Proceedings of the 1994 ACM SIGMOD International Conference on Management of Data, ACM Press, 1994: 419–429.
- [147] Assent I, Wichterich M, Krieger R, et al. Anticipatory DTW for efficient similarity search in time series databases[J]. Proc. VLDB Endow., 2009, 2(1): 826–837.
- [148] Rong C, Lin C, Silva Y N, et al. Fast and scalable distributed set similarity joins for big data analytics[C]. 33rd IEEE International Conference on Data Engineering, ICDE 2017, IEEE Computer Society, 2017: 1059–1070.

-
- [149] Xie D, Li F, Phillips J M. Distributed trajectory similarity search[J]. *Proc. VLDB Endow.*, 2017, 10(11): 1478–1489.
- [150] Ojagh S, Saeedi S, Liang S H L. A person-to-person and person-to-place COVID-19 contact tracing system based on OGC indoorsml[J]. *ISPRS Int. J. Geo Inf.*, 2021, 10(1): 2.
- [151] Taha A A, Hanbury A. An efficient algorithm for calculating the exact hausdorff distance[J]. *IEEE Trans. Pattern Anal. Mach. Intell.*, 2015, 37(11): 2153–2163.
- [152] Kong X, Chen Q, Hou M, et al. Mobility trajectory generation: a survey[J]. *Artif. Intell. Rev.*, 2023, 56(Supplement 3): 3057–3098.
- [153] Zhang R, Lin D, Ramamohanarao K, et al. Continuous intersection joins over moving objects[C]. *ICDE*, 2008: 863–872.
- [154] Zhang R, Qi J, Lin D, et al. A highly optimized algorithm for continuous intersection join queries over moving objects[J]. *VLDB J.*, 2012, 21(4): 561–586.
- [155] Vishnu C, Abhinav V, Roy D, et al. Improving multi-agent trajectory prediction using traffic states on interactive driving scenarios[J]. *IEEE Robotics Autom. Lett.*, 2023, 8(5): 2708–2715.
- [156] Forrest S, Recio M, Seddon P. Moving wildlife tracking forward under forested conditions with the swift gps algorithm[J]. *Animal Biotelemetry*, 2022, 10(1): 19.
- [157] Pfoser D, Jensen C S. Capturing the uncertainty of moving-object representations[C]. *SSD*, 1999: 111–132.
- [158] Trajcevski G, Choudhary A, Wolfson O, et al. Uncertain range queries for necklaces[C]. *MDM*, 2010: 199–208.
- [159] Moore A W. The anchors hierarchy: Using the triangle inequality to survive high dimensional data[C]. *UAI*, 2000: 397–405.
- [160] Samet H. The quadtree and related hierarchical data structures[J]. *ACM Comput. Surv.*, 1984, 16(2): 187–260.
- [161] Zheng Y, Li Q, Chen Y, et al. Understanding mobility based on GPS data[C]. *UbiComp*, 2008: 312–321.
- [162] Zheng Y, Xie X, Ma W. Geolife: A collaborative social networking service among user, location and trajectory[J]. *IEEE Data Eng. Bull.*, 2010, 33(2): 32–39.
- [163] Moreira-Matias L, Gama J, Ferreira M, et al. Predicting taxi-passenger demand using streaming data[J]. *IEEE Trans. Intell. Transp. Syst.*, 2013, 14(3): 1393–1402.

攻读博士学位期间取得的成果

发表论文：

- [1] **Li Ke**, Chen Lisi, Shang Shuo. Towards alleviating traffic congestion: Optimal route planning for massive-scale trips. International Joint Conferences on Artificial Intelligence, 2021: 3400-3406.
- [2] **Li Ke**, Chen Lisi, Shang Shuo, Panos Kalnis, Bin Yao. Traffic congestion alleviation over dynamic road networks: Continuous optimal route combination for trip query streams. International Joint Conferences on Artificial Intelligence, 2021, 62(10): 5344–5348.
- [3] **Li Ke**, Rao Xuan, Pang Xiaobing, Chen Lisi, Fan Siqu. Route search and planning: A survey. Big data research, 2021, 26: 100246.
- [4] Chen Ziwen, **Li Ke**, Zhou Silin, Chen Lisi, Shang Shuo. Towards robust trajectory similarity computation: Representation-based spatio-temporal similarity quantification. World Wide Web, 2023, 26(4): 1271-1294.
- [5] **Li Ke**, Chen Lisi, Shang Shuo, Wang Haiyan, Liu Yang, Panos Kalnis, Bin Yao. Towards Controlling the Transmission of Diseases: Continuous Exposure Discovery over Massive-Scale Moving Objects. International Joint Conferences on Artificial Intelligence, 2022: 3891-3897.
- [6] **Li Ke**, Wang Hongyu, Chen Ziwen, Chen Lisi. Relaxed group pattern detection over massive-scale trajectories. Future Generation Computer Systems, 2023, 144: 131-139.
- [7] Wang Chenhao, **Li Ke**, Chen Lisi. Deep unified attention-based sequence modeling for online anomalous trajectory detection. Future Generation Computer Systems, 2023, 144: 1-11.
- [8] Wang Haiyan, Feng Jiaming, **Li Ke**, Chen Lisi. Deep understanding of big geospatial data for self-driving: Data, technologies, and systems. Future Generation Computer Systems, 2022, 137: 146-163.
- [9] Ding Zhongjun, **Li Ke**, Chen Lisi, Shang Shuo. Parallel online similarity join over trajectory streams. Proceedings of the ACM on Web Conference, 2025: 3426-3437.
- [10] Wang Hongyu, **Li Ke**, Shang Shuo. DLRD: dual-level network for rumor detection on geo-textual data. GeoInformatica, 2024, 28 (2): 335-351.

- [11] Chen Ziwen, **Li Ke**, Chen Lisi, Hu Nan, Yoshiharu Ishikawa. Co-movement aware trajectory generation via waypoint-guided generative adversarial networks. *GeoInformatica*, 2025, 29 (4): 1093-1119.
- [12] Chen Ziwen, **Li Ke**, Yoshiharu Ishikawa. Ca-gen: Trajectory generation with co-movement awareness. *Proceedings of the 19th International Symposium on Spatial and Temporal Data*, 2025: 115-118.
- [13] **Li Ke**, Leong Hou U, Shang Shuo. App2Exa: accelerating exact kNN search via dynamic cache-guided approximation. *International Joint Conferences on Artificial Intelligence*, 2025: 3045-3053.
- [14] **Li Ke**, Chen Lisi, Shang Shuo, Christian S Jensen, Panos Kalnis. Beyond Locations: A Motion Range-Aware Similarity Join. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, 2025: 1424-1434.

发明专利：

参与项目：